

Componentes/Cashier/STWCcloseCash.prw

188 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STWCLOSECASH.CH"
4
5 //-----
6 /*{Protheus.doc} STWPOSCloseCash
7 Realiza Fechamento do Caixa
8
9 @param
10 @return lRet   Retorna se fechou o caixa
11 @obs
12 @sample
13 */
14 //-----
15 Function STWPOSCloseCash()
16
17 Local lUsaDisplay   := !Empty(STFGetStation("DISPLAY")) // Verifica se a estacao possui Display
18 Local cSupervisor   := Space(15)                       // Supervisor que autorizou a transacao
19 Local lRet          := .T.                             // Retorno
20 Local cStation      := STFGetStation("CODIGO")          // Estacao atual
21 Local lLjConff      := SuperGetMV("MV_LJCONFF",.T.,.F.) // Fechamento de caixa
22 Local lObgFecha     := SuperGetMv("MV_LJOBGCF",,.T.)    // Obrigatorio o fechamento de caixa.
23 Local lMobile       := STFGetCfg("lMobile", .F.)        // Smart Client Mobile
24
25 lMobile := ValType(lMobile) == "L" .AND. lMobile
26
27 // Se o caixa estiver aberto, sem vendas pendentes e nao estiver realizando recebimentos
28 If lRet .And. STBOpenCash()
29
30   If lUsaDisplay
31     STFFireEvent(ProcName(0), "STDisplay", { StatDisplay(), "1C"+ STR0001 } ) // "FECHAMENTO DO CAIXA"
32   EndIf
33
34   // Fechamento do caixa
35   lObgFecha := lObgFecha .Or. !IsInCallStack("STFExit") //tratamento para fechamento de caixa nao ser obrigatorio
36   If (!lLjConff .Or. lObgFecha) .And. STWCcloseCash( @cSupervisor )
37
38     // Grava que foi realizado fechamento na tabela de Monitoramento das Estacoes(SLI)
39     STFSLICreate(cStation, "OPE", "", "SOBREPOE")
40
41     //Em versoes Mobile nao faz controle por arquivo
42     //fisico apenas por registros em Base de dados
43     If lMobile
44       //Desabilita os botoes do PDV
45       STIBtnDeActivate()
46     Else
47       // Fecha o arquivo semaforo que esta aberto
48       FClose( STBGetCXHnd() )
49     EndIf
50
51     //Caixa Fechado
52     lRet := .T.

```

```

53
54     ElseIf (!lLjConff .Or. lObgFecha)
55         lRet := .F. // usuario sem acesso a fechar o caixa
56     EndIf
57
58 EndIf
59
60 Return lRet
61
62
63 //-----
64 /*{Protheus.doc} STWCloseCash
65 Realiza Fechamento do caixa
66
67 @param  cSupervisor por referencia
68 @return lCloseCash   Retorna se o caixa esta aberto
69 @obs
70 @sample
71 */
72 //-----
73 Function STWCloseCash(cSupervisor)
74
75 Local lRet           := .F.           // Retorno
76 Local aDtHr          := STBDtCash()   // Array data e hora
77 Local cCupomFim       := ""           // Numeracao final da venda
78 Local cCash          := xNumCaixa()    // Caixa atual
79 Local aPermissao      := {}           // Verifica permissao
80 Local aRet            := {}           // Array de retorno
81 Local aDados          := {}           // Array de informacoes do evento
82 Local lEmitNfce       := Iif(ExistFunc("LjEmitNfCe"), LjEmitNfCe(), .F.) // Sinaliza se utiliza NFC-e
83
84 Default cSupervisor   := Space(15)     // Supervisor
85
86 ParamType 0 Var       cSupervisor As Character Default Space(15)
87
88 // Verifica autorizacao de usuario e caixa para fechar o caixa
89 aPermissao := STBAutCloseCash()
90 cSupervisor := aPermissao[2]
91
92 If aPermissao[1]
93
94     // Fechamento do caixa
95     lRet := STDCloseCash( cCash      , aDtHr[1] , aDtHr[2] )
96
97     If lRet
98
99         If STFGetCfg("lMultCoin") // Se usa Multimoeda
100             // Fecha caixas associados(tratamento multi-moeda)
101             lRet := STDCloseMult( cCash , aDtHr[1] , aDtHr[2] )
102         EndIf
103
104     If lUsaDisplay

```

```

105     DisplayEnv(StatDisplay(), "1C"+ STR0002 ) // CAIXA FECHADO
106     DisplayEnv(StatDisplay(), "2C"+ " " )
107     DisplayEnv(StatDisplay(), '0C'+ STR0002 ) // CAIXA FECHADO
108 EndIf
109
110 If lRet
111     // Caixa tem acesso a utilizar Impressora Fiscal
112     If STFProFile(3)[1]
113         If !lEmitNfce
114             // Pega numero do cupom
115             cCupomFim := space(STBLenReceipt()) //Numero do Cupom
116
117             aDados := {cCupomFim, Nil}
118             aRet := STFFireEvent( ProcName(0) ,; // Nome do processo
119                                 "STGetReceipt" ,; // Nome do evento
120                                 aDados )
121
122             If Len(aRet) == 0 .OR. aRet[1] <> 0 .OR. Len(aDados) == 0
123                 STFShowMessage( STR0003, "STGetReceipt" ) //"Falha na obtenção do cupom"
124             Else
125                 cCupomFim := aDados[1]
126             EndIf
127         Else
128             //Captura o último documento a partir das vendas
129             cCupomFim := STDCSLastSale("L1_DOC")
130         EndIf
131     EndIf
132
133     // Grava Movimento Processos de Venda(SLW)
134     lRet := STDCSaveMov( aDtHr , cCupomFim)
135
136 EndIf
137
138
139 EndIf
140
141 If !lRet
142     STFMessage("STCloseCash","STOP", STR0004) //"Atencao, Não foi possivel fechar o caixa!"
143 Else
144     STFMessage("STCloseCash","STOP", STR0002)
145 EndIf
146 Else
147     STFMessage("STCloseCash","STOP", STR0008) //"Usuario sem permissão para encerrar o caixa!"
148 EndIf
149
150 STFShowMessage("STCloseCash")
151
152 Return lRet
153
154 //-----
155 /*{Protheus.doc} STWCash
156 Realiza Fechamento do Caixa

```

```

157
158 @param
159 @return lRet   Retorna se fechou o caixa
160 @obs
161 @sample
162 */
163 //-----
164 Function STWCash()
165
166 Local llJConff := SuperGetMV("MV_LJCONFF",.T.,.F.) //Fechamento de caixa
167 Local oTotal  := STFGetTot() // Recebe o Objeto totalizador
168 Local nTotSale := oTotal:GetValue( "L1_VLRTOT" ) //Total da venda que esta em andamento
169
170 If nTotSale > 0
171     STFMessage("STCloseCash","STOP", STR0005 ) //"Não é possível Fechar o Caixa, venda já iniciada. Fechar ou cancelar a venda."
172 Else
173     If STBOpenCash()
174         If llJConff
175             // Se usa conferencia de caixa
176             If STWConfCash("2")
177                 STIConfCash()
178             EndIf
179         Else
180             If MsgNoYes(STR0006,STR0007) //"Deseja realmente fechar o caixa?" # "Atenção"
181                 STWPOSCloseCash()
182                 STIExchangePanel( { || STIPanOpenCash() } )
183             EndIf
184         EndIf
185     EndIf
186 EndIf
187
188 Return .T.

```

Componentes/Cashier/STWConfCash.prw

39 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STWCONFCASH.CH"
4
5 //-----
6 /*{Protheus.doc} STWConfCash
7 Realiza a conferencia de caixa
8 @param cTpOpCl - 1 para abertura de caixa, 2 para fechamento de caixa
9 @return lRet - Retorna se realizou a conferencia
10 @obs
11 @sample
12 */
13 //-----
14 Function STWConfCash(cTpOpCl)
15
16 Local lConfCaixa := SuperGetMV( "MV_LJCONFF",,.F. ) //Parametro da conferencia de caixa
17 Local lRet := .T. //Retorno da funcao
18
19 Default cTpOpCl := ""
20
21 ParamType 1 Var cTpOpCl As Character Default ""
22
23 If lConfCaixa
24     If cTpOpCl == "1"
25         If Empty(STBValOpCash())
26             lRet := .T.
27         Else
28             STFMessage("STWConfCash","STOP",STR0001)
29             STFSHOWMESSAGE("STWConfCash")
30             lRet := .F.
31         EndIf
32     EndIf
33 Else
34     If cTpOpCl == "2"
35         lRet := .F.
36     EndIf
37 EndIf
38
39 Return lRet

```

Componentes/Cashier/STWOpenCash.prw

315 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STPOS.CH"
4 #INCLUDE "STWOPENCASH.CH"
5
6 Static lOpenPDV := .F. //Variavel de controle, indica se é uma abertura de caixa para a rotina de suprimento
7
8 // CAIXA_FECHADO          1 // Caixa foi fechado pelo operador
9 // TROCOU_OPERADOR       2 // Houve troca de operador
10 // ULTIMO_OPERADOR       3 // Ultimo operador que usou o sistema e nao fechou caixa
11
12 //-----
13 /*{Protheus.doc} STWPOSOpenCash
14 Function abertura de caixa
15
16 @param
17 @return lOpenCash Retorna se o caixa esta aberto
18 @obs
19 @sample
20 */
21 //-----
22 Function STWPOSOpenCash()
23
24 Local lUsaDisplay      := !Empty(STFGetStation("DISPLAY")) // Verifica se a estacao possui Display
25 Local lOpenCash       := STBOpenCash() // Caixa esta aberto
26 Local aStation        := {} // Array que guarda status da estacao operdores etc
27 Local cStation        := STFGetStation("CODIGO") // Estacao atual
28 Local cNumMov         := AllTrim(STDNumMov()) // Numero da movimento
29 Local cSupervisor     := Space(15) // Supervisor que autorizou a transacao
30 Local dDataMovECF     := dDataBase // Data do movimento do ECF
31 Local cReduction      := ""
32 Local nAtAux          := 0
33 Local aReduction      := {}
34 Local lHomolPaf       := STBHomolPaf()
35 Local cMV_LJGRMR      := IIF(STBIsPAF(SM0->M0_CGC), "S", SuperGetMv("MV_LJGRMR", , "S"))
36 /*
37 Verifica status da estacao e operador
38 */
39
40 aStation := STDStStation( cStation , cUserName )
41
42 //Se o caixa esta fechado
43 If !lOpenCash .AND. STWConfCash("1")
44
45     If lUsaDisplay
46         // Inicia Evento
47         STFFireEvent(ProcName(0), "STDdisplay", { StatDisplay(), "1C" + STR0001 } ) //"A B E R T U R A   D E   C A I X A"
48     EndIf
49     //UAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA;
50     // verifica se houve troca do operador sem que houvesse o fechamento do caixa anterior ,
51     // caso ocorra não prossegue com a operacao³
52     //AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAU

```

```

53 If !Empty(aStation) .AND. !aStation[CAIXA_FECHADO] .AND. aStation[TROCOU_OPERADOR]
54   LjGrvLog( "Abertura de caixa", "[STWPOSOpenCash] - " + STR0005 + " " + AllTrim(aStation[ULTIMO_OPERADOR]) + " " + STR0006 ) //Gera LOG
55   STFMessage("OpenCash","STOP", STR0005 + " " + AllTrim(aStation[ULTIMO_OPERADOR]) + " " + ;
56     STR0006) //"Atencao, 0 usuario nao fechou o caixa. Favor iniciar com este usuario e fechar o caixa."
57   lOpenCash := .F.
58 elseif STWOpenCash( @cSupervisor      )
59   lOpenCash := .T.
60
61   // Grava que foi realizada abertura de caixa, na tabela de Monitoramento das Estacoes(SLI)
62   STFSLICreate(cEstacao, "OPE", If(Empty(cSupervisor),cUserName,cSupervisor), "SOBREPOE")
63
64   //Se nao conseguiu criar arquivo de semaforo
65   If !STBTraffic("OPE", cEstacao)
66
67     STFMessage("OpenCash","STOP", STR0002 + ;
68       STR0003)          //"Atencao, Nao foi possivel inicializar o semaforo de Abertura de Caixa."
69   EndIf                                     //"Verifique se nao existe outra conexao do Front Loja."
70
71   If lUsaDisplay
72     // Inicia Evento
73     STFFireEvent(ProcName(0), "STDdisplay", { StatDisplay(), "2E"+ STR0004 } ) // Exibir Mensagem "Codigo do Produto: " Significa que esta pronto p venda
74     STFFireEvent(ProcName(0), "STDdisplay", { StatDisplay(), "1E"+ " " } )
75   EndIf
76
77   EndIf
78
79 // Se houve troca de operador add msg
80 ElseIf lOpenCash .AND. aStation[TROCOU_OPERADOR]
81   STFMessage("OpenCash","STOP", STR0005 + " " + AllTrim(aStation[ULTIMO_OPERADOR]) + " " + ;
82     STR0006) //"Atencao, 0 usuario nao fechou o caixa. Favor iniciar com este usuario e fechar o caixa."
83 EndIf
84
85 If !lOpenCash
86   STFMessage("OpenCash","STOP", STR0007) //"Atencao, Não foi possivel abrir o caixa!"
87 Else
88
89   // Se for o primeiro movimento do dia e usar PafECF
90   // Cancela orcamentos vencidos
91   If STFGetCfg("lPafEcfc")
92
93     cNumMov := STDNumMov()
94     If Val(cNumMov) == 1
95       If lHomolPaf
96         cReduction := ""
97         cReduction := cMV_LJGRMR + space(800) //PROBLEMAS NO TIMEOUT, CAPTURA OS DADOS DA REDUCAO
98         aDados := {cReduction}
99         aRet := STFFireEvent( ProcName(0)                                     ,;      // Nome do processo
100                               "STRedZData"                                   ,;      // Nome do evento
101                               aDados)
102
103         If Len(aRet) > 0 .AND. aRet[1] == 0 .AND. Len(aDados) > 0
104           cReduction := aDados[1]

```

```

105         Else
106             cReduction := ""
107         EndIf
108
109         While Len(cReduction) > 0
110             nAtAux := At("|", cReduction)
111             If nAtAux > 0
112                 AAdd(aReduction, SUBS(cReduction,1,nAtAux-1) )
113                 cReduction := SUBS(cReduction, nAtAux + 1, LEN(cReduction) )
114             Else
115                 cReduction := ""
116             Endif
117         End
118
119
120         If Len(aReduction) >= 14 .AND. !Empty(aReduction[14])
121             dDataMovECF := CTOD(aReduction[01])
122         EndIf
123     Else
124         dDataMovECF := STDZLastRZDt()
125     EndIf
126
127     STWCancOrc( dDataMovECF )
128 EndIf
129
130 EndIf
131
132 STFMessage("OpenCash","STOP", STR0008) //"Caixa Aberto!"
133
134 //Seta variavel statica para nao permitir sair do sistema sem realizar a conferencia
135 If FindFunction("STFSetPerExit")
136     STFSetPerExit(.F.)
137 EndIf
138
139 EndIf
140
141 STFShowMessage("OpenCash")
142
143 Return lOpenCash
144
145
146 //-----
147 /*{Protheus.doc} STWOpenCash
148 Realiza Abertura do Caixa
149
150 @param cSupervisor Supervisor por referencia
151 @return lRet Retorna se o caixa foi aberto
152 @obs
153 @sample
154 */
155 //-----
156 Function STWOpenCash( cSupervisor )

```

```

157
158 Local dData          := dDataBase          // Data
159 Local aDtHr          := STBDtCash()        // Array data e hora
160 Local lRet           := .F.                // Retorna se o caixa foi aberto
161 Local aRet           := {}                 // Array de retorno
162 Local aPermissao     := {}                 // Verifica permissao
163 Local aDados         := {}                 // Array de informacoes do evento
164 Local cNumMov        := ""                 // Moviemnto
165 Local cCupomIni      := ""                 // Cupom inicial
166 Local cCash          := xNumCaixa()         // Caixa
167 Local lEmitNfce      := Iif(FindFunction("LjEmitNfCe"), LjEmitNfCe(), .F.) // Sinaliza se utiliza NFC-e
168
169
170 Default cSupervisor   := Space(15)         // Supervisor
171
172 ParamType 0 Var       cSupervisor As Character Default Space(15)
173
174 // Verifica nas permissoes de usuario para abrir/reabrir caixa
175 aPermissao := STBAutOpenCash()
176 cSupervisor := aPermissao[2]
177
178 If aPermissao[1]
179
180     lRet := STDOpenCash( cCash      , aDtHr[1] , aDtHr[2] )
181
182     If lRet
183
184         //Emite leitura X segundo a regra de negocios
185         STBOpenLX()
186
187         If STFGetCfg("lMultCoin") // Se usa Multimoeda
188             // Abre caixas associados(tratamento multi-moeda)
189             STDOpenMult( cCash , aDtHr[1] , aDtHr[2] )
190         EndIf
191
192         If lUsaDisplay
193             // Inicia Evento
194             STFFireEvent(ProcName(0), "STDdisplay", { StatDisplay(), "1C"+" " } )
195             STFFireEvent(ProcName(0), "STDdisplay", { StatDisplay(), "2C"+" " } )
196             STFFireEvent(ProcName(0), "STDdisplay", { StatDisplay(), "0C" + STR0009 } ) //"CAIXA DISPONIVEL"
197         EndIf
198
199         // Caixa tem acesso a utilizar Impressora Fiscal
200         If (STFProFile(3)[1])
201
202             // Pega numero do cupom
203             cCupomIni := space(STBLenReceipt()) //Numero do Cupom
204             If !lEmitNfce
205                 aDados := {cCupomIni, Nil}
206                 aRet := STFFireEvent( ProcName(0) , , ; // Nome do processo
207                                     "STGetReceipt" , ; // Nome do evento
208                                     aDados )

```

```

209
210         If Len(aRet) == 0 .OR. aRet[1] <> 0 .OR. Len(aDados) == 0
211             STFSHOWMESSAGE( STR0010 , "STGetReceipt" ) //"Falha na obtenção do cupom"
212         Else
213             cCupomIni := aDados[1]
214         EndIf
215     Else
216         //Se for nfce, captura o último documento
217         cCupomIni := STDCSLastSale("L1_DOC")
218     EndIf
219
220 EndIf
221
222 // Grava Movimento Processos de Venda(SLW)
223 lRet := STDOSaveMov( aDtHr , Soma1( AllTrim(cCupomIni) ) )
224
225
226 EndIf
227
228 EndIf
229
230
231 Return lRet
232
233
234 //-----
235 /*{Protheus.doc} STWOpenSupply
236 Function abertura de caixa com Troco.
237 Esta funcao faz chamadas das funcionalidades de abertura e depois de troco.
238 @param nVlrTroco      Valor de troco
239 @return lOpenCash     Retorna se abriu o caixa
240 @obs
241 @sample
242 */
243 //-----
244 Function STWOpenSupply(nVlrTroco, cCxOrig, cCxDest, aCaixas, lAbreCx)
245
246 Local lRet           := .T.           // Retorno
247 Local aValues        := Array(17)     // Array de valores para operacao
248 Local nCaixas        := 0
249 Local cAgencia       := ""            //Variavel que armazena a Agencia recebita pelo aCaixas
250 Local cDvage         := ""            //Variavel que armazena o Digito da agencia recebita pelo aCaixas
251 Local cConta         := ""            //Variavel que armazena a Conta recebita pelo aCaixas
252 Local cDvConta       := ""            //Variavel que armazena o Digito da conta recebita pelo aCaixas
253 Local aStation       := STBInfoEst( 1, .T. ) // Dados da estação
254
255 Default nVlrTroco    := 0              // Valor de troco
256 Default cCxOrig := ""
257 Default cCxDest := ""
258 Default aCaixas := {}
259 Default lAbreCx := .F.                // Indica que foi acionada pela Abertura de caixa
260

```

```

261 For nCaixas := 1 to len(aCaixas)
262   If Alltrim(cCxOrig) == AllTrim(aCaixas[nCaixas][1][1])
263     cAgencia := aCaixas[nCaixas][2][1]
264     cDvage   := aCaixas[nCaixas][2][2]
265     cConta   := aCaixas[nCaixas][2][3]
266     cDvConta := aCaixas[nCaixas][2][4]
267     Exit
268   EndIf
269 Next
270
271 cCxOrig := SubSTR(cCxOrig,1,TamSX3("A6_COD")[1])//Extraio apenas o A6_COD
272
273 If !Empty(cCxOrig)
274   aValues[CASHIER_ORIGIN] := cCxOrig
275 EndIf
276 If !Empty(cCxDest)
277   aValues[CASHIER_DESTINY] := cCxDest
278 EndIf
279
280 aValues[AGENCY] := cAgencia
281 aValues[DV_AGENCY] := cDvage
282 aValues[ACCOUNT] := cConta
283 aValues[DV_ACCOUNT] := cDvConta
284
285 aValues[VALUE_MONEY] := nVlrTroco
286
287 lOpenPDV := .T.
288
289 //Se for abertura de caixa executa o ponto de entrada aqui, senão for executa na função STWSupplyBleeding (Controle feito pela variavel lOpenPDV e função STWGetOpen() )
290 If ExistBlock("STIMotSa")
291   AADD(aStation,cUserName)
292   lRet := ExecBlock("STIMotSa",.F.,.F.,{lRet, aValues,lAbreCx,aStation})
293 EndIf
294
295 If lRet
296   lRet := STWPOSOpenCash()
297   If lRet .AND. nVlrTroco > 0
298     lRet := STWSupplyBleeding( 2 , aValues , STBGetCurrency() )
299   EndIf
300 EndIf
301
302 lOpenPDV := .F.
303
304 Return lRet
305
306 //-----
307 /*{Protheus.doc} STWOpenSupply
308 Função retorna se é abertura de caixa usada na função STWSupplyBleeding
309 @return lOpenPDV Retorna se é abertura de caixa
310 @obs
311 @sample
312 */

```

```
313 //-----  
314 Function STWGetOpen()  
315 Return lOpenPDV
```

Componentes/CorrespondentBanking/STBCorrespondentBanking.prw

240 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STBCORRESPONDENTBANKING.CH"
4
5 //-----
6 /*/ {Protheus.doc} STBRecCorBank
7 Prepara dados para gravacao do Correspondente Bancario nas funcoes financeiras.
8
9 @param      oRetCB      Objeto com retorno da transacao CB no TEF
10 @param      cSimbCoin    Simblo da moeda
11 @param      cNumCup      Numero do cupom
12 @param      cPref        Prefixo do título
13 @return
14 @obs
15 @sample
16 /*/
17 //-----
18 Function STBRecCorBank( oRetCB , cSimbCoin , cNumCup, cPref )
19
20 Local cParcela      := LjParcela(1, SuperGetMV("MV_1DUP"))      // Parcela
21 Local cNumCupFis     := ""                                     // Numero do cupom fiscal
22 Local cNatureza      := ""                                     // Natureza da operacao
23 Local cCliente       := SuperGetMV("MV_CLIPAD",, "")           // Cliente padrao
24 Local cLoja          := SuperGetMV("MV_LOJAPAD",, "")          // Loja padrao
25 Local dEmiss         := dDataBase                             // Data de emissao
26 Local dVencTo        := dDataBase                             // Data de vencimento
27 Local cHist          := "CORRESP. BANC." + " - " + STFGetStation("CODIGO") // Historico da operacao "CORRESPONDENTE BANCARIO"
28 Local nMoeda         := 1                                     // Tipo da moeda
29 Local cPrw           := "LOJXTEF"                             // Programa que gerou o lancamento
30 Local nValOperac     := 0                                     // Valor da operacao
31 Local cPortado       := xNumCaixa()                           // Caixa que realizou a opercao
32 Local cTipo          := ""                                    // Sigla da forma de pagamento
33 Local aDataCB        := {}                                    // Array com dados do CB para gerar financeiro
34 Local cBanco         := Space(TamSX3("EF_BANCO")[1])          // Banco do cliente
35 Local cAgencia       := Space(TamSX3("EF_AGENCIA")[1])         // Agencia
36 Local cConta         := Space(TamSX3("EF_CONTA")[1])           // Conta
37 Local cNumChq        := Space(TamSX3("EF_NUM")[1])            // Numero do Cheque
38 Local cCompensa      := Space(TamSX3("EF_COMP")[1])           // Compensação
39 Local cRg            := Space(TamSX3("EF_RG")[1] )             // Rg do cliente
40 Local cTel          := Space(TamSX3("EF_TEL")[1])             // Telefone
41 Local lTerceiro      := .F.                                    // Indica se o cheque e de terceiro
42 Local lRet           := .F.                                    // Controla o retorno da funcao
43 Local lRetRmtExc     := .F.                                    // Controla o retorno da funcao
44 Local cLabelDoc      := ""                                    // Contem a string de Label quando o pagamento for com cheque
45 Local cNsuSitef     := ""                                    // Numero de NSU-Sitef de pagamento
46 Local aNotas         := {}                                    //Array com numeração de NFce
47 Local cCupom        := ""                                    //Numero do cupom Fiscal
48 Local cMsgLog        := ""                                    //Log de Mensagem
49 Local lLgSerNFisc    := SLG->(ColumnPos("LG_SERNFIS")) > 0
50 Local lFisPrint      := STFGetCfg("lUseECF")                 //usa impressora fiscal
51
52 Default oRetCB      := Nil

```

```

53 Default cSimbCoin    := ""
54 Default cNumCup      := ""
55 Default cPref        := ""
56
57
58 ParamType 0 Var oRetCB      AS Object      Default Nil
59 ParamType 1 Var cSimbCoin   AS Character   Default ""
60 ParamType 2 Var cNumCup     AS Character   Default ""
61 ParamType 3 Var cPref      AS Character   Default ""
62
63 If Empty(cPref)
64     //Rotina anterior, o numero do cupom é o numero do ecf e já veio incrementado
65     cPref := STFGetStat("LG_SERIE")
66
67     If !lFisPrint
68         If lLgSerNFisc .AND. !Empty(cPref := STFGetStation("SERNFIS"))
69             LjxDNota( cPref , 3, .F., 1, @aNotas ,,,,,,,,,, "DOCNF") // DOC/SERIE
70
71             If Len(aNotas) > 0 .AND. Len(aNotas[1]) > 1
72                 cNumCup := aNotas[1][2]
73             Else
74                 LjGrvLog( "STBRecCorBank2","Falha ao obter numeração de Série Não Fiscal, será mantido o número da última venda " + cNumCup )
75             EndIf
76         Else
77             cMsgLog := STR0003 //"Para transações em Correspondente Bancário é necessário criar e informar o campo LG_SERNFIS, criado no compatibilizador U_UPDL0111"
78             STFMessage("STBRecCorBank", "ALERT", cMsgLog)
79             STFShowMessage("STBRecCorBank")
80             LjGrvLog( "STBRecCorBank",cMsgLog)
81         EndIf
82     EndIf
83
84 EndIf
85
86
87 // Monta informacoes da transacao TEF
88 cNsuSitef      := oRetCB:CAUTORIZ          // Numero de NSU-Sitef de pagamento
89 nValOperac     := oRetCB:NVLRTOTCB         // Pega Valor do Titulo
90 cTipo          := AllTrim(cSimbCoin)       // Simbolo da moeda utilizada no pagamento pelo TEF
91 cNatureza      := STGetDscPag( cTipo )     // Natureza da operacao
92
93 //Itau e BB So realizam pagamento a vista
94
95 If cTipo == "CD"
96     cNatureza      := SuperGetMV("MV_NATTEF")
97 EndIf
98
99 If cTipo == "CH"
100
101     cTel           := IIF(Valtype(oRetCB:cTelefone)== "N", Str(oRetCB:cTelefone), oRetCB:cTelefone)
102     cBanco         := STR(oRetCB:NBANCO      )
103     cAgencia       := STR(oRetCB:NAGENCIA   )
104     cConta         := STR(oRetCB:NCONTA     )

```

```

105     cNumChq      := STR(oRetCB:NCHEQUE )
106     cCompensa    := STR(oRetCB:NCOMPENSA )
107
108 EndIf
109
110
111 // Realizado comunicacao via componente de comunicacao STBRemoteExecute
112 // Efetua a gravacao do cancelamento no server
113 lContinua := STBRemoteExecute( "LjTefGrvCB" , { cPref      , cNumCup      , cParcela      , cTipo      , ;
114          cNatureza , cCliente   , cLoja        , dEmiss     , ;
115          dVencTo , cHist       , nMoeda      , cPrw       , ;
116          nValOperac , cPortado   , cBanco      , cAgencia   , ;
117          cConta , cNumChq      , cCompensa   , cRg        , ;
118          cTel      , lTerceiro   , cNsuSitef   ,             } , ;
119          NIL, .T. , @lRetRmtExc )
120
121 // Se nao executou a rotina no BackOffice
122 If !lContinua .OR. !lRetRmtExc
123     STFMessage("STCorBank","STOP", STR0002 ) //"Atencao, Nao foi possivel gravar as infomacoes financeiras do Correspondente Bancario, será gravado em modo de contingência"
124 EndIf
125
126 Return lRet
127
128
129 // Todo Tirar essa rotina e colocar para buscar do model
130 // TODO
131 Function STGetDscPag(cTypePag)
132
133 Local cDescription := ""           // Descricao da forma de pagamento
134
135 Default cTypePag := ""           // Tipo de pagamento a ser pesquisado
136
137 //Pega a descricao da forma de pagamento no SX5
138 DbSelectArea("SX5")
139 DbSeek(xFilial("SX5") + "24")
140 //Procura a forma na tabela 24
141 While !Eof() .AND. SX5->X5_FILIAL == xFilial("SX5") .AND. SX5->X5_TABELA == "24"
142     //Verifica se encontrou a forma
143     If AllTrim(SX5->X5_CHAVE) == cTypePag
144         //Guarda a descricao da forma
145         cDescription := Alltrim(SX5->X5_DESCRI)
146         Exit
147     EndIf
148     DbSkip()
149 End
150
151 Return cDescription
152
153 //-----
154 /*/ {Protheus.doc} STBGrvCorBkn
155 Gravacao do correspondente bancario
156

```

```

157 @param
158 @return      lRet
159 @obs
160 @sample
161 /*/
162 //-----
163 Function STBTefGrvCB (      cPref      , cNumTit   , cParcela , cTipo   , ;
164                          cNatureza , cCliente  , cLoja   , dEmiss  , ;
165                          dVencto   , cHist    , nMoeda  , cRotina , ;
166                          nValtit   , cPortado , cBanco  , cAgencia , ;
167                          cConta    , cNumChq  , cCompensa , cRg     , ;
168                          cTel      , lTerceiro , cNsuSiTef )
169
170 Local aDados      := {}           // Campos do SEI que necessitam ser enviados
171 Local aArea       := GetArea() //Salva a area
172
173 Private lMsErroAuto := .F.
174
175 aDados := { { "E1_PREFIXO" , cPref      , Nil} , ;
176             { "E1_NUM"      , cNumTit   , Nil} , ;
177             { "E1_PARCELA"  , cParcela  , Nil} , ;
178             { "E1_TIPO"     , cTipo     , Nil} , ;
179             { "E1_NATUREZA" , cNatureza , Nil} , ;
180             { "E1_CLIENTE"  , cCliente  , Nil} , ;
181             { "E1_LOJA"     , cLoja     , Nil} , ;
182             { "E1_EMISSAO"  , dEmiss    , Nil} , ;
183             { "E1_VENCTO"   , dVencto    , Nil} , ;
184             { "E1_VENCREA"  , DataValida(dVencto) , Nil} , ;
185             { "E1_HIST"     , cHist     , Nil} , ;
186             { "E1_MOEDA"    , nMoeda     , Nil} , ;
187             { "E1_ORIGEM"   , cRotina    , Nil} , ;
188             { "E1_FLUXO"    , "S"        , Nil} , ;
189             { "E1_VALOR"    , nValTit    , Nil} , ;
190             { "E1_PORTADO"  , cPortado   , Nil} , ;
191             { "E1_BCOCHQ"   , cBanco     , Nil} , ;
192             { "E1_AGECHQ"   , cAgencia   , Nil} , ;
193             { "E1_CTACHQ"   , cConta    , Nil} , ;
194             { "E1_DOCTEF"   , cNsuSiTef , Nil} }
195
196 MExecAuto( { |x,y| Fina040( x, y ) }, aDados, 3 ) //Inclusao
197
198 If cTipo == "CH"
199     LJRecGrvCH( cBanco , cAgencia , cConta , cNumChq , ;
200               nValTit , dEmiss , cCompensa, cRg , ;
201               cTel , lTerceiro, cPref , cNumTit , ;
202               cParcela, cTipo )
203     RecLock("SEF", .F.)
204     REPLACE SEF->EF_CLIENTE WITH cCliente
205     REPLACE SEF->EF_LOJACLI WITH SA1->A1_LOJA
206     REPLACE SEF->EF_EMITENT WITH SA1->A1_NOME
207     REPLACE SEF->EF_NUMNOTA WITH cNumTit
208     REPLACE SEF->EF_SERIE WITH cPref

```

```

209     MSUnlock()
210 EndIf
211
212 If cTipo == SuperGetMV( "MV_SIMB1" )
213     aDados := { { "E1_PREFIXO" ,E1_PREFIXO      , Nil}, ;// 01
214                 { "E1_NUM"      ,E1_NUM          , Nil}, ;// 02
215                 { "E1_PARCELA" ,E1_PARCELA       , Nil}, ;// 03
216                 { "E1_TIPO"    ,E1_TIPO          , Nil}, ;// 04
217                 { "E1_MOEDA"   ,E1_MOEDA         , Nil}, ;// 05
218                 { "E1_TXMOEDA" ,E1_TXMOEDA       , Nil}, ;// 06
219                 { "E1_ORIGEM"  ,E1_ORIGEM        , Nil}, ;// 07
220                 { "AUTVALREC"  ,E1_VALOR         , Nil}, ;// 06
221                 { "AUTMOTBX"   ,"NOR"           , Nil}, ;// 07
222                 { "AUTDTBAIXA" ,dDataBase      , Nil}, ;// 08
223                 { "AUTDTCREDITO",dDataBase      , Nil}, ;// 09
224                 { "AUTHIST"    ,cHist            , Nil} }// 13
225
226     // Verifica se deu algum erro no MSExecAuto anterior
227     If !lMsErroAuto
228
229         MSExecAuto( { |x,y| FINA070( x, y ) }, aDados, 3 )
230
231     Else
232         DisarmTransaction()
233     EndIf
234 EndIf
235
236 lRet := .T.
237
238 RestArea( aArea )
239
240 Return( lRet )

```

Componentes/CorrespondentBanking/STWCorrespondentBanking.prw

144 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STWCORRESPONDENTBANKING.CH"
4
5 //-----
6 /*{Protheus.doc} STWCorBank
7 Correspondente bancario.
8 Responsavel por pagamento e extorno de contas utilizando funcoes TEF 2.0
9
10 @param  oNewTEF      Objeto TEF
11 @return   Nil
12 @obs
13 @sample
14 */
15 //-----
16 Function STWCorBank( oNewTEF )
17
18 Local oDados              := Nil                      //Instancia da classe de correspondente bancario
19 Local oRetTran            := Nil                      //Retorno da transacao
20 Local aRet                := {}                      //Retorno da funcao
21 Local cStation            := STFGetStat("CODIGO")      //Codigo da estacao atual
22 Local cNumCup             := space(STBLenReceipt())    //Numero do Cupom
23 Local cSimbCoin           := ""                      //Simbolo da moeda do pagamento
24 Local nDecimais           := 0                      //Decimais
25 Local aDados              := {}                      //Array de dados
26 Local lLgSerNFisc := .F. //Campo de série não fiscal
27 Local cMsgLog            := ""                      //Mensagem de Log da Recarga
28 Local cPrefTit           := ""                      //Prefixo do título
29 Local aNotas              := {}                      //Array com numeração de NFce
30 Local lFisPrint          := STFGetCfg("lUseECF")      //usa impressora fiscal
31
32
33 Default oNewTEF          := STBGetTEF()
34
35 If Type("nMoedaCor") <> "U"
36     nDecimais := MsDecimais(nMoedaCor)
37 Else
38     nDecimais := MsDecimais(1)
39 EndIf
40
41
42 aDados := {cNumCup, Nil}
43 If lFisPrint
44 //Buca o Numero do Cupom Fiscal
45
46     aRet := STFFireEvent( ProcName(0) , ; // Nome do processo
47                          "STGetReceipt" , ; // Nome do evento
48                          aDados )
49
50 EndIf
51
52 If lFisPrint .And. (Len(aRet) == 0 .OR. aRet[1] <> 0 .OR. Len(aDados) == 0)

```

```

53   MsgStop( STR0001 , "STGetReceipt" )    //"Falha na obtenção do cupom"
54 Else
55
56
57   cNumCup      := aDados[1]
58
59   If !lFisPrint //Se for NFe
60     //NFCe pega o numero Da última venda
61     cNumCup := STDCSLastSale("L1_DOC")
62   EndIf
63
64   cNumCup := Soma1(cNumCup)
65
66   cPrefTit := STFGetStat("LG_SERIE")
67   lLgSerNFisc := SLG->(ColumnPos("LG_SERNFIS")) > 0
68
69   If ( lLgSerNFisc .AND. !Empty(cPrefTit := STFGetStat("SERNFIS")) ) .OR. lFisPrint
70
71     //Testo se é nfce para pegar o numero do cupom pelo lJxdnota
72
73     If !lFisPrint
74       LjxDNota( cPrefTit , 3, .F., 1, @aNotas,,,,,,,,, "DOCNF" ) // DOC/SERIE
75
76       If Len(aNotas) > 0 .AND. Len(aNotas[1]) > 1
77         cNumCup := aNotas[1][2]
78       Else
79         cNumCup := ""
80       EndIf
81     EndIf
82
83     If !Empty(cNumCup)
84       oDados := LJCDadosTransacaoCB():New( 0, Val(cNumCup), Date(), Time())
85
86       oRetTran := oNewTef:CorrespBanc():EfetuaPgto(oDados)
87
88
89       If oRetTran:oRetorno:lTransOk
90
91         // Pega pagamento utilizado
92         cSimbCoin := oNewTef:CorrespBanc():GetFormaPg()
93
94         //retorna a forma do CB bancario
95
96         oNewTef:Cupom():Inserir( "V"
97                                oRetTran:oRetorno:oViaCaixa    ;;
98                                oRetTran:oRetorno:oViaCliente   ;;
99                                "B"                             ;;
100                                oNewTef:CorrespBanc():GetTotalizador(),;
101                                oNewTef:CorrespBanc():GetFormaPg(),;
102                                oRetTran:oRetorno:nVlrTotCB      ;;
103                                1                                ;;
104                                0                                )

```

```

105
106         If oNewTEF:Cupom():Imprimir()
107
108             // Confirma transacao
109             oNewTEF:Confirmar()
110             //Gravacao da transacao
111             STBRecCorBank( oRetTran:oRetorno , cSimbCoin , AllTrim(Str(Val(cNumCup))), cPrefTit)
112
113         Else
114             oNewTEF:Desfazer()
115         EndIf
116
117     Else
118         oNewTEF:Desfazer()
119         STFMessage("STCorBank","STOP", STR0002) //"Atencao, Nao foi possivel realizar a transacao com correspondente bancario"
120         STFShowMessage("STCorBank")
121     EndIf
122 Else
123
124     STFMessage("STCorBank2","STOP", STR0001)    //"Falha na obtenção do cupom"
125     STFShowMessage("STCorBank2")
126 EndIf
127 Else
128     cMsgLog := STR0003 //"Para transações em Correspondente Bancário é necessário criar e informar o campo LG_SERNFIS, criado no compatibilizador U_UPDL0111"
129     STFMessage("STCorBank", "STOP", cMsgLog)
130     STFShowMessage("STCorBank")
131     LjGrvLog( "STCorBank",cMsgLog)
132
133 EndIf
134 EndIf
135
136
137 If ValType(oDados) == "0"
138     FreeObj(oDados)
139 EndIf
140 If ValType(oRetTran) == "0"
141     FreeObj(oRetTran)
142 EndIf
143
144 Return Nil

```

Componentes/CreditAnalysisIntegration/STDCreditAnalysisIntegration.prw

24 linhas após remoção de metadados de autoria/data

```
1 //-----
2 /*{Protheus.doc} STDFindCart()
3 Pesquisa o número do cartão do cliente
4
5 @param
6 @return aRet
7 @obs
8 @sample
9 */
10 //-----
11 Function STDFindCart()
12
13 Local aRet      := {}
14 Local cCart     := ""
15 Local aCart     := {}
16
17 If ExistFunc("STBGetCrdIdent")
18     aCart := STBGetCrdIdent() //Busco o número do cartão
19 EndIf
20 If Len(aCart) > 0 .AND. !Empty(aCart[1])
21     aAdd(aRet, aCart[1])
22 EndIf
23
24 Return aRet
```

Componentes/CreditAnalysisIntegration/STWCreditAnalysisIntegration.prw

231 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STWCREDITANALYSISINTEGRATION.CH"
4
5 //-----
6 /*{Protheus.doc} STWAvalCRDIntegration
7
8 Rotina de integracao do POS com o SIGACRD. Chama a CRDxVenda, que avalia o credito e realiza a transacao de venda.
9 @param
10 @return lRet - .T. se o credito foi aprovado / .F. se o credito foi reprovado
11 @obs
12 @sample
13 */
14 //-----
15 Function STWAvalCRDIntegration(aParcelas,aRecCrd)
16
17 Local aArea          := GetArea()
18 Local aDadosCrd      := {}          //para CRDxVenda()
19 Local aParcVda       := {}
20 Local aParcCRD       := {}
21 Local aProdCrd       := {}
22 Local aRetCrd        := {}
23 Local cOrcam         := Alltrim(STBRetCup())
24 Local cCliente       := STDGPBasket("SL1","L1_CLIENTE")
25 Local cLojaCli       := STDGPBasket("SL1","L1_LOJA")
26 Local cItemCond      := STBGetCdPg()
27 Local cVendLoja      := STDGPBasket("SL1","L1_VEND")
28 Local oTotal         := STFGetTot() // Recebe o Objeto totalizador
29 Local nVlrTotal      := oTotal:GetValue("L1_VLRTOT") // Valor total da venda
30 Local nVlrAcreTot    := 0
31 Local nTotAcres      := 0
32 Local nMoedaParc     := 0
33 Local nI             := 0
34 Local lCheque        := .F.
35 Local lRet           := .T.
36 Local cRespForcada   := ""
37 Local aCrdCliente    := {"",""} //Seleção Cartão/CNPJ para Integração CRD
38 Local aDadosCli      := {}      //Dados do Cartão/CNPJ para Integração CRD
39 Local cContrato      := ""      //Retorna o contrato na primeira verificação do CRDxVenda
40
41 Default aParcelas    := {}
42 Default aRecCrd      := {}
43
44 //Verifico o CNPJ/CPF e o cartão
45 If ExistFunc("STBGetCrdIdent")
46     aDadosCli := STBGetCrdIdent()
47 EndIf
48 If Len(aDadosCli) >= 4
49     aCrdCliente[2] := aDadosCli[1] //Numero do cartao
50     aCrdCliente[1] := aDadosCli[2] //CNPJ/CPF
51 EndIf
52

```

```

53 //Monta os dados com as a venda para o sistema de credito fazer a
54 //checagem do credito e a liberacao da venda
55 aParcVda := {}
56 For nI := 1 To Len(aParcelas)
57     lCheque      := AllTrim(aParcelas[nI][4])=="CH"
58     nMoedaParc   := IIf(cPaisLoc == "BRA", 1, aParcelas[nI][24])
59     Add(aParcVda,{aParcelas[nI][2],,;
60         aParcelas[nI][3],,;
61         aParcelas[nI][4],,;
62         aParcelas[nI][5],,;
63         aParcelas[nI][6],,;
64         IIf(lCheque,aParcelas[nI][7],""),,;
65         IIf(lCheque,aParcelas[nI][8],""),,;
66         IIf(lCheque,aParcelas[nI][9],""),,;
67         IIf(lCheque,aParcelas[nI][10],""),,;
68         .F.,,;
69         nMoedaParc })
70 Next nI
71
72 aProdCrd := {}
73 SL2->( DbSetOrder( 1 ))
74 SL2->( DbSeek( xFilial("SL2") + cOrcam ) )
75 While SL2->( !Eof() ) .AND. SL2->(xFilial("SL2") + cOrcam == L2_FILIAL+L2_NUM )
76     AAdd( aProdCrd, { SL2->L2_ITEM,,; //Item do Produto
77         SL2->L2_PRODUTO,,; //Codigo do Produto
78         SL2->L2_DESCRI,,; //Descricao do Produto
79         SL2->L2_QUANT,,; //Quantidade de pecas vendidas
80         SL2->L2_VRUNIT,,; //Valor unitario do produto
81         SL2->L2_VLRITEM } ) //Valor total da venda do item
82     SL2->( dbSkip() )
83     Loop
84 End
85
86 If Len(aParcVda) > 0
87
88     //Estrutura da array aDadosCrd
89     //[1] - Numero do cartao
90     //[2] - Numero do CPF
91     //[3] - Valor da venda (valor liquido)
92     //[4] - Juros da venda (%)
93     //[5] - Numeros de parcelas
94     //[6] - Venda forcada (1-Venda Normal 2-Venda forcada)
95     //[7] - Responsavel pela venda forcada
96     //[8] - Array com as parcelas para o financiamento
97     //     [1] - Data de vencdo
98     //     [2] - Valor da parcela
99     //     [3] - Forma de pagto
100    //     [4] - Administradora financeira
101    //[9] - Loja que solicitou a transacao
102    //[10] - Numero do PDV que solicitou a transacao
103    //[11] - Caixa que solicitou a transacao
104    //[12] - Numero do Orcamento selecionado

```

```

// Data de vencimento
// Valor da parcela
// Forma de pagamento
// Codigo da administradora ou numero do banco
// Numero do cartao ou cheque
// Agencia do cheque
// Conta do cheque
// RG do cheque
// Telefone do portador do cheque
// (Mantido por compatibilidade)
// Moeda da parcela

```

```

105 // [13] - Produtos contido na venda atual.
106 // [1] - Item do produto
107 // [2] - Codigo do produto
108 // [3] - Descricao do produto
109 // [4] - Quantidade de pecas
110 // [5] - Valor unitario do produto
111 // [6] - Valor total do item do produto
112 // [14] - Parcelas de uma venda.
113 // [1] - Data de vencimento
114 // [2] - Valor da parcela
115 // [3] - Forma de pagamento
116 // [4] - Administradora
117 // [5] - Numero do cartao / cheque
118 // [6] - Agencia - Cheque
119 // [7] - Conta - Cheque
120 // [8] - Rg - Cheque
121 // [9] - Telefone - Cheque
122 // [10] - Valor logico
123 // [11] - Moeda da parcela (localizacoes)
124 // [12] - ID para identificacao da forma de pagamento na operação TEF
125 // [13] - Acrescimo financeiro
126 // [14] - Cpf/Cnpj do Cliente
127 // [15] - Filial do Caixa que esta sendo utilizado.
128 // [16] - Codigo do cliente
129 // [17] - Loja do cliente
130 // [18] - Nome do usuario
131 // [19] - Condicao de Pagamento
132 // [20] - Modulo que chamou a avaliacao de credito
133 // [21] - Codigo do vendedor
134
135 aDadosCrd := {}
136 aAdd( aDadosCrd, aCrdCliente[2] ) // 01 Numero do cartao
137 aAdd( aDadosCrd, aCrdCliente[1] ) // 02 CNPJ/CPF
138 aAdd( aDadosCrd, nVlrTotal+ nTotAcres ) // 03 Valor da venda
139 aAdd( aDadosCrd, (nVlrAcreTot * 100 / (nVlrTotal-nVlrAcreTot)) ) // 04 Juros da venda
140 aAdd( aDadosCrd, Len(aParcVda) ) // 05 Numero de parcelas
141 aAdd( aDadosCrd, 1 ) // 06 Venda forcada
142 aAdd( aDadosCrd, cRespForcada ) // 07 Responsavel pela venda forcada
143 aAdd( aDadosCrd, aClone(aParcVda) ) // 08 Dados das parcelas [n,1]=Data vencimento [n,2]=Valor da parcela [n,3]=Forma de pagamento
144 aAdd( aDadosCrd, SM0->M0_CODIGO+"-"+FWGETCODFILIAL+"-"+Alltrim(SM0->M0_NOME) ) // 09 Loja da venda
145 aAdd( aDadosCrd, STFGetStation("LG_PDV")) // 10 PDV
146 aAdd( aDadosCrd, xNumCaixa() ) // 11 Caixa
147 aAdd( aDadosCrd, cOrcam ) // 12 Num. do orcamento
148 aAdd( aDadosCrd, aClone(aProdCrd) ) // 13 Produtos selecionados numa venda
149 aAdd( aDadosCrd, aClone(aParcVda) ) // 14 Parcelas de uma venda
150 aAdd( aDadosCrd, FWGETCODFILIAL ) // 15 Filial do Caixa que esta sendo utilizado
151 aAdd( aDadosCrd, cCliente ) // 16 Codigo do cliente
152 aAdd( aDadosCrd, cLojaCli ) // 17 Loja do cliente
153 aAdd( aDadosCrd, cUserName ) // 18 Nome do usuario
154 aAdd( aDadosCrd, cItemCond ) // 19 Condicao de Pagamento
155 aAdd( aDadosCrd, "FRT" ) // 20 Modulo chamador
156 aAdd( aDadosCrd, cVendLoja ) // 21 Codigo do vendedor

```

```

157
158   aRetCrd := aClone(CrdxVenda( "1"      ,@aDadosCrd      , Nil      , .T.      ,;
159                               Nil      , "43"            , Nil      , @aRecCrd))
160
161   If Len(aRetCrd) < 5      //Errorlog
162     STFMessage( ProcName(0),"STOP", STR0001) //"Erro na transmissão da análise de crédito"
163     STFShowMessage(ProcName(0))
164     lRet := .F.
165   EndIf
166
167   If lRet
168     //Verifica se eh venda financiada
169     If Len(aDadosCrd[8]) > 0
170       //0 array recebe as parcelas que foram financiadas
171       aParcCrd:= aClone(aDadosCrd[8])
172       If !aRetCrd[5]
173
174         //A liberacao do credito foi confirmada
175         lTransCrd := .T.
176
177         //Alimenta as variaveis referente aos dados do cliente
178         // Verifica se a venda foi off-line
179         If aRetCrd[1] == 2
180           lForcada := .T.
181           If HasTemplate("DRO")
182             If Empty(cCliente + cLojaCli)
183               cCliente := SuperGetMV("MV_CLIPAD")
184               cLojaCli := SuperGetMV("MV_LOJAPAD")
185             EndIf
186           Else
187             cCliente := SuperGetMV("MV_CLIPAD")
188             cLojaCli := SuperGetMV("MV_LOJAPAD")
189           EndIf
190         Else
191           cCliente := aDadosCrd[16]
192           cLojaCli := aDadosCrd[17]
193         EndIf
194       Else
195
196         //A liberacao do credito nao foi confirmada
197         cCliente := aDadosCrd[16]
198         cLojaCli := aDadosCrd[17]
199         lTransCrd := .F.
200       EndIf
201
202       //[1]-CPF [2]-Numero do Cartao Private Label
203       aCrdCliente[1] := aDadosCrd[2]
204       aCrdCliente[2] := aDadosCrd[1]
205       cContrato := aRetCrd[4]
206     Else
207       cContrato := Space(TamSx3("MAH_CONTRA")[1])
208     EndIf

```

```

209
210     STDSPBasket("SL1","L1_CONTRA",cContrato)    //Indispensável para subida via GrvBatch, excluindo o MAL e gravando o SE1 (E1_NUMCRD).
211
212     If ValType(aRetCrd[5]) == "L"
213         // aRetCrd[5] recebe .T. caso a venda tenha sido rejeitada. Portanto, lRet recebe o seu inverso.
214         lRet := !aRetCrd[5]
215     EndIf
216 EndIf
217
218 If lRet .AND. lTransCrd //Transação
219     aRetCrd := CrdxVenda( "2" ,aDadosCrd ,cContrato ,.T. ,;
220                         Nil ,Nil ,Nil ,aRecCrd)
221 Else //Cancelamento
222     aRetCrd := CrdxVenda( "3" ,aDadosCrd ,cContrato ,.T. ,;
223                         NIL ,NIL )
224     STDSPBasket("SL1","L1_CONTRA","")
225 EndIf
226
227 EndIf
228
229 RestArea(aArea)
230
231 Return lRet

```

Componentes/Customer/STBCustomerSelection.prw

318 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE 'Protheus.ch'
2 #INCLUDE 'STBCUSTOMERSELECTION.CH'
3
4 Static aCliCrd := {"","","",""}
5
6 //-----
7 /*{Protheus.doc} STBCadCli
8 Cadastro de cliente não localizado localmente mediante a busca de orçamento
9
10 @param      aOrcImp, array, Array com o orçamento importado
11 @return Nil, Nulo
12
13 */
14 //-----
15 Function STBCadCli(aOrcImp)
16
17 Local aCustomer      := {} // -- Dados do código do cliente e loja
18 Local cKey           := "" // -- Chave para busca (Cliente + Loja)
19 Local cCod           := "" // -- Código do cliente
20 Local cLoja          := "" // -- Código da loja
21
22 Local aDataCustomers:= {} // -- Dados para a gravação do cliente
23 Local aCampos        := {} // -- Campos a ser gravados
24 Local aDados         := {} // -- Valor dos campos a ser gravados
25 Local nCustomer      := 0 // -- Variável para repetição
26 Local uResult        := Nil // -- Variável com o resultado da consulta na retaguarda
27 Local lRet           := .F. // -- Retorno da consulta na retaguarda
28 Local aParam         := {} // -- Parâmetros a ser enviado para a função que será executada na retaguarda
29 Local nAtuCliVda     := SuperGetMv("MV_LJATCLI",,2) // Força a atualização do cliente
30
31 Default aOrcImp      := {} // -- Dados do orçamento
32
33 If Len(aOrcImp) > 0
34     // Pega o orçamento importado os campos chaves de cliente e loja
35     cCod := aOrcImp[1][1][aScan(aOrcImp[1][1], { |x| x[1] == "L1_CLIENTE"})][2]
36     cLoja := aOrcImp[1][1][aScan(aOrcImp[1][1], { |x| x[1] == "L1_LOJA"})][2]
37
38     If ValType(cCod) <> "C" .OR. ValType(cLoja) <> "C"
39         cCod := ""
40         cLoja := ""
41     EndIf
42
43     cKey := cCod + cLoja
44
45     //Verifica se o cliente existe na base local
46     If !Empty(cKey) .AND. (!STDVerfCadCli(cKey)[1] .OR. nAtuCliVda == 1 )
47
48         aFields := STDWhatfields()
49         aCustomer := {cCod,cLoja}
50
51         //pega dados do cliente na retaguarda
52

```

```

53     aParam := {Nil,1,.F.,aCustomer,aFields}
54
55     STFMessage(ProcName(), "RUN", STR0002 ,{ || lRet := STBRemoteExecute("STDSearchCostumer", aParam,,, @uResult) })
56     STFShowMessage(ProcName())
57
58     aDataCustomers := uResult[2]
59
60     If lRet .AND. Len(aDataCustomers) > 0
61
62         aADD(aDados,{})
63         For nCustomer := 1 To len(aDataCustomers[1])
64             aADD(aCampos,aDataCustomers[1][nCustomer][1])
65             aADD(aDados[Len(aDados)] ,aDataCustomers[1][nCustomer][2])
66         Next
67
68         STIConfCus( aDataCustomers[1][2][2] ,aDataCustomers[1][3][2] ,aDataCustomers[1][1][2] ,aDataCustomers[1][5][2],;
69                 aDataCustomers[1][6][2] ,aDataCustomers[1][4][2] ,aDataCustomers[1][7][2] ,aDataCustomers[1][8][2],;
70                 aDataCustomers[1][9][2] ,aDataCustomers[1][10][2] ,.T. ,.F. ,;
71                 aDataCustomers[1][11][2] ,aDataCustomers[1][12][2] ,aDataCustomers[1][13][2] ,aCampos ,;
72                 aDados ,.F. , "TX" ) //Cadastra Cliente
73     Else
74
75         LjGrvLog(ProcName(),"Não foi possivel incluir o cliente Cod: " + cCod + " Loja: " + cLoja )
76
77     EndIf
78 EndIf
79 EndIf
80
81 Return
82
83 //-----
84 /*{Protheus.doc} STBGetCust
85 Funcao responsavel pela busca de clientes no banco de dados da Retaguarda (Base TOP).
86
87 @param cFil Filial a ser realizada a busca.
88 @param cNome Nome do Cliente a ser pesquisado.
89 @param nLimitRegs Quantidade Limite de registros que se deseja pesquisar.
90 */
91 //-----
92 Function STBGetCust(cFil, cNome, nLimitRegs)
93 Local uDados := {}
94 Local aParam := {}
95 Local uResult := Nil
96 Local aFields := {}
97 Local aTables := {}
98 Local cWhere := ""
99 Local cOrderBy := ""
100 Local cPesqNoLike := "" //Pesquisa nao like
101
102 Default cFil := ""
103 Default cNome := ""
104 Default nLimitRegs := 20

```

```

105
106 cNome := AllTrim(cNome)
107 cPesqNoLike := cNome
108
109 If At(cNome, '*') == 0
110     cNome := '*' + cNome + '*'
111 EndIf
112
113 cNome := Replace(cNome, "*", "%")
114
115 //Monta os campos da query
116 aFields := {"A1_COD", "A1_LOJA", "A1_NOME", "A1_MSBLQL", "A1_CGC"}
117
118 //Tabela da Query
119 aTables := {"SA1"}
120
121 //Monta a Cláusula Where da query
122 cWhere := "A1_FILIAL = '"+cFil+' AND "
123 cWhere += "(A1_NOME LIKE '" + cNome + ;
124             "' OR A1_COD LIKE '" + cNome + ;
125             "' OR A1_CGC = '" + cPesqNoLike + "') AND "
126 cWhere += "D_E_L_E_T_ = ' ' AND A1_MSBLQL <> '1'"
127
128 //Monta a Cláusula Order By da query
129 cOrderBy := "A1_NOME"
130
131 // Busca de Clientes via STBRemoteExecute
132 aParam := {
133     aFields,;
134     aTables,;
135     cWhere,;
136     cOrderBy,;
137     nLimitRegs;
138 }
139 If !STBRemoteExecute("STDQueryDB", aParam,,, @uResult)
140     // Tratamento do erro de conexao
141     uDados := .F.
142 Else
143     uDados := uResult
144 EndIf
145
146 Return uDados
147
148 //-----
149 /*{Protheus.doc} STBGetCrd
150 Funcao responsavel pela busca de clientes e cartão para SIGACRD no banco de dados da Retaguarda (Base TOP).
151 Baseado na função WebService WSCRD013 em WSCRD010.prw
152 @type function
153
154
155 @param      cCartao      , caracter, Cartão Fidelidade
156 @param      cCPF         , caracter, CPF ou CNPJ do Cliente

```

```

157 @param      nLimitRegs      , número, Limite de Registros a ser mostrado no componente
158
159 @return lógico ou array , Conteúdo em array do SA1 ou False
160 /**/
161 //-----
162
163 Function STBGetCrd( cCPF, cCartao, nLimitRegs )
164 Local aRet      := { 0, "", "", {}, {"",""}}      // Retorno da funcao
165 Local nX        := 0                               // Variavel de looping
166 Local nY        := 0                               // Variavel de looping
167 Local lContinua := .T.                             // Variavel para nao continuar o processamento
168 Local cChaveSA1 := ""                              // Chave do SA1. Utilizada para buscar mais de 1 ocorrencia no SA1
169 Local cChaveBusca := ""                            // Chave de busca no SA1. Utilizada para buscar mais de 1 ocorrencia no SA1
170 Local nOrdemSA1 := 0                               // Order de busca no SA1.
171 Local lPesqMatricula := .F.                        // Pesquisa pela matricula
172 Local nRecnoSA1 := 0                               // Guarda o recno do cliente posicionado na SA1
173
174 Local uDados      := { 0, "", "", {}, {"",""}}      // Retorno da funcao
175 Local aParam      := {}
176 Local uResult     := Nil
177 Local aFields     := {}
178 Local aTables     := {}
179 Local cWhere      := ""
180 Local cOrderBy    := ""
181 Local cPesqNoLike := ""                            //Pesquisa nao like
182 Local aAreaSA1    := {}                            //Area SA1
183 Local lRet        := .T.
184
185 Default cCartao   := ""
186 Default cCPF      := ""
187 Default nLimitRegs := 20
188
189 //UAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA;
190 //³Pesquisa o cadastro do cliente
191 //AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAU
192
193 //Monta os campos da query
194 aFields := {"MA6_CODCLI", "MA6_LOJA", "A1_NOME", "A1_MSBLQL", "MA6_NUM", "MA6_CODDEP", "MA6_SITUA", "A1_COD", "A1_LOJA", "A1_CGC"}
195
196 //Tabela da Query
197 aTables := {"MA6","SA1"}
198
199 //Monta a Cláusula Where da query
200 cWhere := "MA6_FILIAL = '"+xFilial("MA6")+"' AND "
201 If !Empty(cCartao)
202     cWhere += "MA6_NUM = '"+cCartao+"' AND "
203 ElseIf !Empty(cCpf)
204     cWhere += "A1_CGC = '"+cCpf+"' AND "
205 EndIf
206 cWhere += "MA6_CODCLI = A1_COD AND "
207 cWhere += "MA6_LOJA = A1_LOJA AND "
208 cWhere += "A1_FILIAL = '"+xFilial("SA1")+"' AND "

```

```

209 cWhere += "MA6.D_E_L_E_T_ = ' ' AND SA1.D_E_L_E_T_ = ' ' AND MA6_SITUA = '1'" //Se cartão ativo
210
211 //Monta a Cláusula Order By da query
212 cOrderBy := "A1_COD,A1_LOJA"
213
214 // Busca de Clientes via STBRemoteExecute
215 aParam := {
216     aFields;;
217     aTables;;
218     cWhere;;
219     cOrderBy;;
220     nLimitRegs;
221 }
222 If !STBRemoteExecute("STDQueryDB", aParam,,, @uResult)
223     uDados := .F.
224 Else
225     LjGrvLog("StbGetCrd",cCpf + " encontrado via STBRemoteExecute!")
226     uDados := uResult
227 EndIf
228
229 If ValType(uDados) = "L" .AND. !uDados
230     // Tratamento do erro de conexao
231     STFMessage(ProcName(), "ALERT", STR0001 ) //Por falta de comunicação com o Back-Office, será selecionado o cliente padrão.
232     STFShowMessage(ProcName())
233     LjGrvLog("StbGetCrd",STR0001)
234     If !Empty(cCpf) //Tratamento do erro de conexao: Caso não encontrar o cartão, pesquisa localmente na SA1.
235         aAreaSA1 := SA1->(GetArea())
236         SA1->(DbSetOrder(3)) //A1_FILIAL+A1_CGC
237         If SA1->( DbSeek(xFilial("SA1")+cCpf))
238             LjGrvLog("StbGetCrd",cCpf + " encontrado na base local!")
239             uDados := {{SA1->A1_COD,SA1->A1_LOJA,SA1->A1_NOME,SA1->A1_MBSLQL,"","","",SA1->A1_COD,SA1->A1_LOJA,SA1->A1_CGC}}
240         Else
241             LjGrvLog("StbGetCrd",cCpf + " NÃO encontrado na base local!")
242             uDados := .F.
243         EndIf
244         RestArea(aAreaSA1)
245     Else
246         uDados := .F.
247     EndIf
248 EndIf
249
250 Return uDados
251
252
253 //-----
254 /*{Protheus.doc} STBSetCrdIdent
255 Gravo os dados do cartão, CPF/CNPJ, código do cliente e loja do cliente
256 @type function
257
258
259 @param      cCartao      , caracter, Cartão Fidelidade
260 @param      cCPF         , caracter, CPF ou CNPJ do Cliente

```

```

261 @param      cCliente      , caracter, Código do Cliente
262 @param      cLoja          , caracter, Loja do Cliente
263
264 @return array {Cartão Fidelidade, CPF ou CNPJ do Cliente, Código do Cliente, Loja do Cliente}
265 */
266 //-----
267 Function STBSetCrdIdent( cCartao, cCPF, cCliente, cLoja )
268
269 Local nCompCartao := TamSX3("MA6_NUM")[1]
270 Local nCompCPF := TamSX3("A1_CGC")[1]
271
272 Default cCartao := ""
273 Default cCPF := ""
274 Default cCliente := ""
275 Default cLoja := ""
276
277 aCliCrd := { PadR(cCartao,nCompCartao),;
278              PadR(cCPF,nCompCPF),;
279              cCliente,;
280              cLoja }
281
282 Return nil
283
284 //-----
285 /*{Protheus.doc} STBGetCrdIdent
286 Leio os dados do cartão, CPF/CNPJ, código do cliente e loja do cliente
287 @type function
288
289
290 @param nil
291
292 @return array {Cartão Fidelidade, CPF ou CNPJ do Cliente, Código do Cliente, Loja do Cliente}
293 */
294 //-----
295 Function STBGetCrdIdent()
296
297 Return aCliCrd
298
299 /*{Protheus.doc} STBCPF CNPJ
300 Valida se o valor informado na consulta cliente é CPF ou CNPJ válido.
301 @type Function
302 @param cWhatSearch, caractere, informação do cliente digitado, pode ser cpf/cnpj, uma parte do nome ou código do cliente
303 @return lRet, lógico, caso CPF/CNPJ for válido retorna .T.
304 */
305 Function STBCPF CNPJ(cWhatSearch)
306 Local lRet := .F.
307 Local cInfDig := AllTrim(cWhatSearch)
308
309 cInfDig := StrTran(cInfDig,".", "")
310 cInfDig := StrTran(cInfDig,"-", "")
311 cInfDig := StrTran(cInfDig,"/", "")
312 cInfDig := StrTran(cInfDig,"*", "")

```

```
313
314 If Len(cInfDig) == 11 .OR. Len(cInfDig) == 14
315     lRet:= CGC(cInfDig, ,.F.)
316 Endif
317
318 Return lRet
```

Componentes/Customer/STBIncCustomer.prw

231 linhas após remoção de metadados de autoria/data

```

1 #Include 'Protheus.ch'
2 #Include 'STBINCCUSTOMER.CH'
3
4 //-----
5 /*{Protheus.doc} STBVldCGC
6 Validacao do campo A1_CGC. Qd Pessoa="J" nao permitir entrada de CPF.
7
8 @param
9 @return lRet          Retorno .T. se nao houver registro com a mesma chave unica na base.
10 @obs
11 @sample
12 */
13 //-----
14 Function STBVldCGC(cCodCliente,cLojCliente,cTipPes,cCNPJ)
15 Local aArea          := GetArea()
16 Local aAreaSA1       := SA1->(GetArea())
17 Local lRetorno       := .T.
18 Local cCNPJBase      := ""
19 Local cMv_ValCNPJ    := GetNewPar("MV_VALCNPJ","1")
20 Local cMv_ValCPF     := GetNewPar("MV_VALCPF","1")
21 Local lAchou         := .F.
22 Local lEleMesmo      := .F.
23
24 DEFAULT cCNPJ        := &(ReadVar())
25
26 //UAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA;
27 //³ Valida o tipo de pessoa
28 //AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAU
29 If cTipPes == "F" .AND. !(Len(AllTrim(cCNPJ))=11)
30   STFMessage("STBVldCGC","STOP",STR0001) //"Dígito verificador incorreto, entre com um código válido."
31   STFShowMessage("STBVldCGC")
32   lRetorno := .F.
33 ElseIf cTipPes == "J" .AND. !(Len(AllTrim(cCNPJ))=14)
34   STFMessage("STBVldCGC","STOP",STR0001) //"Dígito verificador incorreto, entre com um código válido."
35   STFShowMessage("STBVldCGC")
36   lRetorno := .F.
37 EndIf
38
39 //UAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA;
40 //³ Valida a duplicidade do CGC
41 //AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAU
42 If lRetorno .AND. Pcount() > 1
43   If cTipPes == "J" .Or. Empty(cTipPes)
44     DbSelectArea("SA1")
45     DbSetOrder(3)
46     lAchou := DbSeek(xFilial("SA1")+cCNPJ)
47     If lAchou
48       lEleMesmo := cCodCliente+cLojCliente == SA1->A1_COD+SA1->A1_LOJA
49     EndIf
50   If lAchou .AND. !lEleMesmo
51     //UAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA;
52     //³0 parametro MV_VALCNPJ verifica se a validacao do CNPJ deve ser feita:

```

```

53 //³1 = informando ao usuario que ja existe o CNPJ na base e verificando se deseja incluir mesmo assim³
54 //³2 = nao permitindo que o usuario insira o mesmo CNPJ
55 //AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAU
56 If cMv_ValCNPJ == "1"
57     If ! _SetAutoMode()
58         STFMessagem("STBVldCGC","YESN0",STR0002+" "+ AllTrim(SA1->A1_COD) + "/" + AllTrim(SA1->A1_LOJA) + " - " + AllTrim(SA1->A1_NOME) + STR0003) //"0 CNPJ
informado já foi utilizado no cliente" ..". Confirma o cadastro?"
59         If !STFShowMessage("STBVldCGC")
60             STFMessagem("STBVldCGC","STOP",STR0004) //"0 CNPJ informado já foi utilizado, favor alterar."
61             STFShowMessage("STBVldCGC")
62             lRetorno := .F.
63         EndIf
64     EndIf
65 Else
66     STFMessagem("STBVldCGC","STOP",STR0002+" "+ AllTrim(SA1->A1_COD) + "/" + AllTrim(SA1->A1_LOJA) + " - " + AllTrim(SA1->A1_NOME) + ".") //"0 CNPJ informado já
foi utilizado no cliente
67     STFShowMessage("STBVldCGC")
68     lRetorno := .F.
69 Endif
70 ElseIf lRetorno
71     cCNPJBase := SubStr(cCNPJ,1,8)
72     DbSelectArea("SA1")
73     DbSetOrder(3)
74     If DbSeek(xFilial("SA1")+cCNPJBase) .And. cCodCliente <> SA1->A1_COD .And. SA1->A1_PESSOA == "J"
75         If ! _SetAutoMode()
76             STFMessagem("STBVldCGC","YESN0",STR0002+" "+ AllTrim(SA1->A1_COD) + "/" + AllTrim(SA1->A1_LOJA) + " - " + AllTrim(SA1->A1_NOME) + STR0003) //"0 CNPJ
informado já foi utilizado no cliente" ..". Confirma o cadastro?"
77             If !STFShowMessage("STBVldCGC")
78                 STFMessagem("STBVldCGC","STOP",STR0004) //"0 CNPJ informado já foi utilizado, favor alterar."
79                 STFShowMessage("STBVldCGC")
80                 lRetorno := .F.
81             EndIf
82         EndIf
83     EndIf
84 EndIf
85 Else
86     DbSelectArea("SA1")
87     DbSetOrder(3)
88     If DbSeek(xFilial("SA1")+cCNPJ) .And. cCodCliente+cLojCliente <> SA1->A1_COD+SA1->A1_LOJA
89         //UUAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAZ
90         //³0 parametro MV_VALCPF verifica se a validacao do CPF deve ser feita:
91         //³1 = informando ao usuario que ja existe o CPF na base e verificando se deseja incluir mesmo assim³
92         //³2 = nao permitindo que o usuario insira o mesmo CPF
93         //AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAU
94         If cMv_ValCPF == "1"
95             If ! _SetAutoMode()
96                 STFMessagem("STBVldCGC","YESN0",STR0005+" "+ AllTrim(SA1->A1_COD) + "/" + AllTrim(SA1->A1_LOJA) + " - " + AllTrim(SA1->A1_NOME) + STR0003) //"0 CPF
informado já foi utilizado no cliente" ..". Confirma o cadastro?"
97                 If !STFShowMessage("STBVldCGC")
98                     STFMessagem("STBVldCGC","STOP",STR0006) // "0 CPF informado já foi utilizado, favor alterar."
99                     STFShowMessage("STBVldCGC")
100                    lRetorno := .F.

```

```

101         EndIf
102     EndIf
103 Else
104     STFMessage("STBVldCGC","STOP",STR0005+" "+ AllTrim(SA1->A1_COD) + "/" + AllTrim(SA1->A1_LOJA) + " - " + AllTrim(SA1->A1_NOME) + ".") //"0 CPF informado já foi
        utilizado no cliente"
105     STFShowMessage("STBVldCGC")
106     lRetorno := .F.
107 Endif
108 EndIf
109 EndIf
110 EndIf
111
112 //ÚÁÀÄÅÇÈÉÊËÌÍÎÏÐÑÒÓÔÕÖ×ØÙÚÛÜÝÞßàáâãäåæçèéêëìíîïðñòóôõö÷øùúûüýþÿ
113 //³Valida a existencia do CNPJ/CPF nos cadastros de suspect³
114 //³e prospect³
115 //ÁÀÃÄÅÆÇÈÉÊËÌÍÎÏÐÑÒÓÔÕÖ×ØÙÚÛÜÝÞß
116 If lRetorno .AND. (nModulo == 5 .or. nModulo == 13) .And. !lEleMesmo
117     If (cTipPes == "F" .And. cMv_ValCPF <> "1") .Or. ((cTipPes == "J" .Or. Empty(cTipPes)) .And. cMv_ValCNPJ <> "1")
118         If !Empty(cCNPJ)
119             lRetorno := TmkVeEnt(cCNPJ,"SA1")
120         EndIf
121     EndIf
122 EndIf
123
124 RestArea(aAreaSA1)
125 RestArea(aArea)
126 Return lRetorno
127
128 //-----
129 /*{Protheus.doc} STBVldRETCLI
130 Valida se o cliente existe na retaguarda.
131
132 @param
133 @return lRet          Retorno .T. se nao houver registro com a mesma chave unica na base.
134 @obs
135 @sample
136 */
137 //-----
138 Function STBVldRETCLI(cChave)
139 Local lRet := .F.
140
141 If SA1->(DbSeek(xFilial()+cChave))
142     lRet := .T.
143 EndIf
144
145 Return lRet
146
147 //-----
148 /*{Protheus.doc} STBCGCDigVerificador
149 Processa o criterio de validacao dos campos, verificando se o dígito verificador do CPF/CNPJ e valido.
150
151 @param      cCGC - Numero do CGC;
```

```

152          cVar - Variavel de Memoria a ser retornada;
153          lAviso - Valor Logico de Retorno (.T. ou .F.)
154
155 @return      lRet          Retorno .T. se nao houver registro com a mesma chave unica na base.
156 @obs
157 @sample
158 /*/
159 //-----
160 Function STBCGCDigVerificador(cCGC, cVar, lAviso)
161 LOCAL nCnt,i,j,cDVC,nSum,nDIG,cDIG:="",cSavAlias,nSavRec,nSav0rd
162 Local cFormAnt := cCGC
163
164 DEFAULT lAviso := .T.
165
166 STFMesssage("STBCGCDigVerificador","STOP",STR0001) //"Dígito verificador incorreto, entre com um código válido."
167
168 cCGC := strtran(cCGC, ".", "")
169 cCGC := strtran(cCGC, "-", "")
170 cCGC := strtran(cCGC, "/", "")
171
172 cCGC := IIF(cCgc == Nil,&(ReadVar()),cCGC)
173 cVar := If(ValType(cVar) = "U", ReadVar(), cVar)
174
175 If cCgc == "0000000000000000"
176     Return .T.
177 Endif
178
179 nTamanho:=Len(AllTrim(cCGC))
180
181 cDVC:=SubStr(cCGC,13,2)
182 cCGC:=SubStr(cCGC,1,12)
183
184 FOR j := 12 TO 13
185     nCnt := 1
186     nSum := 0
187     FOR i := j TO 1 Step -1
188         nCnt++
189         IF nCnt>9;nCnt:=2;EndIf
190         nSum += (Val(SubStr(cCGC,i,1))*nCnt)
191     Next i
192     nDIG := IIF((nSum%11)<2,0,11-(nSum%11))
193     cCGC := cCGC+STR(nDIG,1)
194     cDIG := cDIG+STR(nDIG,1)
195 Next j
196 lRet:=IIF(cDIG==cDVC,.T.,.F.)
197
198 IF !lRet
199     IF nTamanho < 14
200         cDVC:=SubStr(cCGC,10,2)
201         cCPF:=SubStr(cCGC,1,9)
202         cDIG:=""
203

```

```
204     FOR j := 10 TO 11
205         nCnt := j
206         nSum := 0
207         For i:= 1 To Len(Trim(cCPF))
208             nSum += (Val(SubStr(cCPF,i,1))*nCnt)
209             nCnt--
210         Next i
211         nDIG:=IIF((nSum%11)<2,0,11-(nSum%11))
212         cCPF:=cCPF+STR(nDIG,1)
213         cDIG:=cDIG+STR(nDIG,1)
214     Next j
215
216     IF cDIG != cDVC .And. lAviso
217         STFSHOWMESSAGE("STBCGCDigVerificador")
218     Endif
219
220     lRet:=IIF(cDIG==cDVC,.T.,.F.)
221     IF lRet;&cVar:=cCPF+Space(3);ENDIF
222 Else
223     If lAviso
224         STFSHOWMESSAGE("STBCGCDigVerificador")
225     EndIf
226 EndIF
227 EndIF
228
229 &(ReadVar()) := cFormAnt
230
231 Return lRet
```

Componentes/Customer/STDSearchCostumer.prw

606 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "FWMBROWSE.CH"
4 #INCLUDE "FWMVCDEF.CH"
5 #INCLUDE "PROTHEUS.CH"
6 #INCLUDE "APWIZARD.CH"
7 #INCLUDE "FWADAPTEREAI.CH"
8 #INCLUDE "STPOS.CH"
9
10
11 Static oModelCli := Nil
12 Static oStructSA1 := Nil
13
14 //-----
15 /*{Protoheus.doc} STDSearchCostumer
16 Função responsável por realizar busca de um determinado cliente através do conteúdo informado na variável cWhatSearch ou cCardCrd em caso de integração CRD
17
18 @param      cWhatSearch, Caracter, Conteúdo a ser buscado
19 @param      nLimitRegs, Numérico, Limite de clientes buscado
20 @param      lSearchLike, Lógico, indica se fara busca Like (Apenas para Ctree, Banco de dados relacional sempre realiza busca like)
21 @param      aFields, Array, Campos a serem utilizados na query e devolvidos no retorno
22 @param      lIntCRD, Lógico, Indica se o CRD esta ativo, controla alguns desvios na busca
23 @param      cCardCrd, Caracter, Em ambientes com CRD ativo busca também por Cartão private label
24 @return     Array, Retorna informações sobre os clientes buscado
25 */
26 //-----
27
28 Function STDSearchCostumer(cWhatSearch,nLimitRegs,lSearchLike,aCustomer,aFields,lIntCRD,cCardCrd)
29 Local cSGBD      := "" // -- Banco de dados atualizado (Para ambientes TOP)
30 Local cAliasQuery := "" // -- Alias utilizado para query
31 Local nCont      := 0 // -- Contador de clientes adicionados no retorno (Para ambiente CTREE/DBF)
32
33 Local aDataCustomers := {} // -- Armazenas dados dos clientes buscados (Campos solicitados através do parametro aFields)
34 Local aCustomers     := {} // -- Array com String a ser exibida no TOTVS PDV (NOME / CODIGO / LOJA / CPF / CARTÃO CRD)
35 Local aCardMA6       := {} // -- Dados do cartão CRD (caso exista e esteja com o CRD ativo (lIntCRD = .T. ))
36
37 Local xData          := Nil // -- Conteúdo do campo que esta sendo adicionado no array xData
38 Local nFields        := 0 // -- Variável utilizada em FOR
39 Local nSeek          := 0 // -- Campo de controle para SEEK, baseado no valor dele é indicado qual indice da tabela sera utilizado
40 Local cFilter        := "" // -- Filtro utilizado em busca CTREE / DBF
41
42 Local cStartQuery    := "" // -- Variável para montagem de Query
43 Local cBodyQuery     := "" // -- Variável para montagem de Query
44 Local cEndQuery      := "" // -- Variável para montagem de Query
45 Local cFullQuery     := "" // -- Variável para montagem de Query
46 Local aFieldSeek     := {"SA1->A1_COD","SA1->A1_NOME","SA1->A1_CGC"} // -- Campos que serão comparados baseados no valor de nSeek (Corresponde ao primeiro valor do indice)
47 Local nTamA1Cod      := TamSX3("A1_COD")[1]
48
49 Default cWhatSearch  := "" // -- Conteúdo a ser buscado
50 Default nLimitRegs   := SuperGetMV("MV_LJQTDPL",,20) // -- Limite de clientes buscado (Recebe do solicitante, caso não exista busca do parametro)
51 Default lSearchLike  := .F. // -- Indica se fara busca Like
52 Default aCustomer    := {} // -- Dados com código e loja do cliente em casos de busca especifica.

```

```

53 Default aFields      := {}
54 Default lIntCRD      := .F.
55 Default cCardCrd     := ""
56
57
58 //Quando aCustomer esta preenchido indica que a busca deverá ser exata, buscando cliente + loja e retornando apenas 1 registro
59 If !lSearchLike .AND. Len(aCustomer) > 0
60     nLimitRegs := 1
61 EndIf
62
63 #IFDEF TOP
64
65     cSGBD      := AllTrim(Upper(TcGetDb()))
66     cAliasQuery := GetNextAlias()
67
68     cStartQuery += " SELECT "
69
70     If cSGBD      $ "MSSQL|SYBASE"
71         cStartQuery += " TOP " + AllTrim(Str(nLimitRegs))
72     ElseIf cSGBD  $ "INFORMIX"
73         cStartQuery += " FIRST " + AllTrim(Str(nLimitRegs))
74     EndIf
75
76     If (nLenFields := Len(aFields)) > 0
77         For nFields := 1 To nLenFields
78             cStartQuery += aFields[nFields] + ","
79         Next nFields
80
81         If lIntCRD
82             cStartQuery += " MA6_NUM,"
83         EndIf
84
85         cStartQuery := Left(cStartQuery,Len(cStartQuery)-1)
86     EndIf
87
88     cBodyQuery += " FROM " + RetSQLName("SA1") + " SA1 "
89     cBodyQuery += " LEFT JOIN " + RetSQLName("AI0") + " AI0 ON AI0.AI0_FILIAL = '" + xFilial("AI0") + "' AND SA1.A1_COD = AI0.AI0_CODCLI AND SA1.A1_LOJA = AI0.AI0_LOJA AND
90     AI0.D_E_L_E_T_ = ' ' "
91
92     If lIntCRD
93         cBodyQuery += " LEFT JOIN " + RetSQLName("MA6") + " MA6 ON MA6_FILIAL = '" + xFilial("MA6") + "' AND SA1.A1_COD = MA6_CODCLI AND SA1.A1_LOJA = MA6.MA6_LOJA AND
94         MA6.D_E_L_E_T_ = ' ' "
95     EndIf
96
97     cBodyQuery += " WHERE SA1.A1_FILIAL = '" + xFilial("SA1") + "' "
98
99     If cSGBD      $ "ORACLE"
100         cBodyQuery += " AND ROWNUM <= " + AllTrim(Str(nLimitRegs))
101     ElseIf cSGBD  $ "DB2"
102         cEndQuery   += " FETCH FIRST " + AllTrim(Str(nLimitRegs)) + " ROWS ONLY"
103     ElseIf cSGBD  $ "POSTGRES|MYSQL|SQLITE"
104         cEndQuery   += " LIMIT " + AllTrim(Str(nLimitRegs))

```

```

103 EndIf
104
105 If !Empty(cCardCrd)
106   cBodyQuery += " AND (MA6.MA6_NUM = '' + cCardCrd + '' ) "
107 ElseIf !lSearchLike .AND. Len(aCustomer) > 0
108   cBodyQuery += " AND SA1.A1_COD = '' + aCustomer[1] + '' "
109   cBodyQuery += " AND SA1.A1_LOJA = '' + aCustomer[2] + '' "
110 ElseIf ExistFunc("STBCPFCNPJ") .AND. STBCPFCNPJ(cWhatSearch) //Valida se é CPF/CNPJ, caso sim consulta por A1_CGC para melhora de performance.
111   cBodyQuery += " AND SA1.A1_CGC = '' + cWhatSearch + '' "
112 ElseIf Len(Alltrim(cWhatSearch))<= nTamA1Cod //Se o tamanho for igual ou menor que o tamanho do campo A1_COD, inclui na consulta
113   IF Isnumeric(cWhatSearch)
114     cBodyQuery += " AND (SA1.A1_COD LIKE '%" + cWhatSearch + "%' OR SA1.A1_CGC LIKE '%" + cWhatSearch + "%' OR SA1.A1_NOME LIKE '%" + cWhatSearch + "%' ) "
115   Else
116     cBodyQuery += " AND (SA1.A1_COD LIKE '%" + cWhatSearch + "%' OR SA1.A1_NOME LIKE '%" + cWhatSearch + "%' ) "
117   EndIf
118 ElseIf Isnumeric(cWhatSearch) // se for maior q tamanho do campo A1_COD e não é CPF/CNPJ válido e for numérico, então inclui na consulta
119   cBodyQuery += " AND (SA1.A1_CGC LIKE '%" + cWhatSearch + "%' OR SA1.A1_NOME LIKE '%" + cWhatSearch + "%' ) "
120 Else //se não, consulta o campo A1_NOME
121   cBodyQuery += " AND SA1.A1_NOME LIKE '%" + cWhatSearch + "%' "
122 EndIf
123
124 cBodyQuery += " AND SA1.A1_MSBLQL <> '1' "
125 cBodyQuery += " AND SA1.D_E_L_E_T_ = ' ' "
126
127 cFullQuery := ChangeQuery(cStartQuery + cBodyQuery + cEndQuery)
128
129 DbUseArea(.T., "TOPCONN", TcGenQry(, cFullQuery), cAliasQuery, .T., .T.)
130
131 // -- Atualiza o tipo do campo na tabela de memoria baseado no tipo do campo no dicionario.
132 For nFields := 1 To nLenFields
133   aStructField := TamSX3(aFields[nFields])
134   If aStructField[3] $ 'DNL'
135     TCSetField(cAliasQuery, aFields[nFields], aStructField[3], aStructField[1], aStructField[2] )
136   Endif
137 Next
138
139 While (cAliasQuery)->(!EOF())
140
141   //-- Posição 1 do retorno, String contendo NOME / CODIGO / LOJA / CPF / CARTÃO CRD
142   aAdd(aCustomers, AllTrim((cAliasQuery)->A1_NOME)+" / "+AllTrim((cAliasQuery)->A1_COD)+" / "+AllTrim((cAliasQuery)->A1_LOJA)+" / "+AllTrim((cAliasQuery)->A1_CGC) +
IIF(lIntCRD, " / " + AllTrim((cAliasQuery)->MA6_NUM), ""))
143
144   // -- Posição 2 do retorno, Array contendo o conteudo dos campos solicitados
145   aAdd(aDataCustomers, {})
146   For nFields := 1 To nLenFields
147     xData := (cAliasQuery)->&(aFields[nFields])
148     If !SubStr(aFields[nFields], 1, 4) == "AI0_" .Or. ( SubStr(aFields[nFields], 1, 4) == "AI0_" .AND. !Empty(xData) )
149       aAdd(aDataCustomers[Len(aDataCustomers)], {aFields[nFields], xData} )
150     EndIf
151   Next
152
153   // -- Posição 3 do retorno, contem o numero do cartão private label CRD caso exista.

```

```

154      If lIntCRD
155          aAdd(aCardMA6,(cAliasQuery)->MA6_NUM)
156      EndIf
157
158      (cAliasQuery)->(DbSkip())
159
160  End
161
162  (cAliasQuery)->(DbCloseArea())
163
164 #ELSE
165
166  DbSelectArea("SA1")
167
168  If lSearchLike
169
170      cFilter := " ('" + cWhatSearch + "' $ SA1->A1_NOME .OR. '" + cWhatSearch + "' $ SA1->A1_CGC .OR. '" + cWhatSearch + "' $ SA1->A1_COD ) .AND. SA1->A1_MSBLQL <> '1' "
171
172      SA1->(DbSetFilter({ || &cFilter }, cFilter))
173      SA1->(DbGoTop())
174
175      While SA1->(!EOF())
176
177          nCont++
178
179          //-- Posição 1 do retorno, String contendo NOME / CODIGO / LOJA / CPF / CARTÃO CRD
180          aAdd(aCustomers,AllTrim(SA1->A1_NOME)+" / "+AllTrim(SA1->A1_COD)+" / "+AllTrim(SA1->A1_LOJA)+" / "+AllTrim(SA1->A1_CGC))
181
182          // -- Posição 2 do retorno, Array contendo o conteúdo dos campos solicitados
183          aAdd(aDataCustomers,{})
184          For nFields := 1 To Len(aFields)
185              If SubStr(aFields[nFields],1,3) == "A1_"
186                  aAdd(aDataCustomers[Len(aDataCustomers)],{aFields[nFields],&("SA1->" + aFields[nFields])})
187              EndIf
188          Next
189
190          If nCont == nLimitRegs
191              Exit
192          EndIf
193
194          SA1->(DbSkip())
195      EndDo
196
197      SA1->(DbClearFilter())
198  Else
199
200      For nSeek := 1 To Len(aFieldSeek)
201          SA1->(DbSetOrder(nSeek)) // -- 1 = Filial + Cod, 2 = Filial + Nome, 3 = Filial + CGC.
202          If SA1->(DbSeek(xFilial("SA1") + cWhatSearch))
203
204              While AllTrim(cWhatSearch) $ AllTrim(&(aFieldSeek[nSeek]))
205

```

```

206         If SA1->A1_MSBLQL <> '1'
207             cCustomer := AllTrim(SA1->A1_NOME)+" / "+AllTrim(SA1->A1_COD)+" / "+AllTrim(SA1->A1_LOJA)+" / "+AllTrim(SA1->A1_CGC)
208
209             If aScan(aCustomers,{|x| x == cCustomer}) == 0
210                 nCont++
211
212                 //-- Posição 1 do retorno, String contendo NOME / CODIGO / LOJA / CPF / CARTÃO CRD
213                 aAdd(aCustomers,cCustomer)
214
215                 //-- Posição 2 do retorno, Array contendo o conteúdo dos campos solicitados
216                 aAdd(aDataCustomers,{})
217                 For nFields := 1 To Len(aFields)
218                     If SubStr(aFields[nFields],1,3) == "A1_"
219                         aAdd(aDataCustomers[Len(aDataCustomers)],{aFields[nFields],&("SA1->" + aFields[nFields])} )
220                     EndIf
221                 Next
222
223                 If nCont == nLimitRegs
224                     Exit
225                 EndIf
226             EndIf
227         EndIf
228         SA1->(DbSkip())
229     End
230
231     If Len(aCustomers) > 0
232         Exit
233     EndIf
234
235 EndIf
236 Next
237 EndIf
238
239 #ENDIF
240
241 Return {aCustomers,aDataCustomers,aCardMA6}
242
243 //-------------------------------------
244 /*/{Protheus.doc} STDWhatfields()
245 Função responsável por determinar quais campos serão utilizados na importação
246
247 @return      Array, Array contendo campos a serem importados.
248 */
249 //-------------------------------------
250
251 Function STDWhatfields()
252 Local aFields      := {}    // -- Campos para a importação
253 Local aRetPE       := {}    // -- Campos do ponto de entrada
254 Local nI           // -- Variável para for
255
256 aAdd(aFields,'A1_NOME')
257 aAdd(aFields,'A1_COD')

```

```

258 aAdd(aFields,'A1_LOJA')
259 aAdd(aFields,'A1_CGC')
260 aAdd(aFields,'A1_NREDUZ')
261 aAdd(aFields,'A1_PESSOA')
262 aAdd(aFields,'A1_TIPO')
263 aAdd(aFields,'A1_END')
264 aAdd(aFields,'A1_EST')
265 aAdd(aFields,'A1_MUN')
266 aAdd(aFields,'A1_BAIRRO')
267 aAdd(aFields,'A1_DDD')
268 aAdd(aFields,'A1_TEL')
269 aAdd(aFields,'A1_CEP')
270 aAdd(aFields,'A1_COD_MUN')
271 aAdd(aFields,'A1_MSBLQL')
272 aAdd(aFields,'A1_DTNASC')
273 aAdd(aFields,'A1_FILIAL')
274 aAdd(aFields,'A1_GRPVEN')
275 aAdd(aFields, "AI0_CLIFUN")
276
277 //Ponto de entrada para retornar campos adicionais no aFields
278 If ExistBlock("STSELFIEld")
279     LjGrvLog(," STDWhatfields - Antes de chamar o Ponto de Entrada STSELFIEld",,)
280     aRetPE := ExecBlock("STSELFIEld",.F.,.F.,{"SA1"})
281     LjGrvLog(," STDWhatfields - Depois de chamar o Ponto de Entrada STSELFIEld",aRetPE)
282 EndIf
283
284 For nI := 1 To Len(aRetPE)
285     aAdd(aFields, aRetPE[nI])
286 Next nI
287
288 Return aFields
289 //-----
290 /*{Protheus.doc} STDCustomerData
291 Cria estrutura de dados do Model de clientes, sem suas validacoes.
292 @param cKey          Chave de Pesquisa. Contém Filial, Cliente e Loja.
293 @param lOffline      Pesquisa offline
294 @return xRet - Retorna estrutura de dados do Model
295 @obs
296 @sample
297 */
298 //-----
299 Function STDCustomerData( cKey , lOffline )
300
301 Local aArea      := GetArea()           // Guarda area
302 Local oStruct    := Nil                 // Estrutura
303 Local xRet       := IIF(lOffline,Nil,{}) // Se a busca for offline, sera retornado um model. Caso contrario, sera retornado um array
304 Local aCampos    := {}                  // Array de campos
305 Local nX         := 0                   // Contador
306
307 Default cKey      := ""
308 Default lOffline  := .F.
309

```

```

310
311 ParamType 0 Var cKey      AS Character   Default ""
312 ParamType 1 var lOffline  As Logical     Default .F.
313
314 /*/
315     Monta estrutura de tabela de clientes
316 /*/
317 oStruct := NoVldStruct()
318
319 aCampos := oStruct:GetFields()
320
321 If lOffline
322     /*/
323         Instacia Objeto
324     /*/
325     xRet := MPFormModel():New( 'SA1', /*bPreValidacao*/, /*bPosValidacao*/, /*bCommit*/, /*bCancel*/ )
326
327     xRet:AddFields("SA1MASTER",/*cOwner*/,oStruct)
328     xRet:SetDescription("Customers")
329     xRet:SetOperation(3)
330     xRet:DeActivate()
331     xRet:Activate()
332 EndIf
333
334
335 /*
336     Preenche o Modelo de dados do cliente com as informacoes da tabela SA1
337 */
338
339 DbSelectArea("SA1")
340 DbSetOrder(1)//A1_FILIAL+A1_COD+A1_LOJA
341 If DbSeek ( cKey )
342     For nX := 1 to Len(aCampos)
343         If lOffline
344             xRet:SetValue('SA1MASTER', aCampos[nX][MODEL_FIELD_IDFIELD], &("SA1->" + aCampos[nX][MODEL_FIELD_IDFIELD]))
345         Else
346             AAdd(xRet,{aCampos[nX][MODEL_FIELD_IDFIELD],&("SA1->" + aCampos[nX][MODEL_FIELD_IDFIELD])})
347         EndIf
348     Next nX
349 EndIf
350
351 If lOffline
352     If ValType(oModelCli) == '0'
353         oModelCli:DeActivate()
354         oModelCli := Nil
355     EndIf
356
357     oModelCli := xRet
358 EndIf
359
360
361 RestArea(aArea)

```

```

362
363 Return xRet
364
365
366 //-----
367 /*{Protheus.doc} NoVldStruct
368 Cria estrutura do Model de clientes, sem suas validacoes.
369 @param
370 @return oStruct - retorna estrutura do Model
371 @obs
372 @sample
373 */
374 //-----
375 Static Function NoVldStruct()
376
377 Local aArea      := Nil
378 Local cAlias     := "SA1"                // Alias que criara estrutura
379 Local nCount     := 0
380 Local cX2Unico   := "A1_FILIAL+A1_COD+A1_LOJA" //X2_UNICO da tabela SA1
381
382 If ValType(oStructSA1) = '0'
383     oStructSA1:Deactivate()
384     oStructSA1:Activate()
385 Else
386
387     oStructSA1 := FWFormModelStruct():New() // Estrutura
388
389     aArea := GetArea() // Guarda area
390     //Carrega informacoes da Tabela
391     SX2->( DbSetOrder( 1 ) )
392     SX2->( DbSeek( cAlias ) )
393
394     If ExistFunc('FWX2Unico')
395         cX2Unico := FWX2Unico(cAlias)
396     EndIf
397
398     oStructSA1:AddTable(
399         FWX2CHAVE()           , ; // [01] Alias da tabela
400         StrTokArr( cX2Unico, '+' ) , ; // [02] Array com os campos que correspondem a primary key
401         FWX2Nome(cAlias)      ) // [03] Descrição da tabela
402
403
404     //Carrega informacoes de campos
405     DbSelectArea("SX3")
406     SX3->(DbSetOrder(1))
407     SX3->(DbSeek(cAlias))
408
409     While !SX3->(EOF()) .AND. SX3->X3_ARQUIVO == cAlias
410
411         nCount++
412         If nCount > 1000
413             Conout('NoVldStruct - Count>1000',nCount)

```

```

414     EndIf
415
416     If Upper(AllTrim(SX3->X3_TIPO)) <> "M" .AND. SX3->X3_CONTEXT <> 'V'
417         oStructSA1:AddField(
418             AllTrim( X3Titulo() )           , ; // [01] Titulo do campo
419             AllTrim( X3Descric() )         , ; // [02] ToolTip do campo
420             AllTrim( SX3->X3_CAMPO )       , ; // [03] Id do Field
421             SX3->X3_TIPO                    , ; // [04] Tipo do campo
422             SX3->X3_TAMANHO                 , ; // [05] Tamanho do campo
423             SX3->X3_DECIMAL                 , ; // [06] Decimal do campo
424             Nil                             , ; // [07] Code-block de validacao do campo
425             Nil                             , ; // [08] Code-block de validacao When do campo
426             StrTokArr( AllTrim( X3CBox() ),';') , ; // [09] Lista de valores permitido do campo
427             Nil                             , ; // [10] Indica se o campo tem preenchimento obrigatorio
428             Nil                             , ; // [11] Code-block de inicializacao do campo
429             NIL                             , ; // [12] Indica se trata-se de um campo chave
430             NIL                             , ; // [13] Indica se o campo pode receber valor em uma operacao de update.
431             ( SX3->X3_CONTEXT == 'V' )      ) // [14] Indica se o campo e virtual
432     EndIf
433
434     SX3->(DbSkip())
435 End
436
437 RestArea(aArea)
438 EndIf
439
440 Return oStructSA1
441
442
443 //-----
444 /* {Protheus.doc} STDGcliModel
445 Funcao que retorna o Model do cliente preenchido.
446
447 @return oModelCli - Model do Cliente
448 @obs
449 @sample
450 */
451 //-----
452
453 Function STDGcliModel()
454 Return oModelCli
455
456
457 //-----
458 /* {Protheus.doc} STDFilPBCustomerData
459 Funcao que retorna o Model do cliente preenchido.
460 @param cCliente      Cliente
461 @param cLoja          Loja
462 @return Nil
463 @obs
464 @sample
465 */

```

```

466 //-----
467 Function STDFilPBCustomerData( cCliente , cLoja )
468
469 Default cCliente      := ""
470 Default cLoja         := ""
471
472 ParamType 0 Var cCliente      AS Character      Default ""
473 ParamType 1 var cLoja        As Character      Default ""
474
475 STDSPBasket("SL1","L1_CLIENTE" ,oModelCli:GetValue("SA1MASTER","A1_COD"))
476 STDSPBasket("SL1","L1_LOJA"    ,oModelCli:GetValue("SA1MASTER","A1_LOJA"))
477 STDSPBasket("SL1","L1_TIPOCLI" ,oModelCli:GetValue("SA1MASTER","A1_TIPO"))
478
479 Return Nil
480
481
482 //-----
483 /* {Protheus.doc} STDFilPBCustomerData
484 Funcao que retorna o Model do cliente preenchido.
485 @param aDados - array de dados do cliente
486 @return oModel - Model do cliente
487 @obs
488 @sample
489 */
490 //-----
491 Function STDFilCliData( aDados )
492
493 Local oStruct      := Nil      // Estrutura
494 Local oModel       := Nil      // Model do Cliente
495 Local nX           := 0        // Contador
496
497 Default aDados     := {}
498
499 ParamType 0 Var aDados AS Array Default {}
500
501 /*
502   Monta estrutura de tabela de clientes
503 */
504 oStruct := NoVldStruct()
505
506 /*
507   Instacia Objeto
508 */
509 oModel := MPFormModel():New( 'SA1', /*bPreValidacao*/, /*bPosValidacao*/, /*bCommit*/, /*bCancel*/ )
510
511 oModel:AddFields("SA1MASTER",/*cOwner*/,oStruct)
512 oModel:SetDescription("Customers")
513 oModel:SetOperation(3)
514 oModel:Activate()
515
516 For nX := 1 to Len(aDados)
517   oModel:LoadValue('SA1MASTER', aDados[nX][FIELD_NAME], aDados[nX][FIELD_VALUE])

```

```

518 Next nX
519
520 oModelCli := oModel
521
522 Return oModel
523
524
525 //-----
526 /*{Protheus.doc} STDFindCust()
527 Pesquisa o cliente em especifico
528
529 @param
530 @return aRet
531 @obs
532 @sample
533 */
534 //-----
535 Function STDFindCust()
536
537 Local aRet      := {}
538 Local cCustomer := STDGPBasket("SL1","L1_CLIENTE")
539 Local cLoja     := STDGPBasket("SL1","L1_LOJA")
540
541 SA1->(dbSetOrder(1))
542 SA1->(dbSeek(xFilial('SA1')+cCustomer+cLoja))
543 If SA1->(!EOF())
544
545     Aadd(aRet,SA1->A1_NOME)
546     Aadd(aRet,SA1->A1_CGC)
547
548 EndIf
549
550 Return aRet
551
552 //-----
553 /*{Protheus.doc} STDCliRecCGC()
554 Retorna o Recno do cliente pela busca por CGC
555
556 @param    cCGC recebe CPF ou CNPJ
557 @return   nRecno Recno da busca
558 @obs
559 @sample
560 */
561 //-----
562 Function STDCliRecCGC(cCGC)
563
564 Local aArea := GetArea()// Guarda area
565 Local nRecno := 0 //Recno
566
567 Default cCGC := ""
568
569 DbSelectArea("SA1")

```

```

570 SA1->(DbSetOrder(3)) //FILIAL + CGC
571 If DbSeek(xFilial("SA1")+AllTrim(cCGC))
572     nRecno := SA1->(Recno())
573 EndIf
574
575 RestArea(aArea)
576
577 Return nRecno
578
579 //-----
580 /*{Protheus.doc} STDVerfCadCli()
581 Verificar cadastrdo do cliente
582 @param cKey COD + LOJA
583 @return Array, Retorno contendo se achou e recno
584 */
585 //-----
586 Function STDVerfCadCli(cKey)
587
588 Local aArea := GetArea()// Guarda area
589 Local lRet := .F.
590 Local nRecno := 0
591
592 Default cKey := ""
593
594 DbSelectArea("SA1")
595 DbSetOrder(1) //FILIAL + COD + LOJA
596
597 If DbSeek( xFilial("SA1") + cKey )
598     nRecno := SA1->(Recno())
599     lRet := .T.
600 Else
601     lRet := .F.
602 EndIf
603
604 RestArea(aArea)
605
606 Return {lRet,nRecno}

```

Componentes/Customer/STWCustomerSelection.prw

145 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STPOS.CH"
4 #INCLUDE "STWCustomerSelection.CH"
5
6
7 //-----
8 /* {Protheus.doc} STWCustomerSelection
9 Workflow responsavel pela selecao de clientes.
10 @param cKey          - Chave para busca no Back-Office
11 @return oModelCliente - Retorna model do cliente
12 */
13 //-----
14 Function STWCustomerSelection(cKey,cOrc, aMDJ)
15
16 Local loffline      := .T.                // Por questoes tecnicas, a busca pelo cliente sera feita apenas offline, porem a busca online ja esta implementada
17 Local aDadosCli     := {}                 // Array de dados do cliente
18 Local aParam        := {}                 // Array de parametros
19 Local cCliente      := ""                 // Cliente
20 Local cLojaCli       := ""                 // Loja do cliente
21 Local cCliPad        := SuperGetMv("MV_CLIPAD") // Cliente padrao
22 Local cLojaPad       := SuperGetMv("MV_LOJAPAD") // Loja padrao
23 Local oModelCliente := Nil                 // Model de cliente
24 Local uResult        := Nil                 // Resultado generico
25 Local lSelNcc         := ExistBlock("STSelNcc") //Existe ponto de entrada para selecao da NCC?
26 Local aSelNcc         := {}                 //Recebe o retorno do ponto de entrada STSelNcc
27 Local lImport         := .F.                 //Variavel de controle para importação de orçamento com NCC
28
29 Default cKey          := ""
30 Default cOrc          := ""
31 Default aMDJ          := {}
32
33 ParamType 0 var cKey          As Character      Default ""
34
35 If !loffline // Busca na retaguarda
36
37     While .T.
38         // Trecho para recebimento do model do cliente
39
40         aParam := {xFilial("SA1")+cKey,loffline}
41
42         // Tratamento do erro de conexao
43         If !STBRemoteExecute("STDCustomerData", aParam,,, @uResult)
44
45
46             //T0 D0: STFMessage(ProcName(), "STOP", oServer:cStatusDescription)
47             STFMessage(ProcName(), "ALERT", STR0001 ) //"Por falta de comunicação com o Back-Office, será selecionado o cliente padrão."
48
49             // Como houve falha na conexao, sera selecionado o cliente padrao e sera iniciado o processo offline.
50             cKey          := cCliPad+cLojaPad
51             loffline      := .T.
52

```

```

53
54         //Esse Exit sai do processo Online, iniciando logo em seguida o processo offline,
55         //onde sera setado na cesta os dados do cliente padrao.
56         Exit
57
58     Else
59         oModelCliente := STDFilCliData(uResult)
60         uResult := Nil
61     EndIf
62
63     //TO DO: RECEBER OS DADOS DO CLIENTE DA RETAGUARDA E ATUALIZAR AS INFORMACOES NO PDV.
64     cCliente := oModelCliente:GetValue("SAIMASTER","A1_COD")
65     cLojaCli := oModelCliente:GetValue("SAIMASTER","A1_LOJA")
66
67     Exit
68 End
69 EndIf
70
71
72 If lOffline
73     // Atribuicao do model do cliente à variavel de Retorno.
74
75     oModelCliente := STDCustomerData(xFilial("SA1")+cKey,lOffline)
76     cCliente := oModelCliente:GetValue("SAIMASTER","A1_COD")
77     cLojaCli := oModelCliente:GetValue("SAIMASTER","A1_LOJA")
78
79 EndIf
80
81 If Empty(cCliente) .AND. Empty(cLojaCli) .AND. !Empty(cKey)
82     cCliente := SubStr(cKey,1,Len(CriaVar("A1_COD")))
83     cLojaCli := SubStr(cKey,Len(CriaVar("A1_COD")) + 1,Len(CriaVar("A1_LOJA")))
84 EndIf
85
86
87 If Substr(SuperGetMV("MV_USACRED"),2,1) == "S" .AND. (AllTrim(cCliente)+AllTrim(cLojaCli)) <> (AllTrim(cCliPad)+AllTrim(cLojaPad))
88     /*
89     Trecho para recebimento do model das NCCs do cliente
90     */
91
92     //Como será utilizado o cliente do PDV listamos as NCCs do cliente informado no PDV.
93     If (!Empty(cOrc) .And. SuperGetMV("MV_LJIPCOR",.F.,0) <> 1 )
94         lImport := .T.
95     EndIf
96     aParam := {cCliente,cLojaCli,lImport,cOrc}
97     If lSelNcc
98         aSelNcc := ExecBlock("STSelNcc",.F.,.F.,aParam)
99         If Len(aSelNcc) > 0
100             STDSetNCCs("1",aSelNcc)
101             aSelNcc := {}
102         EndIf
103     Else
104

```

```

105      If (LjGetCPDV()[1] .OR. LjGetCPDV()[2]) .AND. Len(aMDJ)>0
106          aParam := {cCliente, cLojaCli, lImport, cOrc, aMDJ}
107      Endif
108
109      If !STBRemoteExecute("STDFindNCCModel", aParam,,, @uResult)
110          // Tratamento do erro de conexao
111          STFMessage(ProcName(), "ALERT", STR0002 ) //"Por falta de comunicação com o Back-Office, as NCCs do cliente não serão carregadas."
112          STFShowMessage(ProcName())
113      Else
114          STDSetNCCs("1",uResult)
115          uResult := Nil
116      EndIf
117  EndIf
118 EndIf
119
120 Return oModelCliente
121
122 //-----
123 /*{Protheus.doc} STBValidCliPad
124 Funcao que verifica se o cliente digitado pelo usuario eh o cliente padrao. Caso seja, seta lSearchOffline para True,
125 a fim de forçar todo o processo de selecao de clientes para ocorrer em ambiente local.
126 @param  cKey          - Chave para busca no Back-Office
127 @return  lRet          - Retorna True caso o cliente digitado seja o cliente padrao, o que força a pesquisa em ambiente local.
128 */
129 //-----
130
131 Function STBValidCliPad( cKey )
132
133 Local cCliPad          := SuperGetMv( "MV_CLIPAD" )           //Cliente Padrao
134 Local cLojaPad         := SuperGetMV( "MV_LOJAPAD" )         //Loja Padrao do cliente
135 Local lRet             := .F.                                //Retorno
136
137 Default cKey          := ""
138
139 ParamType 0 var  cKey          As Character      Default ""
140
141 If (cKey == cCliPad+cLojaPad)
142     lRet := .T.
143 EndIf
144
145 Return lRet

```

Componentes/Customer/STWInfoCNPJ.PRW

240 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STWInfoCNPJ.CH"
4 #INCLUDE "POSCSS.CH"
5
6 Static lInfCPF      := .F. // Indica se a tela de CPF ja foi apresentada ao cliente.
7
8 //-----
9 /*{Protheus.doc}STWInfoCNPJ
10 Informar CNPJ do Cliente
11 @param1      ExpC1 - Codigo Cliente
12 @param2      ExpC2 - Loja do Cliente
13 @param3      ExpC3 - Nome do Cliente
14 @param4      ExpC4 - Endereco do Cliente
15 @param5      ExpL1 - Informa dados
16 @param6      ExpL2 - Informa Endereco
17 @return      Nil
18 @obs
19 @sample
20 */
21 //-----
22 Function STWInfoCNPJ(lInfoCnpj,lTelaFimVnd,lObrijaLJ950)
23 Local lRet      := .F.
24 Local cParCgc   := SPACE(TamSX3("A1_CGC")[1]) // Variavel criada para enviar cgc
25 Local oCliModel := STDGCLiModel() // Model do Cliente
26 Local lPrintCNPJ := .T. // Indica se o CNPJ sera impresso na nota
27 Local cCliente  := oCliModel:GetValue("SA1MASTER","A1_COD")
28 Local cLojaCli  := oCliModel:GetValue("SA1MASTER","A1_LOJA")
29 Local cEndCli   := ""
30 Local cNomeCli  := ""
31 Local cDocCliSL1 := ""
32 Local lInfoEnd  := .T.
33 Local lCliPad   := .F.
34
35 Default lInfoCnpj := .F. //Opção do menu "Informar CNPJ/CPF"
36 Default lTelaFimVnd := .F.
37 Default lObrijaLJ950:= .F.
38
39 /* Caso a venda tenha sido recuperada, significa que o CPF/CNPJ ja foi fornecido anteriormente. */
40 If STBIsRecovered()
41     lInfCPF := .T.
42 ElseIf lInfoCnpj
43     lInfCPF := !STWValInfCPF()
44 EndIf
45
46 If !lInfCPF .Or. IsInCallStack('STWNxtTelaIt')
47     If !STBIsPaf()
48         lPrintCNPJ := LJAnalisaLeg(30)[1] .Or. lObrijaLJ950 //Verificar se é São Paulo?
49         lInfoEnd := LJAnalisaLeg(58)[1] //RS Informa o Endereço?
50     Else
51         lPrintCNPJ := .T.
52         lInfoEnd := .T.

```

```

53 EndIf
54
55 If lPrintCNPJ
56     lRet := .T.
57
58     lCliPad := (Alltrim(cCliente) == Alltrim(SuperGetMV("MV_CLIPAD"))) .AND. (Alltrim(cLojaCli) == Alltrim(SuperGetMV("MV_LOJAPAD")))
59
60     If lCliPad
61         //Executa a tela para perguntar o CGC
62         cNomeCli := SPACE(TamSX3("A1_NOME")[1])
63         cEndCli := SPACE(TamSX3("A1_END")[1])
64     Else
65         //Executa a tela para perguntar o CGC
66         If lInfoEnd
67             cNomeCli := IF(Empty(oCliModel:GetValue("SA1MASTER","A1_NOME")),SPACE(TamSX3("A1_NOME")[1]),SPACE(TamSX3("A1_NOME")[1]) ,;
68                         AllTrim(oCliModel:GetValue("SA1MASTER","A1_NOME")))
69             cEndCli := IF(Empty(oCliModel:GetValue("SA1MASTER","A1_END")),SPACE(TamSX3("A1_END")[1]),SPACE(TamSX3("A1_END")[1]) ,;
70                        AllTrim(oCliModel:GetValue("SA1MASTER","A1_END")))
71         EndIf
72     EndIf
73
74     //Caso jah tenha digitado ou cliente cadastrado, mostro o mesmo CPF para não ter que digitar novamente
75     If STIGInfCPF()
76         If !Empty(STDGPBasket("SL1","L1_CGCCLI"))
77             cDocCliSL1 := STDGPBasket("SL1","L1_CGCCLI")
78         ElseIf !Empty(AllTrim(STDGPBasket("SL1","L1_PFISICA")))
79             cDocCliSL1 := STDGPBasket("SL1","L1_PFISICA")
80         EndIf
81     EndIf
82     If !Empty(AllTrim(cDocCliSL1))
83         cParCgc := cDocCliSL1
84     Else
85         If lCliPad
86             cParCgc := Nil
87         Else
88             cParCgc := IF(Empty(oCliModel:GetValue("SA1MASTER","A1_CGC")) .And. Empty(oCliModel:GetValue("SA1MASTER","A1_PFISICA")),SPACE(TamSX3("A1_CGC")[1]) ,;
89                        IF(!Empty(oCliModel:GetValue("SA1MASTER","A1_CGC")), oCliModel:GetValue("SA1MASTER","A1_CGC"),
90                        AllTrim(oCliModel:GetValue("SA1MASTER","A1_PFISICA"))))
91         EndIf
92     EndIf
93
94     STIInfoCNPJ(cParCgc,cNomeCli, cEndCli, lInfoEnd,lTelaFimVnd)
95 Else
96     //Configura a variável do CNPJ
97     STI7InfCPF(.T.)
98 EndIf
99
100 Return lRet
101
102 //-----
103 /*{Protheus.doc} STI7InfCPF

```

```

104 Controla o valor da variavel static lInfCPF, que indica se a tela de CPF ja foi solicitada.
105
106 @param1      lSet          .T.
107 @return       Nil
108 @obs
109 @sample
110 /*/
111 //-----
112 Function STI7InfCPF(lSet)
113 DEFAULT lSet := .F.
114
115 lInfCPF := lSet
116
117 If !lSet //Na reinicialização, eu deixo em branco o CPF,nome,endereço digitado
118     STD7CPF0verReceipt("", "", "")
119 EndIf
120
121 Return
122
123 //-----
124 /*{Protheus.doc} STIGInfCPF
125 Retorna o valor da variavel static lInfCPF, que indica se a tela de CPF ja foi solicitada.
126
127 @return       lInfCPF - Indica se a tela de CPF ja foi solicitada
128 @obs
129 @sample
130 /*/
131 //-----
132 Function STIGInfCPF()
133 Return lInfCPF
134
135 //-----
136 /*{Protheus.doc} STWValInfCPF
137 Verifica se a venda foi iniciada e o cupom ja foi aberto, pois
138 não poderá ser informado CPF com cupom em andamento.
139
140 @return       lRet - Indica se poderá (.T.) ser informado o CPF.
141 @obs
142 @sample
143 /*/
144 //-----
145 Static Function STWValInfCPF()
146 Local lRet      := .T. //Retorno da função
147 Local oTotal    := STFGetTot() //Total da venda
148 Local lEmitNFce := STBGetNFCE() //Valida se é NFC-e
149 Local nLjInfDocCli := IIF(ExistFunc("LjInfDocCli"), LjInfDocCli(), 1) // Retorno do parametro MV_LJDCCLI
150
151 If nLjInfDocCli <= 1 .And. !lEmitNFce .And. oTotal:GetValue( "L1_VLRTOT" ) > 0
152     STFMessage(ProcName(), "STOP", STR0012) //"Não é possível informar o CPF, pois o cupom fiscal já foi aberto."
153     STFShowMessage( ProcName() )
154     lRet := .F.
155 EndIf

```

```

156
157 Return lRet
158
159 //-----
160 /*{Protoheus.doc} STWDitaCPF
161 Envia comando para ser executado na DLL e retorna a resposta.
162
163 @return      cRet      - Retorno da DLL ( CPF do Cliente )
164 @obs
165 */
166 //-----
167 Function STWDitaCPF()
168 Local cRet      := SPACE(TamSX3("A1_CGC")[1])           // Retorno do CPF
169 Local aRet      := {.F., ""}                             // Array de retorno, [01] = Comunicou com PinPad, [02] = CPF
170 Local cEntrada  := "011111DIGITE SEU CPF                CONFIRME O CPF  " // Mensagem a ser apresentada no visor do PinPad
171 Local cSaida    := "011100000000000"                  // Usado para retorno com o CPF digitado pelo cliente
172 Local cRetDLL   := "-1"                                // Retorno de sucesso ou não ao comando da DLL
173 Local oTEF20    := STBGetTEF()
174 Local lDigtCPF  := GetAPOInfo("LOJA1926.PRW")[4] >= Ctod("26/08/2019")
175 Local aTxtTerm  := {.F., ""}                             // Retorno do metodo ImptTxTerm
176 Local lPPadAdyen:= .F.                                    // Processo com pinpad Adyen. Projeto parado, enviar default .F.
177
178 LjGrvLog( "STWDitaCPF", "Inicio" )
179
180 If lDigtCPF
181
182     STFMessage("Waiting","FWMSGRUN",STR0023) //Aguardando informação via PinPad...
183     STFShowMessage("Waiting")
184
185     ProcessMessage()//Garante que a mensagem de "aguarde" seja exibida
186
187     // Chama o metodo para digitação do CPF
188     If lPPadAdyen .AND. ExistFunc("LjUsePayHub") .And. LjUsePayHub() .And. ValType(oTEF20:oConfig:oComPaymentHub) == "0" .AND. oTEF20:oConfig:ISPgtoDig()
189         aTxtTerm := oTEF20:oConfig:oComPaymentHub:ImptTxTerm()
190         If aTxtTerm[1]
191             cSaida := aTxtTerm[2]
192             cRetDLL := aTxtTerm[3]
193         EndIf
194     Else
195         cRetDLL := oTef20:oConfig:oComSitef:CPFPinPad(cEntrada, @cSaida )
196     EndIf
197
198     Do Case
199         Case cRetDLL == "0"
200
201             cSaida := SubStr(cSaida,1,11)
202
203             // Valida o CPF digitado
204             If CGC( cSaida, , .F. ) .AND. !STWldNRep(cSaida)
205                 aRet[01] := .T.
206                 aRet[02] := cSaida
207                 LjGrvLog( "STWDitaCPF", "CPF digitado via PinPad válido!",cRetDLL )

```

```

208         Else
209             LjGrvLog( "STWDitaCPF", "CPF Inválido!",cRetDLL )
210             STFMessage("CPF", "POPUP", STR0017 ) // "CPF Inválido!"
211             STFShowMessage( "CPF")
212         EndIf
213     Case cRetDLL == "-43"
214         LjGrvLog( "STWDitaCPF", "Problema com o PinPad. Verifique!",cRetDLL )
215         STFMessage("CPF", "POPUP", STR0018 ) // "Problema com o PinPad. Verifique!"
216         STFShowMessage( "CPF")
217
218     Case cRetDLL == "-2"
219         LjGrvLog( "STWDitaCPF", "Operação cancelada!",cRetDLL )
220         STFMessage("CPF", "POPUP", STR0019 ) // "Operação cancelada!"
221         STFShowMessage( "CPF")
222
223     Otherwise
224         LjGrvLog( "STWDitaCPF", "Verifique o serviço do Sitef! O serviço está indisponível ou o tempo de espera foi excedido.",cRetDLL )
225         STFMessage("CPF", "POPUP", STR0020 + CHR(13)+CHR(10) + STR0021 ) //"Verifique o serviço TEF! O serviço está indisponível ou o tempo de espera foi excedido."
226         STFShowMessage( "CPF")
227     EndCase
228
229     If aRet[01]
230         cRet := aRet[02]
231     EndIf
232
233     STFCleanMessage("Waiting")
234     STFCleanInterfaceMessage()
235
236 EndIf
237
238 LjGrvLog( "STWDitaCPF", "Fim" )
239
240 Return cRet

```

Componentes/Discount/STBDiscountTotalSale.PRW

672 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STBDISCOUNTTOTALSALE.CH"
4
5 Static lDiscTotPDV := .F.    // Indica se foi concedido desconto no Total da Venda pelo PDV
6
7 //-----
8 /*{Protheus.doc} STBValScreenDisc
9 Valida abertura de tela para desconto na venda
10
11 @param
12 @return lRet                Indica se pode abrir a tela de desconto no total
13 @obs
14 @sample
15 */
16 //-----
17 Function STBValScreenDisc()
18 Local lRet      := .T.      // Retorna se pode realizar a abertura de tela
19 Local oTotal    := STFGetTot() // Recebe o Objeto totalizador
20
21 /*
22 Valida Total Zerado
23 */
24 If lRet
25     If oTotal:GetValue( "L1_VLRTOT" ) <= 0
26         lRet := .F.
27     EndIf
28 EndIf
29
30 Return lRet
31
32 //-----
33 /*{Protheus.doc} STBValDisc
34 Function Valida Desconto no Total da Venda
35
36 @param nValDesc      Valor do Desconto
37 @param cType          Tipo do Desconto. Se P - Percentual ou V - Valor.
38 @param cOrigin        Define a procedência do desconto
39 @return lRet          Indica se pode ser realizado o desconto
40 @obs
41 @sample
42 */
43 //-----
44 Function STBValDisc( nValDesc , cType , cOrigin, aDescOrc )
45
46 Local      lRet      := .T.      // Encontrou item?
47 Local      nTotPerc   := 0        // Percentual total( Percentual dos Itens + Percentual Total )
48 Local      nTotValor  := 0        // Valor total( Valor dos Itens + Valor Total )
49 Local      nTotSale   := 0        // Valor total da venda
50 Local      lValUser   := .F.      // Define se valida permissao de caixa
51 Local      oTotal     := STFGetTot() // Recebe o Objeto totalizador
52

```

```

53 Default      nValDesc      := 0           // Valor do Desconto
54 Default      cType         := ""          // Tipo do Desconto. Se P - Percentual ou V - Valor.
55 Default      cOrigin       := ""          // Define a procedência do desconto
56 Default      aDescOrc      := {0,0}       // Array com Informações sobre desconto no orçamento, sendo ele; valor total e percentual total.
57
58 nTotSale := oTotal:GetValue( "L1_VLRTOT" )
59
60 If cOrigin == "US"          // Desconto de usuário
61     lValUser := .T.
62 EndIf
63
64 // Parametros errados
65 If lRet .AND. ( nValDesc <= 0 ) .OR. Empty(cType)
66     lRet := .F.
67     STFMessage("TotDesc","STOP", STR0001) //"Parametros do desconto nao compatíveis."
68 EndIf
69
70 // Valor Percentual acima do limite
71 If lRet .AND. cType == "P"
72     If nValDesc > 99.00 // 99.91 ja da erro no ecf
73         lRet := .F.
74         STFMessage("TotDesc","STOP", STR0002) //"Percentual de desconto acima do limite."
75     EndIf
76 EndIf
77
78 // Valor Percentual acima do limite
79 If lRet .AND. cType == "V"
80     If nValDesc >= nTotSale
81         lRet := .F.
82         STFMessage("TotDesc","STOP", "Valor do desconto acima do limite.") //"Valor do desconto acima do limite."
83     EndIf
84 EndIf
85
86 // Total da venda zerado
87 If lRet .AND. nTotSale == 0
88     lRet := .F.
89     STFMessage("TotDesc","STOP", STR0003) //"Não é possível realizar desconto para venda com total igual a zero."
90 EndIf
91
92 // Desconto maior ou igual ao total da venda
93 If lRet .AND. cType == "V"
94     If nValDesc >= nTotSale
95         lRet := .F.
96         STFMessage("TotDesc","STOP", STR0004) //"Não é possível realizar desconto de um valor maior que o valor total da venda."
97     EndIf
98 EndIf
99
100 // Verifica permissao do caixa
101 If lRet .AND. lValUser
102     lRet := STBValPermission( nValDesc , cType, aDescOrc )
103 EndIf
104

```

```

105 Return lRet
106
107
108 //-----
109 /*{Protheus.doc} STBValPermission
110 Verifica permissão do caixa para conceder desconto
111
112 @param  nValDesc      Valor do Desconto
113 @param  cType         Tipo do Desconto. Se P - Percentual ou V - Valor.
114 @return  lRet         Indica se pode ser realizado o desconto no valor desejado
115 @obs
116 @sample
117 */
118 //-----
119 Static Function STBValPermission( nValDesc , cType, aDescOrc )
120
121 Local      nTotSale      := 0              // Valor total da venda
122 Local      oTotal       := NIL            // Objeto Total
123 Local      nAux         := 0              // Variavel auxiliar
124 Local      nValueDesc   := 0              // Valor efetivo do desconto a validar
125 Local      nPercDesc    := 0              // Percentual efetivo do desconto a validar
126 Local      oTotal       := STFGetTot()    // Recebe o Objeto totalizador
127 Local      aRet         := {}            // Retorno da liberação do Supervisor- retorna: retorno (T ou F), nome do supervisor e código do supervisor
128
129 Default   nValDesc      := 0              // Valor do Desconto
130 Default   cType         := ""            // Tipo do Desconto. Se P - Percentual ou V - Valor.
131 Default   aDescOrc      := {0,0}        // Array com Informações sobre desconto no orçamento, sendo ele; valor total e percentual total.
132
133
134 nTotSale  := oTotal:GetValue( "L1_VLRTOT" )
135 aDiscount := STBDiscConvert( nValDesc , cType , nTotSale + aDescOrc[1]) // -- Somo o valor do desconto no orçamento apenas calculo do percentual
136 nValueDesc := aDiscount[1]
137 nPercDesc  := aDiscount[2]
138
139 aRet := STFProfile( 11 , NIL , nPercDesc , nValueDesc , "T", , ,nTotSale )
140
141 If aRet[1] .AND. STDRsnDesc()
142     STDAddSupD (0,,aRet[3])
143 EndIf
144
145 Return aRet[1]
146
147
148 //-----
149 /*{Protheus.doc} STBTotDiscApply
150 Verifica permissão do caixa para conceder desconto
151
152 @param  nValDesc      Valor do Desconto
153 @param  cType         Tipo do Desconto. Se P - Percentual ou V - Valor.
154 @param  cOrigin       Define a procedência do desconto
155 @return  lRet         Indica se foi aplicado desconto
156 @obs

```

```

157 @sample
158 /*/
159 //-----
160 Function STBTotDiscApply( nValDesc , cType , cOrigin , lAccumulate )
161 Local lRet := .F. // Retorno funcao
162 Local nValDiscount := 0 // Valor do desconto
163 Local nPerDiscount := 0 // Percentual do desconto
164 Local nTotSale := 0 // Valor total da venda
165 Local oTotal := STFGetTot() // Recebe o Objeto totalizador
166 Local lRecebTitle := STIGetRecTit() // Indica se eh recebimento de titulos
167 Local lDesTot := ExistFunc("STBCDPGDes") .And. STBCDPGDes()
168 Local lDesTotAnt := .F. // Indica se ja foi concedido desconto no Total (F7)
169 Local nTotDA1 := IIF(ExistFunc('STDGetDA1'),STDGetDA1(),0) // Total dos itens para conceder o desconto
170 Local oRaas := Nil
171 Local oFidelityC := Nil
172 Local nValDFidel := 0
173 Local nDescTot := 0
174
175 Default nValDesc := 0 // Valor do Desconto
176 Default cType := "" // Tipo do Desconto. Se P - Percentual ou V - Valor.
177 Default cOrigin := "" // Define a procedência do desconto
178 Default lAccumulate := .F. // Acumula com descontos anteriores
179
180
181 LjGrvLog( ,"STBTotDiscApply | Inicio | Chamada por: " + ProcName(1), )
182
183 nDescTot:= Iif(cOrigin == "RULEDISC", oTotal:GetValue("L1_DESCONT"), STBTaxRet( Nil ,"NF_DESCTOT" ) )
184
185 nTotSale := IIF(nTotDA1 > 0, nTotDA1, oTotal:GetValue( "L1_VLRTOT" ))
186 lDesTotAnt := !nDescTot == 0
187 aDiscount := STBDiscConvert( nValDesc , cType , nTotSale + Iif(cOrigin == "RULEDISC",oTotal:GetValue("L1_DESCONT"),0 ) )
188 nValDiscount := aDiscount[1]
189 nPerDiscount := aDiscount[2]
190
191 //Acumula valor do desconto
192 If lAccumulate
193     nValDiscount += nDescTot
194     nPerDiscount += STBDiscConvert( nDescTot , "V" , oTotal:GetValue( "L1_VLRTOT" ) + nDescTot)[2]
195 EndIf
196
197 LjGrvLog( ,"STBTotDiscApply | nTotSale -> " + cValToChar(nTotSale ), )
198 LjGrvLog( ,"STBTotDiscApply | lDesTotAnt -> " + cValToChar( lDesTotAnt ), )
199 LjGrvLog( ,"STBTotDiscApply | nTotDA1 -> " + cValToChar( nTotDA1 ), )
200 LjGrvLog( ,"STBTotDiscApply | lDesTot -> " + cValToChar( lDesTot ), )
201 LjGrvLog( ,"STBTotDiscApply | nValDiscount -> " + cValToChar( nValDiscount ), )
202
203 /*
204 Criada referencia para que seja possível validar
205 posteriormente se foi utilizado desconto no
206 total da venda e se existe a referencia no model;
207 Além de não poder pegar o desconto acumulado pois a referência
208 da matxfis NF_DESCTOT vai somar todos os descontos dados

```

```

209 no TotvsPDV
210 */
211
212 If lDesTot
213
214     LjGrvLog( ,"STBTotDiscApply | Tipo do Desconto (cType): " + cType, )
215     LjGrvLog( ,"STBTotDiscApply | Valor do Desconto: " + STR(nValDiscount), )
216
217     If cOrigin == "US" //Desconto no Total - F7
218         STDSPBasket("SL1","L1_DESCTOT", nValDiscount)
219         lRet := .T.
220     EndIf
221
222     If cOrigin == "RULEDISC" .AND. !(lDesTotAnt .AND. procname(2) $ "STWRULETOTAL")
223
224         If ExistFunc("LjxRaasInt") .And. LjxRaasInt()
225             oRaas := STBGetRaas()
226
227             If Valtype(oRaas) == "0" .And. oRaas:ServiceIsActive("TFC")
228                 oFidelityC := oRaas:GetFidelityCore()
229                 If oFidelityC:ChoseToUse()
230                     nValDFidel := oFidelityC:GetBonus()
231                 EndIf
232             EndIf
233         EndIf
234
235         If lDesTotAnt // Zera desconto anterior
236             If nTotDA1 > 0
237                 nValDiscount := ((nTotDA1 - nValDFidel) * ( nPerDiscount / 100 )) + nValDFidel
238             Else
239                 // Calcula o desconto desconsiderando o desconto concedido via F7
240                 nValDiscount := (((nTotSale + nDescTot) - nValDFidel) * ( nPerDiscount / 100 )) + nValDFidel
241             EndIf
242         EndIf
243
244         STDSPBasket("SL1","L1_DESCTOT", nValDiscount)
245         lRet := .T.
246
247     ElseIf cOrigin == "RULEDISC"
248         LjGrvLog( ,"STBTotDiscApply | Desconto referente a Regra de Desconto não aplicado, verificar campo LF_OPDESC !" , )
249     EndIf
250
251     If cOrigin == "CP" //Desconto Financeiro da Condição de Pagamento (SE4)
252         oTotal:SetValue("L1_DESCFIN",nValDiscount)
253         lRet := .T.
254     EndIf
255
256     If cOrigin == "FIDELITYCORE"
257         STDSPBasket("SL1","L1_DESCTOT", nValDiscount)
258         lRet := .T.
259     EndIf
260

```

```

261 EndIf
262
263 If nValDiscount >= 0
264     STBTaxAlt( "NF_DESCTOT" , nValDiscount )
265
266     If ExistFunc ("STBAjusICM") //Ao Aplicar o Desconto Ajusto a Base e o valor do ICMS
267         STBAjusICM()
268     EndIf
269 EndIf
270
271 /* Se for recebimento de títulos não traz o
272    desconto da condição de pagamento
273    Necessário para evitar erro na baixa do título.*/
274 If lRecebTitle .AND. nValDiscount > 0
275     oTotal:SetValue("L1_VLRTOT",nTotSale)
276     STFMessage("TotDesc", "ALERT",STR0007) //"Desconto financeiro só é permitido na seleção do título."
277 EndIf
278
279 LjGrvLog( ,"STBTotDiscApply | Fim | lRet: " + cValToChar( lRet ) , )
280
281 Return lRet
282
283 //-----
284 /*/{Protheus.doc} STBDiscConvert
285 Retorna valores percentual em valor e vice-versa
286
287 @param  nValDesc      Valor do Desconto
288 @param  cType         Tipo do Desconto. Se P - Percentual ou V - Valor.
289 @param  nTotSale      Total da venda
290 @return aRet          Retorna valores percentual em valor e vice-versa
291 @obs
292 @sample
293 */
294 //-----
295 Function STBDiscConvert( nValDesc , cType , nTotSale )
296
297 Local      aRet          := {}           // Retorno funcao
298 Local      nValueDisc    := 0           // Valor efetivo do desconto a validar
299 Local      nPercDesc     := 0           // Percentual efetivo do desconto a validar
300 Local      nAux          := 0           // Variavel auxiliar
301 Local      oTotal        := STFGetTot() // Totalizador
302
303 Default    nValDesc      := 0           // Valor do Desconto
304 Default    cType         := ""          // Tipo do Desconto. Se P - Percentual ou V - Valor.
305 Default    nTotSale      := 0           // Total da venda
306
307 If nTotSale <= 0
308     nTotSale := oTotal:GetValue("L1_VLRTOT")
309 EndIf
310
311 If ( nValDesc >= 0 ) .AND. ( nTotSale > 0 ) .AND. ( !Empty(cType) )
312

```

```

313 Do Case
314     Case cType = "P"
315         // Procura saber qual o VALOR do total corresponde ao PERCENTUAL desejado
316         nAux := nTotSale * ( nValDesc / 100 )
317
318         nValueDisc := STBRound( nAux )
319         nPercDesc  := STBRound( nValDesc )
320
321     Case cType = "V"
322         // Procura saber qual o PERCENTUAL do total corresponde ao VALOR desejado
323         nAux := ( nValDesc / nTotSale ) * 100
324
325         nValueDisc := STBRound( nValDesc )
326         nPercDesc  := STBRound( nAux )
327 EndCase
328
329 EndIf
330
331 AADD( aRet , nValueDisc )
332 AADD( aRet , nPercDesc )
333
334 Return aRet
335
336
337 //-----
338 /*{Protheus.doc} STBDiscIncrease
339 Tratamento desconto e acréscimo
340
341 @param
342 @return lRet          Retorna se registrou desconto/acrécimo
343 @obs
344 @sample
345 */
346 //-----
347 Function STBDiscIncrease()
348
349 Local lRet          := .T.          // Retorna se registrou desconto/acrécimo
350 Local oTotal        := STFGetTot()  // Totalizador
351 Local nDiscount      := 0           // Valor do desconto
352 Local nIncrease      := 0           // Valor do acréscimo
353 Local nDiff          := 0           // Valor de IPI e ICMS Solidario
354 Local lItemNF        := STBExistNItemFiscal() // Indica se existe item não-fiscal na venda
355 Local lItEntrega     := STBTemEntr() // Indica se existe item de entrega na venda
356
357 nDiscount := oTotal:GetValue("L1_DESCONT")
358
359 If STBIsImpOrc() .And. lItEntrega .And. lItemNF
360     nIncrease := ( oTotal:GetValue( "L1_ACRSFIN" ) )
361 Else
362     nIncrease := ( oTotal:GetValue( "L1_ACRSFIN" ) + ;
363                   oTotal:GetValue( "L1_FRETE" ) + ;
364                   oTotal:GetValue( "L1_SEGURO" ) + ;

```

```

365             oTotal:GetValue( "L1_DESPESA" ) )
366 EndIf
367
368 nDiff := STBGValDifTx() //Calcula valores de IPI e ICMS Solidario
369
370 If nDiff > 0
371     nIncrease += nDiff
372 EndIf
373
374 Do Case
375
376     Case nDiscount > 0 .AND. nIncrease > 0
377
378         lRet := STBDiscAndIncrease(nDiff)
379
380     Case nDiscount > 0
381
382         lRet := STBPrintDiscount(nDiscount)
383
384     Case nIncrease > 0
385
386         lRet := STWPrintIncrease(nIncrease)
387
388 EndCase
389
390 If lRet .AND. (nDiscount > 0 .OR. nIncrease > 0)
391     STBDiscRefresh()
392 EndIf
393
394 Return lRet
395
396
397 //-----
398 /*{Protheus.doc} STBDiscAndIncrease
399 Tratamento para desconto e acréscimo na mesma venda
400
401 @param
402 @return lRet          Retorna se registrou desconto e/ou acréscimo na impressora
403 @obs
404 @sample
405 */
406 //-----
407 Function STBDiscAndIncrease(nTxDiff)
408
409 Local lRet          := .T.          // Retorno da funcao
410 Local oTotal        := STFGetTot()  // Totalizador
411 Local nDiscount      := 0           // Valor do desconto
412 Local nIncrease      := 0           // Valor do acréscimo
413 Local nDifference     := 0           // Diferença entre Desconto e Acréscimo
414
415 Default nTxDiff      := 0           // Diferença
416

```

```

417 nDiscount    := oTotal:GetValue("L1_DESCONT")
418
419 nIncrease     := (    oTotal:GetValue( "L1_ACRSFIN"    ) + ;
420                     oTotal:GetValue( "L1_FRETE"      ) + ;
421                     oTotal:GetValue( "L1_SEGURO"      ) + ;
422                     oTotal:GetValue( "L1_DESPESA"     ) )
423
424 If nTxDiff > 0
425     nIncrease += nTxDiff
426 EndIf
427
428 If nDiscount > 0 .AND. nIncrease > 0
429
430     Do Case
431
432         /*/
433         Desconto
434         /*/
435         Case nDiscount > nIncrease
436
437             nDifference := nDiscount - nIncrease
438             lRet := STBPrintDiscount(nDifference)
439
440         /*/
441         Acrécimo
442         /*/
443         Case nIncrease > nDiscount
444
445             nDifference := nIncrease - nDiscount
446             lRet := STWPrintIncrease(nDifference)
447
448         /*/
449         Ambos são iguais
450         /*/
451         Otherwise
452
453             lRet := .T.
454
455     EndCase
456
457 EndIf
458
459 Return lRet
460
461
462 //-----
463 /*/{Protheus.doc} STBPrintDiscount
464 Registra desconto no total na impressora
465
466 @param  nDiscount      Valor do desconto no total
467 @return lRet           Retorna se registrou desconto na impressora
468 @obs

```

```

469 @sample
470 /*/
471 //-----
472 Function STBPrintDiscount( nDiscount )
473
474 Local lRet           := .T.           // Retorna se registro desconto na impressora
475 Local nTotal         := 0             // Valor Total
476 Local aRetPrinter    := {}           // Armazena retorno da impressora
477 Local oTotal         := STFGetTot()   // Totalizador
478 Local nOpt           := 2             // Opção selecionada pelo usuário para tentar novamente
479 Local lTentaTiva     := .T.           // controle de tentativa
480
481 Default nDiscount    := 0             // Valor do desconto no total
482
483 ParamType 0 Var nDiscount As Numeric   Default 0
484
485 If STFUseFiscalPrinter()
486
487     If STWGetIsOpenReceipt() // Cupom Aberto?
488
489         If nDiscount <= 0
490             nDiscount := oTotal:GetValue( "L1_DESCONT" )
491         EndIf
492
493         If nDiscount > 0
494
495             nDiscount := nDiscount * STBFactor()[1]
496
497             nTotal     := STBFactor()[3]
498
499             aRetPrinter := STFFireEvent(   ProcName(0)           , ; // Nome do processo
500                                           "STDescTot"          , ; // Nome do evento
501                                           {AllTrim(Str(STBRoundCurrency( nDiscount ),14,2)) , ; // 01 - VALOR DESCONTO
502                                           AllTrim(Str(STBRoundCurrency( nTotal ),14,2))   } ) // 02 - Valor Total
503
504             If Len(aRetPrinter) > 0 .AND. ValType(aRetPrinter[1]) == "N" .AND. aRetPrinter[1] <> 0
505
506                 While lTentaTiva
507
508                     STFMessage("TEFImprime", "YESNO", STR0006 ) //"Impressora não responde. Deseja imprimir novamente?"
509                     nOpt := If(STFShowMessage("TEFImprime"),2,0)
510
511                     //2=SIM
512                     If nOpt == 2
513
514                         Sleep(1000)
515
516                         aRetPrinter := STFFireEvent(   ProcName(0)           , ; // Nome do processo
517                                                         "STDescTot"          , ; // Nome do evento
518                                                         {AllTrim(Str(STBRoundCurrency( nDiscount ),14,2)) , ; // 01 - VALOR DESCONTO
519                                                         AllTrim(Str(STBRoundCurrency( nTotal ),14,2))   } ) // 02 - Valor Total
520

```

```

521
522         //Erro na impressao
523         If Len(aRetPrinter) > 0 .AND. ValType(aRetPrinter[1]) == "N"      .AND. aRetPrinter[1] <> 0
524             lTentaTiva := .T.
525         Else
526             lTentaTiva := .F.
527         EndIf
528
529     Else
530         lTentaTiva := .F.
531     EndIf
532
533 EndDo
534
535 EndIf
536
537 If Len(aRetPrinter) > 0 .AND. ValType(aRetPrinter[1]) == "N"
538
539     If aRetPrinter[1] == 0
540         lRet := .T.
541     Else
542         lRet := .F.
543         STFMessage("STDDiscount","STOP", STR0005) //"Não foi possível registrar o Desconto no Total do Cupom. Operação não efetuada."
544     EndIf
545
546 Else
547     lRet := .F.
548     STFMessage("STDDiscount","STOP", STR0005) // "Não foi possível registrar o Desconto no Total do Cupom. Operação não efetuada."
549 EndIf
550
551 Else
552     lRet := .T.
553 EndIf
554
555 Else
556     lRet := .T.
557 EndIf
558
559 Else
560     lRet := .T.
561 EndIf
562
563 STFShowMessage("STDDiscount")
564
565 Return lRet
566
567
568 //-----
569 /*{Protheus.doc} STBDiscRefresh
570 Atualiza os valores de todos os itens da MATXFIS para a cesta e para o banco.
571 Caso concedido desconto ou acréscimo por exemplo
572

```

```

573 @param
574
575
576 @return
577 @obs
578 @sample
579 /*/
580 //-----
581 Function STBDiscRefresh()
582
583 Local nI          := 0          // Contador
584
585 For nI := 1 To STDPBLength("SL2")
586     STBRefshItBasket( nI )
587     STDSaveSale(nI)
588
589 Next nI
590
591
592 Return
593
594 //-----
595 /*/{Protoheus.doc} STBValDifTx
596 Apura os valores de IPI e Valor de ICMS Solidario a serem acrescidos
597 no total da venda.
598
599 @param
600 @return nTxDiff - Valor de IPI + ICMS solidario.
601 @obs
602 @sample
603 /*/
604 //-----
605 Static Function STBGValDifTx()
606 Local nTxDiff := 0
607 Local nI      := 0
608 Local nValSol := 0
609 Local nValIpi := 0
610 Local lCfgTrib as logical //Verifica se Configurador de Tributos esta habilitado
611 Local jCfgTaxes as json
612
613 lCfgTrib := If(FindFunction("LjCfgTrib"), LjCfgTrib(), .F.)
614 jCfgTaxes := JsonObject():New()
615
616 If STBTaxRet(, "NF_VALIPI") > 0 .Or. STBTaxRet(, "NF_VALSOL") > 0
617
618     For nI := 1 To STDPBLength("SL2")
619
620         If !STDPBIsDeleted( "SL2", nI )
621             If lCfgTrib
622                 jCfgTaxes := STBGetTax(nI)
623             EndIf
624

```

```

625         //Verifica se TES configurado para agregar ICMS Solidario
626         If Len(jCfgTaxes) > 0 .And. jCfgTaxes[1]:getJsonObject("ident_trib") == "000056" //Configurador de Tributos - Id ICMS-ST = 000056
627             If STBVerTotTax(jCfgTaxes[1]:getJsonObject("ident_trib"), 1)
628                 nValSol += STBTaxRet(nI, "IT_VALSOL")
629             EndIf
630         ElseIf AllTrim( STBGetTESInfo( "F4_INCSOL" , AllTrim(STBTaxRet(nI,"IT_TES"))) ) != "N"
631             nValSol += STBTaxRet(nI, "IT_VALSOL")
632         EndIf
633
634         nValIpi += STBTaxRet(nI, "IT_VALIPI")
635
636     EndIf
637     Next nI
638
639     nTxDiff := (nValSol + nValIpi)
640
641 EndIf
642
643 FreeObj(jCfgTaxes)
644 jCfgTaxes := Nil
645
646 Return nTxDiff
647
648 //-----
649 /*{Protheus.doc} STBSetDiscTotPDV
650 Seta valor a variavel estatica lDiscTotPDV
651 @param      lDiscount      Logico - Desonto no Total da Venda pelo PDV
652 @return
653 @obs
654 @sample
655 */
656 //-----
657 Function STBSetDiscTotPDV( lDiscount )
658     lDiscTotPDV := lDiscount
659 Return
660
661
662 //-----
663 /*{Protheus.doc} STBGetDiscTotPDV
664 Retorna o valor de lDiscTotPDV
665 @param
666 @return      lDiscTotPDV
667 @obs
668 @sample
669 */
670 //-----
671 Function STBGetDiscTotPDV()
672 Return(lDiscTotPDV)

```

Componentes/Discount/STBItemDiscount.PRW

570 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STPOS.CH"
4 #INCLUDE "STBITEMDISCOUNT.CH"
5 Static lDescOK := .T.
6
7 //-----
8 /*{Protheus.doc} STBIDUser
9 Desconto via usuário
10
11 @param  nItemLine      Linha do item
12 @param  cTime          Indica momento de realizar o desconto
13 @param  lItemFiscal    Indica se o item pertence ao cupom fiscal
14 @return  lRet          Retorna se o desconto pode ser concedido
15 @obs
16 @sample
17 */
18 //-----
19 Function STBIDUser( nItemLine , cTime , lItemFiscal, cCodSup )
20
21 Local lRet          := .F.          // Retorna se efetuou desconto
22 Local aDiscount     := {}           // Armazena Valor e tipo de desconto via usuário
23 Local lContinue     := .T.          // Fluxo função
24
25 Default nItemLine   := 0
26 Default cTime       := ""
27 Default lItemFiscal := .F.
28
29 ParamType 0 Var  nItemLine As Numeric Default 0
30 ParamType 1 Var  cTime     As Character  Default ""
31 ParamType 2 Var  lItemFiscal As Logical   Default .F.
32
33 If Empty(cTime)
34     cTime := STDIDTime()
35 EndIf
36
37 Do Case
38     /*/
39     Desconto Antes/Junto do Item
40     /*/
41     Case cTime == "A"
42
43         /*/
44         Get do desconto dado via usuário
45         /*/
46         aDiscount := STBGetItDiscount()
47
48         /*/
49         Validação
50         /*/
51         If lContinue
52

```

```

53         lContinue := STBIDValid( nItemLine , aDiscount[1] , aDiscount[2] , .T. ,;
54                                 "A" , , @cCodSup )
55
56     EndIf
57
58
59     /*/
60     Aplicar desconto
61     /*/
62     If lContinue
63
64         lRet := STBIDApply( nItemLine , aDiscount[1] , aDiscount[2] , lItemFiscal ,;
65                             "A" )
66
67     EndIf
68
69     /*/
70     Desconto depois de registrado o Item
71     /*/
72     Case cTime == "D"
73
74         /*/
75         Get do desconto dado via usuário
76         /*/
77         aDiscount := STBGetItDiscount()
78
79         /*/
80         Validação
81         /*/
82         lContinue := STBIDValid( nItemLine , aDiscount[1] , aDiscount[2] , .T. ,;
83                                 "D" , , @cCodSup )
84
85         /*/
86         Aplicar desconto
87         /*/
88         If lContinue
89
90             /*/
91             Procura saber se o item é fiscal. Caso positivo deve mandar desconto para impressora fiscal
92             /*/
93             lItemFiscal := STDGPBasket( "SL2" , "L2_FISCAL" , nItemLine )
94
95             lRet := STBIDApply( nItemLine , aDiscount[1] , aDiscount[2] , lItemFiscal ,;
96                                 "D" )
97
98         EndIf
99
100 EndCase
101
102 // Seta variavel de permissao de desconto
103 lDescOK := lRet
104 Return lRet

```

```

105
106
107 //-----
108 /*/{Protheus.doc} STBIDValid
109 Efetua validacao do desconto
110
111 @param  nItemLine      Linha do Item
112 @param  nValDesc       Valor do desconto
113 @param  cTypeDesc      Tipo do desconto. P - Percentual, V - Valor
114 @param  lValUser       Indica se valida permissões do caixa.
115 @param  cTime          Indica momento de realizar o desconto
116 @param  nItemTotal     Total do Item. Quantidade * Valor unitário
117 @return  lRet          Retorna se o desconto pode ser concedido
118 @obs
119 @sample
120 */
121 //-----
122 Function STBIDValid(  nItemLine      , nValDesc  , cTypeDesc , lValUser , ;
123                     cTime          , nItemTotal , cCodSup   )
124
125 Local lRet          := .T.          // Retorna se o desconto pode ser concedido
126 Local nPercDesc     := 0            // Percentual do desconto
127 Local nValueDisc    := 0            // Valor do desconto
128 Local nAux          := 0            // Variavel auxiliar
129 Local aRet          := {}
130
131 Default nItemLine   := 0
132 Default nValDesc    := 0
133 Default cTypeDesc   := ""
134 Default lValUser    := .T.
135 Default cTime       := ""
136 Default nItemTotal  := 0
137
138 ParamType 0 Var nItemLine      As Numeric      Default 0
139 ParamType 1 Var nValDesc       As Numeric      Default 0
140 ParamType 2 Var cTypeDesc      As Character    Default ""
141 ParamType 3 Var lValUser       As Logical      Default .T.
142 ParamType 4 Var cTime          As Character    Default ""
143 ParamType 5 Var nItemTotal     As Numeric      Default 0
144
145 Do Case
146
147     /*/
148     Antes/junto com registro do item
149     /*/
150     Case cTime == "A"
151
152         If ( nValDesc <= 0 )      .OR.  !( cTypeDesc $ "V|P" )
153
154             lRet := .F.
155
156         EndIf

```

```

157
158     If lRet
159
160         If nItemTotal <= 0
161             nItemTotal := STBTaxRet( nItemLine , "IT_PRCUNI"      ) * STBTaxRet( nItemLine , "IT_QUANT"      )
162         EndIf
163
164         If cTypeDesc == "V"
165             If nValDesc >= nItemTotal      // Desconto maior que valor total do Item?
166                 lRet := .F.
167                 STFMessage("STBIDValid","STOP", STR0001) //"Valor do desconto maior que o valor do Item. Desconto não concedido."
168             EndIf
169         EndIf
170
171     EndIf
172
173     If lRet
174
175         /*/
176         Validacao usuario, permissao do caixa
177         /*/
178         If lValUser
179
180             Do Case
181
182                 Case cTypeDesc = "P"
183                     /*/
184                     Procura saber qual o VALOR do total corresponde ao PERCENTUAL desejado
185                     /*/
186                     nAux := nItemTotal * ( nValDesc / 100 )
187
188                     nValueDisc := nAux
189                     nPercDesc := nValDesc
190
191                 Case cTypeDesc = "V"
192                     /*/
193                     Procura saber qual o PERCENTUAL do total corresponde ao VALOR desejado
194                     /*/
195                     nAux := ( nValDesc / nItemTotal ) * 100
196
197                     nValueDisc := nValDesc
198                     nPercDesc := nAux
199
200             EndCase
201
202             aRet := STFPROFILE( 11      , NIL , nPercDesc , nValueDisc , ;
203                               "I"      )
204             lRet  := aRet[1]
205             cCodSup := aRet[3]
206
207         EndIf
208

```

```

209
210     EndIf
211
212     /*/
213     Depois do item registrado
214     /*/
215     Case cTime == "D"
216
217         If nItemTotal <= 0
218             nItemTotal := STBTaxRet( nItemLine , "IT_PRCUNI" ) * STBTaxRet( nItemLine , "IT_QUANT" )
219         EndIf
220
221         If nValDesc >= nItemTotal          // Desconto maior que valor total do Item?
222
223             lRet := .F.
224             STFMessage("STBIDValid","STOP", STR0001) //"Valor do desconto maior que o valor do Item. Desconto não concedido."
225
226         EndIf
227
228         If lRet
229
230             Do Case
231
232                 /*/
233                 Indica que solicitou zerar o desconto
234                 /*/
235                 Case ( nValDesc == 0 ) .AND. ( cTypeDesc $ "V|P" )
236
237                     /*/
238                     Caso já teve desconto no Item, pode zerar
239                     /*/
240                     If STBTaxRet( nItemLine , "IT_DESCONTO" ) > 0
241
242                         lRet := .T.
243
244                         /*/
245                         Chamada Delecao do Motivo de Desconto
246                         /*/
247                         STDDelReason( nItemLine )
248
249                     Else
250                         lRet := .F.
251                     EndIf
252
253                 /*/
254                 Indica que nao informou desconto algum
255                 /*/
256                 Case ( nValDesc <= 0 ) .AND. !( cTypeDesc $ "V|P" )
257
258                     lRet := .F.
259
260                 /*/

```

```

261         Indica que informou desconto
262     /*/
263     Case ( nValDesc > 0 ) .AND. ( cTypeDesc $ "V|P" )
264
265         If lValUser
266
267             Do Case
268
269                 Case cTypeDesc = "P"
270                     /*/
271                     Procura saber qual o VALOR do total corresponde ao PERCENTUAL desejado
272                     /*/
273                     nAux := nItemTotal * ( nValDesc / 100 )
274
275                     nValueDisc := nAux
276                     nPercDesc := nValDesc
277
278                 Case cTypeDesc = "V"
279                     /*/
280                     Procura saber qual o PERCENTUAL do total corresponde ao VALOR desejado
281                     /*/
282                     nAux := ( nValDesc / nItemTotal ) * 100
283
284                     nValueDisc := nValDesc
285                     nPercDesc := nAux
286
287             EndCase
288
289             aRet := STFPROFILE( 11 , NIL , nPercDesc , nValueDisc , "I" )
290
291             lRet := aRet[1]
292             cCodSup := aRet[3]
293
294         EndIf
295
296     EndCase
297
298 EndIf
299
300 EndCase
301 EndCase
302
303 STFS showMessage("STBIDValid")
304
305 Return lRet
306
307
308 //-----
309 /*/{Protheus.doc} STBIDApply
310 Aplica desconto no Item
311
312 @param nItemLine      Linha do Item

```

```

313 @param  nValDesc      Valor do desconto
314 @param  cTypeDesc     Tipo do desconto. P - Percentual, V - Valor
315 @param  lItemFiscal   Indica se registra o item na impressora fiscal
316 @param  cTime         Indica momento de realizar o desconto
317 @param  nItemTotal    Total do Item. Valor unitário * Quantidade
318 @return  lRet         Retorna se o desconto foi aplicado
319 @obs
320 @sample
321 /*/
322 //-----
323 Function STBIDApply(  nItemLine , nValDesc , cTypeDesc , lItemFiscal, ;
324                     cTime      , nItemTotal, lRgDescVrj )
325
326 Local lRet          := .T.      // Retorna se o desconto foi aplicado
327 Local nDiscount     := 0        // Valor do Desconto
328 Local nPercDiscount := 0        // Percentual do Desconto
329 Local nAux          := 0        // Variável auxiliar
330 Local aRetDiscPrint := {}       // Armazena retorno desconto na impressora
331 Local lCRdesItTt    := SuperGetMv("MV_LJRGDES",,.F.) .AND. SuperGetMv("MV_LJCRDPT",,"0") == "1" .AND.
    FindFunction("totvs.protheus.retail.desconto.RegraDescProdutoTotal.LjCallCalcRegDescProdTotal", .T.) // Verifica se o calculo do desconto por item esta sendo feito no
    final da venda
332 Local lDesligaRD    := .F.      // Desabilita a Regra de Desconto do Varejo
333
334 Default nItemLine    := 0
335 Default nValDesc     := 0
336 Default cTypeDesc    := ""
337 Default lItemFiscal  := .F.
338 Default cTime        := ""
339 Default nItemTotal   := 0
340 Default lRgDescVrj   := .F.
341
342 ParamType 0 Var nItemLine    As Numeric    Default 0
343 ParamType 1 Var nValDesc     As Numeric    Default 0
344 ParamType 2 Var cTypeDesc    As Character  Default ""
345 ParamType 3 Var lItemFiscal  As Logical   Default .F.
346 ParamType 4 Var cTime        As Character  Default ""
347 ParamType 5 Var nItemTotal   As Numeric    Default 0
348
349 If nItemTotal <= 0
350     nItemTotal := STBTaxRet( nItemLine , "IT_PRCUNI" ) * STBTaxRet( nItemLine , "IT_QUANT" )
351 EndIf
352
353 If FindFunction("totvs.protheus.retail.desconto.RegraDescProdutoTotal.HasDescManu", .T.)
354     lDesligaRD := totvs.protheus.retail.desconto.RegraDescProdutoTotal.HasDescManu("TOTVSPDV", STDGPBasket('SL1','L1_NUM'), STDGPBasket('SL1','L1_CLIENTE'),
    STDGPBasket('SL1','L1_LOJA'))
355 EndIf
356
357 Do Case
358
359     Case cTypeDesc = "P"
360         /*/
361         Procura saber qual o VALOR do total corresponde ao PERCENTUAL desejado

```

```

362      /*/
363      nAux := nItemTotal * ( nValDesc / 100 )
364
365      nDiscount      := STBArred(nAux)
366      nPercDiscount  := STBRound( nValDesc , 2 )
367
368      Case cTypeDesc = "V"
369      /*/
370      Procura saber qual o PERCENTUAL do total corresponde ao VALOR desejado
371      /*/
372      nAux := ( STBArred(nValDesc) / nItemTotal ) * 100
373
374      nDiscount      := STBArred(nValDesc)
375      nPercDiscount  := STBRound( nAux , 2 )
376
377      EndCase
378
379      /*/
380      Não realizar desconto no item 2 vezes pra item fiscal(impresso em cupom fiscal)
381      /*/
382      If lItemFiscal .AND. cTime == "D" .AND. STBTaxRet( nItemLine , "IT_DESCONTO" ) > 0
383      lRet := .F.
384      STFMessage("STBIDApply","ALERT", STR0002) //"Só é possível aplicar desconto no item apenas uma vez"
385      Else
386      lRet := .T.
387      EndIf
388
389      /*/
390      Aplica desconto
391      /*/
392      If lRet
393
394      lRet := STBTaxAlt( "IT_DESCONTO" , nDiscount , nItemLine )
395
396      EndIf
397
398      /*/
399      Depois do item registrado, se o item for fiscal, registrar desconto na impressora
400      /*/
401      If lRet
402
403      If cTime == "D"
404
405      If lItemFiscal
406
407      aRetDiscPrint := STFFireEvent( ProcName(0)           , , // Nome do processo
408      "STItemReg"      , , // Nome do evento
409      {Nil              , , // 01 -Codigo do Item
410      Nil              , , // 02 -Descricao
411      Nil              , , // 03 - Quantidade
412      Nil              , , // 04 - Valor Unitario
413      AllTrim(STR(nDiscount)) , , // 05 - Desconto do Item

```

```

414 Nil ,; // 06 - Situacao tributaria
415 Nil ,; // 07 - Total do Item
416 Nil ,; // 08 - Unidade de Medida
417 Nil } ) // 09 - TIPO TES ( 2 - Saida )
418
419 /*/
420     Valida Retorno da impressora
421 /*/
422 If ValType(aRetDiscPrint) == "A" .AND. Len(aRetDiscPrint) > 0 .AND. aRetDiscPrint[1] <> 0
423
424     lRet := .F.
425     STBTaxAlt( "IT_DESCONT0" , 0 , nItemLine )
426     STFMessage("STBIDApply","ALERT",STR0003 ) //"Não foi possível registrar o desconto no item na impressora, o desconto será desconsiderado."
427
428 EndIf
429
430 EndIf
431
432 /*/
433     Atualizar valores da cesta se o item já foi registrado.
434 /*/
435 STBRefshItBasket( nItemLine, , lRgDescVrj ) // OBS: Caso for Antes de registrar já atualiza no registro de item
436
437 If lCRdesItTt .AND. !lDesligaRD
438     STDSaveSale(nItemLine)
439 EndIf
440 EndIf
441
442 EndIf
443
444 STFShowMessage("STBIDApply")
445
446 Return lRet
447
448
449 //-----
450 /*/{Protheus.doc} STBIDBothApply
451 Aplica Desconto de Regra de Desconto e de Usuário. Desconto sobre desconto
452
453 @param nItemLine      Linha do Item
454 @param aRuleDiscount  Array desconto via Regra de Desconto
455 @param aDiscount      Array Desconto via Usuário
456 @param lItemFiscal    Indica se registra o item na impressora fiscal
457 @param cTime          Indica momento de realizar o desconto
458 @return lRet          Retorna se o desconto foi aplicado
459 @obs
460 @sample
461 /*/
462 //-----
463 Function STBIDBothApply( nItemLine , aRuleDiscount , aDiscount , lItemFiscal , ;
464                         cTime
465 )

```

```

466 Local lRet           := .F.      // Retorna se o desconto foi aplicado
467 Local nDiscount      := 0        // Valor do Desconto
468 Local nPercDiscount  := 0        // Percentual do Desconto
469 Local nAux           := 0        // Variável auxiliar
470 Local nBothDiscount  := 0        // Armazena os valores de desconto de regra e de usuário somados
471 Local nValDesc       := 0
472
473
474 Default nItemLine     := 0
475 Default aRuleDiscount := 0
476 Default aDiscount    := ""
477 Default lItemFiscal   := .F.
478 Default cTime        := ""
479
480 ParamType 0 Var nItemLine As Numeric      Default 0
481 ParamType 1 Var aRuleDiscount As Array    Default {}
482 ParamType 2 Var aDiscount As Array       Default {}
483 ParamType 3 Var lItemFiscal As Logical    Default .F.
484 ParamType 4 Var cTime As Character       Default ""
485
486
487 If aRuleDiscount[LRULEAPPLY] .AND. ( aDiscount[1] >= 0 ) .AND. ( aDiscount[2] $ "V|P" )
488 /* Igual a zero pq pode ser que queira zerar o desconto de usuário */
489 lRet := .T.
490
491 EndIf
492
493
494 If lRet
495 /*
496 Calcular o desconto via usuário em relação ao desconto via regra
497 */
498 Do Case
499
500 Case aDiscount[2] = "P"
501 /*
502 Procura saber qual o VALOR do total corresponde ao PERCENTUAL desejado
503 */
504 nAux := aRuleDiscount[NNEWTOTAL] * ( aDiscount[1] / 100 )
505
506 nDiscount := nAux
507 nPercDiscount := nValDesc
508
509 Case aDiscount[2] = "V"
510 /*
511 Procura saber qual o PERCENTUAL do total corresponde ao VALOR desejado
512 */
513 nAux := ( aDiscount[1] / aRuleDiscount[NNEWTOTAL] ) * 100
514
515 nDiscount := aDiscount[1]
516 nPercDiscount := nAux
517

```

```

518
519     EndCase
520
521     nBothDiscount := nDiscount + aRuleDiscount[NDESCTOTAL]
522
523     If cTime == "D"
524
525         /*
526             Procura saber se o item é fiscal. Caso positivo deve mandar desconto para impressora fiscal
527         */
528         lItemFiscal := STDGPBasket( "SL2" , "L2_FISCAL" , nItemLine )
529
530     EndIf
531
532
533     lRet := STBIDApply(      nItemLine      , nBothDiscount      , "V" , lItemFiscal , ;
534                          cTime          ,
535                          )
536 EndIf
537
538
539 Return lRet
540
541 //-----
542 /*{Protheus.doc} STBValidDesc
543 Retorna se desconto pode ser concedido
544
545 @param      lRet          .T.
546 @return     lRet
547 @obs
548 @sample
549 */
550 //-----
551 Function STBValidDesc(lRet)
552 DEFAULT lRet := .T.
553
554 lRet := lDescOK
555
556 Return lRet
557
558 //-----
559 /*{Protheus.doc} STBIDSet
560 Seta valor (.T. ou .F.) à variável lDescOK (default = .T.)
561
562 @sample
563 */
564 //-----
565 Function STBIDSet(lVal)
566 Default lVal := .T.
567
568 lDescOK := lVal
569

```

570 Return

Componentes/Discount/STBReasonForDiscount.prw

81 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 //-----
5 /*{Protheus.doc} STBRFDShowObs
6 Verifica configurações se exhibe campo observação na interface
7
8 @param
9 @return lShow          Retorna se exhibe campo obs na interface
10 @obs
11 @sample
12 */
13 //-----
14 Function STBRFDShowObs()
15
16 Local lShow          := .T.      // Retorno funcao
17
18 /*/
19     Facilitador que diz se:
20     "N" - Não Aparece
21     "B" - Aparece e é obrigatório
22     "P" - Aparece e é opcional
23 /*/
24 If  STFCfg("cRFDShowObs") == "N"
25     lShow := .T.
26 Else
27     lShow := .F.
28 EndIf
29
30 Return lShow
31
32
33 //-----
34 /*{Protheus.doc} STBRFD0brigat
35 Verifica configurações se exhibe campo observação na interface
36
37 @param
38 @return lShow          Retorna se exhibe campo obs na interface
39 @obs
40 @sample
41 */
42 //-----
43 Function STBRFD0brigat()
44
45 Local l0brigat        := .T.      // Retorno funcao
46
47 /*/
48     Facilitador que diz se:
49     "N" - Não Aparece
50     "B" - Aparece e é obrigatório
51     "P" - Aparece e é opcional
52 /*/

```

```

53 If STGetCfg("cRFD0brigat") $ "B|P"
54     l0brigat := .T.
55 Else
56     l0brigat := .F.
57 EndIf
58
59
60 Return l0brigat
61
62 //-----
63 /*{Protheus.doc} STBRFDAtiv
64 Verifica se o motivo de desconto esta ativo para itens
65
66 @param
67 @return lRet Existem dados na tabela MDT e esta ativo o tratamento
68 @obs
69 @sample
70 */
71 //-----
72 Function STBRFDAtiv(nItemLine,cCodItem,cTesPad,nTotItem)
73
74 Local lRet      := .T.
75 Local lMotDesIt := SuperGetMV("MV_LJMTDIT",,.T.)
76
77 If lMotDesIt .And. STDRsnDesc()
78     STIExchangePanel( { || STIReasonD(nItemLine,cCodItem,cTesPad,nTotItem) } )
79 EndIf
80
81 Return lRet

```

Componentes/Discount/STBRuleDiscountItem.PRW

603 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STPOS.CH"
4 #INCLUDE "STBRULESDISCOUNTITEM.CH"
5
6 Static lImpRegra      := .F.
7 Static aProdsVda     := {} //Array contendo os produtos da venda e suas respectivas quantidades vendidas, utilizadas e nao utilizadas ainda para aplicacao das regras de
    desconto.
8
9 //-----
10 /*/{Protheus.doc} STBItemRldi
11 Function Regra de Desconto
12
13 @param  nItemLine      Número do Item
14 @param  cItemCode      Codigo Produto
15 @param  nValueItem     Valor
16 @param  nAmount        Quantidade
17 @param  cTypeProd      Tipo do Produto Mostruario/Saldao
18 @param  cCliCode       Codigo do Cliente
19 @param  cCliStore      Codigo da Loja
20 @return aRet[1]        LRULEAPPLY          1 // Alguma regra foi aplicada?
21 @return aRet[2]        NNEWTOTAL          2 // Valor total depois da aplicada(s) a(s) regra(s)
22 @return aRet[3]        NPERDESCTO        3 // Percentual de desconto total aplicado em relacao ao valor total antigo
23 @return aRet[4]        NOLDTOTAL          4 // Valor total antes da aplicada(s) a(s) regra(s)
24 @obs
25 @sample
26 */
27 //-----
28 Function STBItemRldi( nItemLine , cItemCode , nValueItem , nAmount , ;
29                     cTypeProd , cCliCode , cCliStore )
30
31 Local aRet          := {} // Retorno da Funcao
32 Local lApplyRule    := .F. // Aplicou alguma Regra de Desconto?
33 Local nNewValueItem := 0 // Novo valor do Item
34 Local nValueDiscount := 0 // Valor do desconto que foi aplicado
35 Local nDiscPerTotal := 0 // Percentual total do desconto pela(s) regras(s). Pode ocorrer mais de uma regra.
36 Local nTotValueDiscount := 0 // Valor total dos descontos aplicados acumulados
37 Local aRules        := {} // Regras de Desconto no Total
38 Local aAppRules      := {} // Armazena Informacoes das Regras Aplicadas
39 Local nI             := 0 // Contador
40 Local oModelCesta    := STDGPBModel() // Model da cesta
41 Local nQuantBuy      := 0 // Contador da quantidade de produtos já comprados e que estao na cesta
42 Local yI             := 0 // Contador
43 Local nX             := 0 // Contador
44 Local aProduto       := {} // Array com as informações do produto a ser consultado se existe estoque
45 Local aLocal         := {} // Array com as informações sobre a loja do produto que devera ser consultado se existe estoque
46 Local aSldEst       := {} // Array de retorno da função "Lj7SldEst" que consta as informações sobre o estoque
47 Local aBrindes       := {} // Array com os brindes disponiveis em estoque
48 Local aBonificados   := {} // Array com as bonificacoes da regra
49 Local nReGRAQTDE     := 0 // Quantidade de produtos necessarios comprar para ganhar o brinde.
50 Local nCont          := 0 // Contador do array que armazena o brindes disponiveis no estoque
51 Local nPrice         := 0 // preco do item bonificado

```

```

52 Local cTPRegra           := ""           // Tipo de regra (Item ou Total)
53 Local cRefGrid           := ""           // referencia do item(produto ou regra do total) na grid
54 Local cProduto           := ""           // Produto da grid MB8
55 Local oTotal             := STFGetTot()   // Objeto ref. ao valor da venda para controle se exibe ou não as opções de brinde/bonificacao
56 Local nSumRegraQtde      := 0            // Variavel para controle de regra acumulativa por item - MB8
57 Local lOK                := .T.          // Controla a aplicacao das regras de Desconto
58 Local lGrava             := .T.          // Controle sobre a gravacao do brinde
59 Local nPosQuant          := 0            // Posição Quantidade (*)
60 Local nQuantLanc         := 0            // Quantidade (*)
61 Local cGetProd           := ""           // Código do Produto definido na quantidade
62 Local nRegMulti          := 0            // Utilizado para verificar se regra é multipla
63 Local lDesQtdIte        := .F.          // Indica se concede o desconto no item lançado caso a quantidade lançada for maior ou igual a "Qtd. Venda" configurada no campo
    MB8_QTDPRO
64 Local aSTItRlDisc        := {}           // Retorno ponto de entrada
65 Local nQtdSaldo          := 0            // Quantidade Saldo do produto para quantificar se atinge a Regra de Quantidade para conceder o desconto
66 Local nPosProd           := 0
67 Local lMultQtde          := .F.          // Indica se a regra é multiplo de quantidade
68 Local nQtdeApply         := 1            // Quantidade considerada para aplicação da Regra
69 Local nRecnoMB8          := 0
70 Local lAchouMB8          := .F.
71 Local lSTBConstCt        := ExistFunc("STBCONSTCT") //Verifica se a função STBConstCt() existe
72 Local lBrinde            := .F. //Variavel para não permitir a execução do Brinde até que seja homologada corretamente.
73
74 Default nItemLine        := 0
75 Default cItemCode        := ""
76 Default nValueItem       := 0
77 Default nAmount          := 1
78 Default cTypeProd        := ""
79 Default cCliCode         := ""
80 Default cCliStore        := ""
81
82 // Abaixo avalia a string digitada. Se na string cGetProd houver '*' sera setada quantidade
83 // Necessário para aplicar o desconto da regra de desconto varejo
84 cGetProd := AllTrim(STIGetProd())
85 nPosQuant := AT(";",cGetProd)
86
87 If nPosQuant > 0
88     nQuantLanc := STIComma(SubStr(cGetProd,1,nPosQuant-1))
89 Else
90     nQuantLanc := nAmount
91 EndIf
92
93 nValueItem := nValueItem * nAmount
94 nNewValueItem := nValueItem
95
96 aRules := STDItemRlDi( cItemCode , nValueItem , cTypeProd , cCliCode , ;
97     cCliStore, "I" )
98
99 If ExistBlock("STItRlDisc")
100     aSTItRlDisc := ExecBlock("STItRlDisc",.F.,.F.,{cItemCode,nValueItem,nAmount})
101     If Valtype(aSTItRlDisc) == "A" .And. Len(aSTItRlDisc) == 3
102         lApplyRule := aSTItRlDisc[1] //Se aplica a regra de desconto customizado.

```

```

103     nNewValueItem      := aSTItRlDisc[2] //Novo valor total do item (valor unitario*quantidade) - desconto customizado.
104     nTotValueDiscount  := aSTItRlDisc[3] //Novo valor de desconto do item (valor total do desconto do item).
105     LjGrvLog("Regra de desconto", "STItRlDisc - Desconto no item " + cItemCode + " no valor de:",nTotValueDiscount) //#"Regra de desconto"
106     LjGrvLog("Regra de desconto", "STItRlDisc - Valor do item apos desconto: ",nNewValueItem) //#"Regra de desconto"
107     If nNewValueItem + nTotValueDiscount <> nValueItem
108         LjGrvLog("Regra de desconto", "STItRlDisc - Valor retornado difere do valor original: ",nValueItem) //#"Regra de desconto"
109         lApplyRule := .F.
110     Endif
111 EndIf
112 EndIf
113
114 If Len(aRules) > 0
115
116     //-----
117     // Ordena o array pela ordem de priorizacao no qual serao aplicadas as regras
118     //-----
119     aRules := STBOrdRegra(aRules)
120
121     oModelSL2 := oModelCesta:GetModel("SL2DETAIL")
122
123     If oModelSL2:Length() == 0 .Or. oModelSL2:IsEmpty()
124         aProdsVda := {} //Limpa o array caso seja o primeiro item que esteja sendo lançado
125     EndIf
126
127     /*/
128     Checando a quantidade já comprada do produto
129     /*/
130     For yI := 1 To oModelSL2:Length()
131         If !oModelSL2:IsEmpty()
132             oModelSL2:GoLine(yI)
133             If AllTrim(oModelSL2:GetValue("L2_PRODUTO")) == AllTrim(cItemCode)
134                 If !oModelSL2:IsDeleted(yI)
135                     nQuantBuy += oModelSL2:GetValue("L2_QUANT")
136                 EndIf
137             EndIf
138         Endif
139     Next yI
140
141
142     nPosProd := aScan( aProdsVda, { |x| x[1] == AllTrim(cItemCode) } )
143     If nPosProd > 0
144         nQtdSaldo := aProdsVda[nPosProd][3]
145
146         aProdsVda[nPosProd][2] += nQuantLanc
147         aProdsVda[nPosProd][3] += nQuantLanc
148     Else
149         aAdd( aProdsVda, { AllTrim(cItemCode) ,; //01-Codigo do Produto
150                         nQuantLanc ,; //02-Quantidade Vendida
151                         nQuantLanc ,; //03-Saldo de Quantidade
152                         {} } ,; //04-Regras Aplicadas
153
154         nPosProd := Len(aProdsVda)

```

```

155 EndIf
156
157 nQuantBuy := nQuantBuy + nQuantLanc //Quantidade Total Comprada do Produto
158 nQtdSaldo := nQtdSaldo + nQuantLanc //Quantidade Saldo do produto que ainda nao foi aplicada nenhuma regra
159
160 For nI := 1 To Len(aRules)
161
162     //Reinicializa variaveis
163     lOK := .T.
164     nQtdeApply := 1
165     cTPRegra := ""
166     cRefGrid := ""
167     cProduto := ""
168     nRecnoMB8 := 0
169     lAchouMB8 := .F.
170
171     If Len(aRules[nI]) >= 14
172         nRecnoMB8 := aRules[nI][14]
173     EndIf
174
175     If aRules[nI][8] == "PR0"
176         DbSelectArea("MB8")
177
178         If nRecnoMB8 > 0
179             MB8->(DbGoTo(nRecnoMB8))
180             lAchouMB8 := MB8->(!EoF())
181         Else
182             MB8->(DbSetOrder(4)) //MB8_FILIAL+MB8_CODREG+MB8_CODPRO
183             //Tenta buscar por Produto
184             lAchouMB8 := MB8->(DbSeek(xFilial("MB8")+aRules[nI][1]+aRules[nI][7]))
185             If !lAchouMB8
186                 //Tenta buscar por Categoria
187                 MB8->(DbSetOrder(1)) //MB8_FILIAL+MB8_CODREG+MB8_REFGRD
188                 lAchouMB8 := MB8->(DbSeek(xFilial("MB8")+aRules[nI][1]))
189             EndIf
190         EndIf
191
192         If lAchouMB8
193             cTPRegra := MB8->MB8_TPREGR //1=Desconto;2=Bonificacao;3=Brinde
194             cRefGrid := PADL(MB8->MB8_REFGRD, TamSX3("MB8_REFGRD")[1], "0")+ "MB8"
195
196             If Empty(cProduto)
197                 cProduto:= MB8->MB8_CODPRO
198             Endif
199
200             If Empty(alltrim(cProduto)) .And. (aRules[nI][12] == .T.)//Produto de Categoria
201                 cProduto := alltrim(cItemCode)
202             Endif
203         Endif
204     Else
205
206         cTPRegra := aRules[nI][9] // que corresponde ao campo MB2_TPREGR

```

```

207         cRefGrid := aRules[nI][10]+"MB2"      // que corresponde ao campo MB2_REFGRD
208         cProduto := ""                        // quando a regra for por total nao existe produto
209
210     EndIf
211     // Verifica campo MB8_DESCTO
212     /*
213     Sobre o campo MB8_DESCTO:  MB8_DESCTO == "1"
214     Ex: Item: 002 --- Preço: R$ 200,00 --- Desconto %: 10% --- Qtd Venda: 5 --- Desc Total = Sim
215     Ao adicionar 5 vezes o item 002 separadamente na tela de atendimento o sistema não irá conceder o desconto. Para esta funcionalidade ser aplicada é preciso lançar
5*002.
216     Assim, o sistema irá aplicar o desconto de 10% (R$ 100,00), sendo R$ 20,00 em cada item. O total da venda será de R$ 900,00.
217     Esta regra é aplicada toda vez que a quantidade de itens lançados for igual ou superior ao campo Qtd Venda. Esta regra não é cumulativa,
218     o desconto somente será aplicado quando a quantidade do item atingir a quantidade de venda estabelecida na regra.
219     Fonte: http://tdn.totvs.com/pages/viewpage.action?pageId=215483464
220     */
221
222     //Verifica se concede o desconto no item lançado caso a quantidade lançada for maior ou igual a "Qtd. Venda" configurada no campo MB8_QTDPRO
223     lDesQtdIte := MB8->MB8_DESCTO == "1" //1=Sim
224
225     /*
226     Sobre o campo MB8_QTDMUL ( Multiplo de quantidade )
227     Para uma regra configurada com o campo Múltiplo Qtd = Não, o sistema fornece o desconto apenas uma vez na venda, assim que a quantidade de itens for igual ou superior
ao campo de Qtd. Venda (MB8_QTDPRO).
228     Caso o campo Múltiplo Qtd (MB8_QTDMUL) = SIM o desconto é aplicado toda vez que a quantidade atingir o campo de Qtd. Venda. O desconto é aplicado somente no último
item da venda.
229
230     Ex: Item: 001 --- Preço: R$ 100,00 --- Desconto %: 50% --- Multiplo Qtd: Sim --- Qtd Venda: 5
231     Ao adicionar 5 vezes o item 001 na tela de atendimento o sistema irá conceder o desconto de 50% (R$ 50,00) no último item, sendo assim, o total da venda será de R$
450,00.
232     Esta regra é cumulativa e o sistema verifica o total de itens na tela para verificar se pode ou não aplicar a regra de desconto.
233     */
234
235     lMultQtde := MB8->MB8_QTDMUL == "1" //1=Sim
236
237     Do Case
238         Case cTPRegra = '1'//DESCONTO
239
240             //-----
241             // Validacao por Quantidade
242             //-----
243             If MB8->MB8_QTDPRO > 0
244
245                 lOK := .F.
246
247                 If lDesQtdIte // Concede o desconto no item lançado caso a quantidade lançada for maior ou igual a "Qtd. Venda" configurada no campo MB8_QTDPRO
248                     //Verifica se a Quantidade Lançada do Item Atinge a quantidade configurada para conceder o desconto
249                     If nQuantLanc >= MB8->MB8_QTDPRO
250                         lOK := .T.
251                         nQtdeApply := nQuantLanc
252                     Endif
253                 Else
254                     // Regra por Compra acumulativa do mesmo produto, verifica se a quantidade comprada atinge a "Qtd. Venda" configurada no campo MB8_QTDPRO

```

```

255         If nQtdSaldo >= MB8->MB8_QTDPRO
256             lOK := .T.
257             nQtdeApply := nQtdSaldo
258         Endif
259     Endif
260 Endif
261
262 If lOK
263
264     If MB8->MB8_QTDPRO == 0
265         nRegMulti := Int(nQtdeApply / 1)
266     Else
267         nRegMulti := Int(nQtdeApply / MB8->MB8_QTDPRO)
268     EndIf
269
270     nQtdSaldo := nQtdSaldo - (MB8->MB8_QTDPRO * nRegMulti) //Atualiza o Saldo da Qdade do Produto
271
272 Do Case
273     Case aRules[nI][MB8_DESCPR] > 0 // PERCENTUAL
274
275         If lMultQtde
276             nValueDiscount := ( ( nNewValueItem / nQuantLanc ) * ( aRules[nI][MB8_DESCPR] / 100 ) ) * nRegMulti
277         Else
278             nValueDiscount := ( ( nNewValueItem ) * ( aRules[nI][MB8_DESCPR] / 100 ) )
279         EndIf
280
281         LjGrvLog("Regra de desconto", "STItRlDisc - Aplicado a regra: " + MB8->MB8_CODREG, cProduto)
282         LjGrvLog("Regra de desconto", "STItRlDisc - Percentual do Desconto: " + STR(aRules[nI][MB8_DESCPR]) , cProduto)
283         LjGrvLog("Regra de desconto", "STItRlDisc - Valor do Desconto: " + STR( nValueDiscount ) , cProduto)
284
285         If (nNewValueItem - nValueDiscount) >= 0.01
286             /*
287              Chamada Desconto na Proxima Venda
288             */
289             If STWNxtSalD(      "I"          , nItemLine      , cItemCode      , nNewValueItem , ;
290                           nAmount      , aRules[nI][MEI_CODREG] , aRules[nI][MB8_DESCPR] , NIL          )
291
292             Else
293
294                 lApplyRule := .T.
295
296                 nTotValueDiscount := nTotValueDiscount + nValueDiscount
297                 nNewValueItem      := nNewValueItem - nValueDiscount
298
299                 AADD( aAppRules , { aRules[nI] } )
300
301                 aAdd( aProdsVda[nPosProd][4], MB8->MB8_CODREG+MB8->MB8_REFGRD ) //Adiciona no array a Regra aplicada
302                 aProdsVda[nPosProd][3] := nQtdSaldo //Atualiza o Saldo de Quantidade do Produto que ainda nao foi aplicado Regra de Desconto
303
304             EndIf
305
306         EndIf

```

```

307
308         Case aRules[nI][MB8_DESCVL] > 0 // VALOR
309
310             If lMultQtde
311                 nValueDiscount := aRules[nI][MB8_DESCVL] * nRegMulti
312             Else
313                 nValueDiscount := aRules[nI][MB8_DESCVL]
314             EndIf
315
316             LjGrvLog("Regra de desconto", "STItRlDisc - Aplicado a regra: " + MB8->MB8_CODREG, cProduto)
317             LjGrvLog("Regra de desconto", "STItRlDisc - Valor do Desconto: " + STR( aRules[nI][MB8_DESCVL] ) , cProduto)
318
319             If (nNewValueItem - nValueDiscount) >= 0.01
320                 /*
321                  Chamada Desconto na Proxima Venda
322                 */
323                 If STWNxtSalD( "I"           , nItemLine       , cItemCode       , nNewValueItem       , ;
324                             nAmount       , aRules[nI][MEI_CODREG] , NIL             , aRules[nI][MB8_DESCVL] )
325
326                     Else
327
328                         lApplyRule := .T.
329
330                         nTotValueDiscount := nTotValueDiscount + nValueDiscount
331                         nNewValueItem      := nNewValueItem - nValueDiscount
332
333                         AADD( aAppRules , { aRules[nI] } )
334
335                         aAdd( aProdsVda[nPosProd][4], MB8->MB8_CODREG+MB8->MB8_REFGRD ) //Adiciona no array a Regra aplicada
336                         aProdsVda[nPosProd][3] := nQtdSaldo //Atualiza o Saldo de Quantidade do Produto que ainda nao foi aplicado Regra de Desconto
337
338                     EndIf
339
340             EndIf
341
342         EndCase
343     Endif
344
345     Case cTPRegra = '2'//BONIFICACAO
346
347         /*
348          Verifica a quantidade necessaria de compra do produto para ganhar a Bonificacao
349         */
350
351         //Deve-se considerar o While porque o usuario pode ter o mesmo produto cadastrado varias vezes dentro da MB8 para
352         //atender a quantidade diferentes
353         If aRules[nI][8] == "PRO" // Ira executar somente se a regra for por item
354             while MB8->(BOF() .And. MB8->MB8_FILIAL+MB8->MB8_CODREG = xFilial("MB8")+aRules[nI][1]) //Regra com regra
355                 If (MB8->MB8_TPREGR = '2')//Bonificacao
356                     If lSTBConstCt .AND. aRules[nI][12] //por Categoria
357                         If STBConstCt(MB8->MB8_CATEGO, cItemCode)
358                             nRegraQtde := MB8->MB8_QTDPRO //Quantidade necessaria de Produtos para comprar

```

```

359      If nQuantBuy >= nRegraQtde          //Aqui fica assegurado que o sistema trata a regra do mesmo produto qdo o mesmo atingir a qtde
necessaria de compra.
360          Exit
361      Endif
362  EndIf
363      ElseIf (MB8->MB8_CODPRO = alltrim(cItemCode))          //por Produto
364          nRegraQtde := MB8->MB8_QTDPRO          //Quantidade necessaria de Produtos para comprar
365          If nQuantBuy >= nRegraQtde          //Aqui fica assegurado que o sistema trata a regra do mesmo produto qdo o mesmo atingir a qtde necessaria
de compra.
366              Exit
367          Endif
368      Else
369          nRegraQtde := MB8->MB8_QTDPRO          //Quantidade necessaria de Produtos para comprar
370      Endif
371  EndIf
372
373      MB8->(DbSkip())
374  EndDo
375  Else
376      nRegraQtde:= 0// LEANDRO TESTAR ESSE TRECHO PQ SE É POR TOTAL NAO TEM QUE EXIBIR BONIFICACAO QDO A REGRA(aRules[nI][8] == "PRO") É IGUAL A TOT
377  Endif
378
379  //Bonificacao
380  If nQuantBuy >= nRegraQtde
381
382      DbSelectArea("MGB")
383      MGB->(DbSetOrder(1))
384      If MGB->(DbSeek(xFilial("MGB")+aRules[nI][1]+cRefGrid)) //MGB_FILIAL+MGB_CODREG+MGB_IDPROD
385          While MGB->(!BOF() .And. MGB->MGB_FILIAL+MGB->MGB_CODREG+MGB->MGB_IDPROD = xFilial("MGB")+aRules[nI][1]+cRefGrid)
386              nPrice := STBSearchPrice(MGB->MGB_CODPRO)
387
388              If len(aBonificados) > 0
389                  For nX := 1 to len(aBonificados)
390                      If alltrim(aBonificados[nX][5]) == alltrim(MGB->MGB_CODPRO )
391                          lGrava := .F.
392                      Endif
393                  Next nX
394              Endif
395
396              If lGrava
397                  aAdd(aBonificados,{AllTrim(MGB->MGB_CODPRO)+" - "+AllTrim(Posicione( "SB1",1,xFilial( "SB1" )+alltrim(MGB->MGB_CODPRO),"SB1->B1_DESC" )),;
398                      MGB->MGB_DESCPR, ;
399                      MGB->MGB_DESCVL,;
400                      nPrice, ;
401                      MGB->MGB_CODPRO,;
402                      .F.})
403              EndIf
404
405              MGB->(DbSkip())
406          EndDo
407      Endif
408

```

```

409         Endif
410     Case cTPRegra = '3' .AND. lBrinde //BRINDE
411
412         /*/
413         Verifica a quantidade necessaria de compra do produto para ganhar o Brinde
414         /*/
415         If aRules[nI][8] == "PRO" // somente ira executar se a regra for por item
416             while MB8->!BOF() .And. MB8->MB8_FILIAL+MB8->MB8_CODREG = xFilial("MB8")+aRules[nI][1]) //Regra com regra
417                 If (cProduto = alltrim(cItemCode)) .And. (MB8->MB8_TPREGR = '3')//Brinde
418                     nRegraQtde := MB8->MB8_QTDPRO
419                     If nRegraQtde == (nQuantBuy +1)//Aqui fica assegurado que o sistema trata a regra do mesmo produto qdo o mesmo atingir a qtde necessaria de
compra.
420                         If !lMultQtde // se a regra não for acumulativa, deve-se sair do loop
421                             Exit
422                         Endif
423                     Else
424                         If lMultQtde
425                             //Regra para exibir as opções de brinde por Compra acumulativa do mesmo produto
426                             nSumRegraQtde := nRegraQtde
427                             If nQuantBuy <> 0
428                                 While !(nSumRegraQtde > (nQuantBuy+1))
429                                     If ((nQuantBuy+1) == nSumRegraQtde)
430                                         Exit
431                                     Endif
432                                     nSumRegraQtde := nSumRegraQtde + nRegraQtde
433                                 EndDo
434                                 nRegraQtde := nSumRegraQtde
435                             Endif
436                         Endif
437                     Endif
438                 Endif
439                 MB8->(DbSkip())
440             EndDo
441         Else
442             //Quando a regra for Brinde por Total
443             If oTotal:GetValue("L1_VALBRUT") < aRules[nI][11]
444                 DbSelectArea("MB4")
445                 MB4->(DbSetOrder(1))
446                 If MB4->(DbSeek(xFilial("MB4")+aRules[nI][1]))
447                     nRegraQtde:= 0 // Nao podera exibir as opções de brinde
448                 Else
449                     nRegraQtde:= 1 // Para poder exibir as opções de brinde porque esta associada a Forma de Pagamento
450                 Endif
451             Else
452                 nRegraQtde:= 0 // Nao ira exibir as opções de brinde porque o valor da compra ultrapassou o intervalo que merece o brinde
453             Endif
454         Endif
455         //Brinde
456         If nRegraQtde == (nQuantBuy +1)
457
458
459             DbSelectArea("MGB")

```

```

460         MGB->(DbSetOrder(1))
461         If MGB->(DbSeek(xFilial("MGB")+aRules[nI][1]+cRefGrid)) //MGB_FILIAL+MGB_CODREG+MGB_IDPROD
462             nCont := 1
463             While MGB->(!BOF() .And. MGB->MGB_FILIAL+MGB->MGB_CODREG+MGB->MGB_IDPROD = xFilial("MGB")+aRules[nI][1]+cRefGrid)
464
465                 aadd( aProduto , { .T.,alltrim(oModelCesta:GetValue("SL1MASTER" , "L1_LOJA")), MGB->MGB_CODPRO ,AllTrim(Posicione( "SB1",1,xFilial( "SB1"
)+alltrim(MGB->MGB_CODPRO),"SB1->B1_DESC" )) , MGB->MGB_QTDPRO })
466                 aAdd( aLocal, alltrim(oModelCesta:GetValue("SL1MASTER" , "L1_LOJA")))
467
468                 //lContinua := STBRemoteExecute("Lj7SldEst" , { aProduto , aLocal} , NIL, .F. , @aSldEst )
469
470                 //If lContinua
471                 //If aSldEst[nCont][5][nCont][2] > 0 //Produtos disponiveis em estoque.
472                 If len(aBrindes) > 0
473                     For nX := 1 to len(aBrindes)
474                         If alltrim(substr(aBrindes[nX],1,at('-', aBrindes[nX])-1)) == alltrim(MGB->MGB_CODPRO )
475                             lGrava := .F.
476                         Endif
477                     Next nX
478                 Endif
479
480                 If lGrava
481                     aAdd(aBrindes,AllTrim(MGB->MGB_CODPRO)+" - "+AllTrim(Posicione( "SB1",1,xFilial( "SB1" )+alltrim(MGB->MGB_CODPRO),"SB1->B1_DESC" )))
482                 Endif
483                 //Endif
484             //Endif
485
486             MGB->(DbSkip())
487
488             nCont := nCont +1
489
490         EndDo
491     Endif
492
493     //Executa a tela para selecionar as opções de brinde
494     If !Empty(aBrindes)
495         STIPanBrindeSelection(aRules[nI][1],cRefGrid,cProduto,aBrindes, .T.)
496     Endif
497
498 Endif
499 EndCase
500
501 // Se acumulativo verifica as demais regras, ou seja, caso exista mais de uma regra para o mesmo produto sera executado,
502 //caso não seja acumulativo ira executar somente a primeira regra
503 If lApplyRule .AND. aRules[nI][MEI_ACUMUL] == "N"
504     Exit
505 EndIf
506 Next nI
507 EndIf
508
509 If lApplyRule
510

```

```

511     nDiscPerTotal := ( nTotValueDiscount * 100 ) / nValueItem // Percentual total depois da(s) regras(s) aplicadas
512
513     If STFGetCfg("lRuleDiscShow") //Se mostra regras aplicadas
514
515         STFMessage("STItemRlDi","ALERT", STR0001 + AllTrim(Transform(nTotValueDiscount,PesqPict("SL2","L2_VALDESC"))) ) //"Regra de Desconto: Aplicado desconto no item de: "
516
517         For nI := 1 To Len(aAppRules)
518
519             STFMessage("STItemRlDi","ALERT", STR0002 + aAppRules[nI][1][4] ) // "Regra de Desconto: "
520
521         Next nI
522
523     EndIf
524
525 EndIf
526
527 AADD( aRet , lApplyRule      )
528 AADD( aRet , nNewValueItem   )
529 AADD( aRet , nTotValueDiscount )
530 AADD( aRet , nDiscPerTotal   )
531 AADD( aRet , nValueItem      )
532
533 STFSHOWMESSAGE("STItemRlDi")
534
535 Return aRet
536
537 //-----
538 /*{Protheus.doc} STBLimpRegra
539 Limpa variavel de verificação da regra de desconto no item
540 Utilizado para validar se venda ja teve desconto no item.
541 Necessario quando regra de de desconto não é multipla
542 @param
543 @return      lRet
544 @obs
545 @sample
546 */
547 //-----
548
549 Function STBLimpRegra(lRet)
550 Default lRet := .F.
551
552 lLimpRegra := lRet
553
554 Return
555
556 //-----
557 /*{Protheus.doc} STBOrdRegra
558 Ordena no array as regras de desconto por item.
559 Faz a ordenacao na sequencia em que serao analisadas cada regra.
560
561 @type      function
562

```

```

563 @param aRegraItem, Array, Relacao de Regras de Desconto disponiveis/ativas para aplicacao
564
565 @return Array, Relacao de Regras de Desconto disponiveis/ativas para aplicacao ordenada
566 na sequencia em que serao analisadas cada regra.
567 */
568 //-----
569 Static Function STBOrdRegra(aRegraItem)
570 Local nX      := 0
571 Local nY      := 0
572 Local cCodRegra := ""
573 Local nCount  := 0
574 Local nPosIni := 0
575
576 //Primeiro ordena por Prioridade de Regra (MEJ_PRINUM)
577 aSort(aRegraItem,,,{|x,y| x[2]+[1] < y[2]+[1]})
578
579 //Agora, ordena por ordem decrescente, a quantidade minima de compra (MB8_QTDPRO) para aplicacao de desconto
580 If Len(aRegraItem) > 0 .And. Len(aRegraItem[1]) > 12
581     For nX:=1 To Len(aRegraItem)
582         cCodRegra := aRegraItem[nX][1]
583         nCount := 0
584         nPosIni := nX
585
586         For nY := nX To Len(aRegraItem)
587             If aRegraItem[nY][1] <> cCodRegra
588                 nY := nY-1
589                 Exit
590             EndIf
591             nCount++
592         Next nY
593
594         //Ordena por ordem decrescente a quantidade minima de compra para aplicacao de desconto de um determinado produto
595         //Isso eh necessario para que seja comparada primeiramente se a maior quantidade vendida eh atingida, pois normalmente eh a que possui maior valor de desconto
596         aSort( aRegraItem, nPosIni, nCount, { |x,y| x[13] > y[13] } )
597
598         nX := nY
599     Next nX
600 EndIf
601 EndIf
602
603 Return aRegraItem

```

Componentes/Discount/STBRuleDiscountPay.PRW

276 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STPOS.CH"
4 #INCLUDE "STBRULESDISCOUNTPAY.CH"
5
6 //-----
7 /*{Protheus.doc} STBPayRlDi
8 Function Regra de Desconto Pagamento
9
10 @param nValueTotal      Valor Total da Venda
11 @param nValueBalancePay  Valor Pagamento
12 @param cFormPay          Forma Pagamento
13 @param aAllPay           Array com todos pagamentos
14 @param cCliCode          Codigo do Cliente
15 @param cCliStore         Loja do Cliente
16 @return Aret[1]          lApplyRule // Aplica regra
17 @return Aret[2]          nNewValueTotal // Valor total
18 @return Aret[2]          nTotValueDiscount // Valor Total de desconto
19 @return Aret[2]          nDiscPerTotal // Percentual de desconto
20 @return Aret[2]          nValueTotal // Valor Total
21 @return Aret[2]          aNewAllPay // Array com todos pagamentos
22 @obs
23 @sample
24 */
25 //-----
26
27
28 Function STBPayRlDi( nValueTotal , nValueBalancePay , cFormPay , aAllPay , ;
29                    cCliCode , cCliStore )
30
31 Local aRet := {} // Retorno da Funcao
32 Local lApplyRule := .F. // Aplicou alguma Regra de Desconto?
33 Local lIsProportional := .F. // Indica se a Regra eh proporcional
34 Local lFormsOk := .T. // Verifica se TODAS as formas de pagamento da venda estao na regra (NAO PROPORCIONAL)
35 Local nNewValueTotal := 0 // Novo Total. Desconto aplicado
36 Local nValueDiscount := 0 // Valor do desconto que foi aplicado
37 Local nDiscPerTotal := 0 // Percentual total do desconto pela(s) regras(s). Pode ocorrer mais de uma regra.
38 Local nTotValueDiscount := 0 // Valor total dos descontos aplicados acumulados
39 Local nTotPay := 0 // Valor total dos pagamentos passados no aAllPay
40 Local aNewAllPay := {} // Array de pagamentos baseado no passado por parâmetro com os valores modificados para o ultimo pagamento(coloca o desconto no ultimo)
41 Local aRules := {} // Regras de Desconto no Total
42 Local aRange := {} // Range de valores da Regra
43 Local aForms := {} // Formas de Pagamento da Regra
44 Local aFormsOK := {} // Formas de pagamento utilizadas na venda que estao na regra e serao aplicadas
45 Local aAppRules := {} // Armazena Informacoes das Regras Aplicadas
46 Local nPosForm := 0 // Armazena posicao da forma de pagamento
47 Local nJ := 0 // Contador
48 Local nI := 0 // Contador
49 Local nX := 0 // Contador
50 Local nY := 0 // Contador
51

```

```

52 Default nValueTotal      := 0
53 Default nValueBalancePay := 0
54 Default cFormPay         := ""
55 Default aAllPay          := {}
56 Default cCliCode         := ""
57 Default cCliStore        := ""
58
59 ParamType 0 Var nValueTotal AS Numeric      Default 0
60 ParamType 1 Var nValueBalancePay AS Numeric Default 0
61 ParamType 2 Var cFormPay AS Character      Default ""
62 ParamType 3 Var aAllPay AS Array           Default {}
63 ParamType 4 Var cCliCode AS Character      Default ""
64 ParamType 5 Var cCliStore AS Character      Default ""
65
66 nNewValueTotal := nValueTotal
67
68 If nValueTotal >= 0.01
69
70     aRules := STDTotalDi( cCliCode , cCliStore , .T. )
71
72     If Len(aRules) > 0
73
74         /*/
75         Ordena o array pelo segundo campo MEJ->MEJ_PRINUM (Prioridades)
76         /*/
77         aRules := aSort(aRules,,,{|x,y| x[MEJ_PRINUM] < y[MEJ_PRINUM]})
78
79         For nI := 1 To Len( aRules )
80
81             /*/
82             Busca o Range da Regra
83             /*/
84             aRange := STDRangRul( aRules[nI][MEI_CODREG] )
85             /*/
86             Busca as Formas de Pagamento
87             /*/
88             aForms := STDPayForms( aRules[nI][MEI_CODREG] )
89             /*/
90             Verifica se eh proporcional
91             /*/
92             lIsProportional := STDPropForm( aRules[nI][MEI_CODREG] )
93
94             For nJ:= 1 To Len( aRange )
95
96                 If nValueTotal >= aRange[nJ][MB2_VLRINI] .AND. nValueTotal <= aRange[nJ][MB2_VLRFIM] // Valida Range
97
98                     If lIsProportional // PROPORCIONAL
99
100                         /*/
101                         Verifica se a forma de pagamento utilizada na venda pertence a regra
102                         /*/
103                         For nX:=1 To Len( aAllPay )

```

```

104
105         nPosForm := aScan( aForms, {|x| x == aAllPay[nX][2] })
106
107         If nPosForm > 0
108
109             AADD( aFormsOK , aAllPay[nX][2] ) // Formas de pagamento que estao na venda e no cadastro da regra
110
111         EndIf
112
113     Next nX
114
115     /*/
116     Aplicar a regra para as formas validas
117     /*/
118     For nY:=1 To Len( aFormsOK )
119
120         nPosForm := aScan( aAllPay, {|x| x[2] == aFormsOK[nY] })
121
122         Do Case
123
124             Case aRange[nJ][MB2_PDESC] > 0 // PERCENTUAL
125
126                 lApplyRule := .T.
127                 /*/
128                 Realizar calculos
129                 /*/
130                 aAllPay[nPosForm][1] := aAllPay[nPosForm][1]
131                 nValueDiscount      := aAllPay[nPosForm][1] * ( aRange[nJ][MB2_PDESC] / 100 )
132                 nTotValueDiscount   := nTotValueDiscount + nValueDiscount
133                 nNewValueTotal      := nNewValueTotal - nValueDiscount
134
135                 AADD( aAppRules , { aRules[nI] , aRange[nJ] } )
136
137             Case aRange[nJ][MB2_VLRFEX] > 0 // VALOR
138
139                 If !((aAllPay[nPosForm][1] - aRange[nJ][MB2_VLRFEX]) < 0.01)
140
141                     lApplyRule := .T.
142                     /*/
143                     Realizar calculos
144                     /*/
145                     nValueDiscount      := aRange[nJ][MB2_VLRFEX]
146                     nTotValueDiscount   := nTotValueDiscount + nValueDiscount
147                     nNewValueTotal      := nNewValueTotal - nValueDiscount
148
149                     AADD( aAppRules , { aRules[nI] , aRange[nJ] } )
150
151                 EndIf
152             EndCase
153
154         Next nY
155

```

```

156
157 Else // NAO PROPORCIONAL
158     /*/
159     Para Regra NAO PROPORCIONAL, TODAS as formas do pagamento da VENDA devem estar cadastradas na Regra
160     /*/
161     lFormsOk := .T. // Volta para o Default
162
163     For nX:=1 To Len( aAllPay )
164
165         nPosForm := aScan( aForms, {|x| x == aAllPay[nX][2] })
166
167         If nPosForm == 0
168
169             lFormsOk := .F.
170             Exit
171
172         EndIf
173
174     Next nX
175
176     If lFormsOk // Todas as formas estao na venda
177         /*/
178         Calculo dos valores da regra. (Acumulativo para mais de uma regra)
179         /*/
180         Do Case
181             Case aRange[nJ][MB2_PDESC] > 0 // PERCENTUAL
182
183                 If !((nNewValueTotal - (nNewValueTotal * (aRange[nJ][MB2_PDESC] / 100 ))) < 0.01)
184
185                     lApplyRule := .T.
186                     /*/
187                     Realizar calculos sempre por valor
188                     /*/
189                     nValueDiscount      := nNewValueTotal * ( aRange[nJ][MB2_PDESC] / 100 )
190                     nTotValueDiscount  := nTotValueDiscount + nValueDiscount
191                     nNewValueTotal     := nNewValueTotal - nValueDiscount
192
193                     AADD( aAppRules , { aRules[nI] , aRange[nJ] } )
194
195                 EndIf
196
197             Case aRange[nJ][MB2_VLRFx] > 0 // VALOR
198
199                 If !(aRange[nJ][MB2_VLRFx] > nNewValueTotal) .AND. !((nNewValueTotal - aRange[nJ][MB2_VLRFx]) < 0.01)
200
201                     lApplyRule := .T.
202                     /*/
203                     Realizar calculos sempre por valor
204                     /*/
205                     nValueDiscount      := aRange[nJ][MB2_VLRFx]
206                     nTotValueDiscount  := nTotValueDiscount + nValueDiscount
207                     nNewValueTotal     := nNewValueTotal - nValueDiscount

```

```

208
209             AADD( aAppRules , { aRules[nI] , aRange[nJ] } )
210
211             EndIf
212
213             EndCase
214
215             EndIf
216
217             EndIf
218
219             EndIF
220
221             Next nJ
222
223             // Se acumulativo nao verifica as demais
224             If lApplyRule .AND. aRules[nI][MEI_ACUMUL] == "N"
225                 Exit
226             EndIf
227
228             Next nI
229             EndIf
230 EndIf
231
232 If lApplyRule
233
234     nDiscPerTotal := ( nTotValueDiscount * 100 ) / nValueTotal // Percentual total depois da(s) regras(s) aplicadas
235
236     For nI:=1 To Len( aAllPay )
237
238         nTotPay := nTotPay + aAllPay[nI][1]
239
240     Next nI
241
242     aNewAllPay := aClone(aAllPay)
243
244     /*/
245     Joga a diferenca no ultimo pagamento informado
246     /*/
247     aNewAllPay[Len(aNewAllPay)][1] := aNewAllPay[Len(aNewAllPay)][1] - (( nTotPay + nTotValueDiscount ) - nValueTotal )
248
249     If STFGetCfg("lRuleDiscShow") //Se mostra regras aplicadas!!!
250
251         STFMessage("STPayRlDi","ALERT", STR0001) //"Regra de Desconto: Aplicado desconto no total segundo a(s)s seguinte(s) regra(s): "
252
253         For nI:=1 To Len(aAppRules)
254
255             STFMessage("STPayRlDi","ALERT",STR0002 + aAppRules[nI][1][4] ) // "Regra de Desconto: "
256
257         Next nI
258
259     EndIf

```

```
260
261 Else
262
263     //Se nao aplicou nenhuma regra, ver sugestao
264
265 EndIf
266
267 AADD( aRet , lApplyRule      )
268 AADD( aRet , nNewValueTotal  )
269 AADD( aRet , nTotValueDiscount )
270 AADD( aRet , nDiscPerTotal   )
271 AADD( aRet , nValueTotal      )
272 AADD( aRet , aNewAllPay       )
273
274 STFShowMessage( "STPayRldi" )
275
276 Return  aRet
```

Componentes/Discount/STBRuleDiscountTotal.PRW

711 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STPOS.CH"
4 #INCLUDE "STBRULESDISCOUNTTOTAL.CH"
5
6 Static ldescreg      := .F.
7 Static lVerReg       := .T. //Variavel responsavel em fazer o controle de validacao da regra de desconto no momento que inclui o pagamento no model de pagamentos
8 Static lContCart     := .T. //Variavel de controle para aplicar o desconto quando for CC ou CD
9 Static cFormaPg      := ""
10
11 //-----
12 /*/{Protheus.doc} STBTotRlDi
13 Function Regra de Desconto Total
14
15 @param  nValueTotal      Valor Total da Venda
16 @param  cCliCode        Codigo do Cliente
17 @param  cCliStore       Loja do Cliente
18 @return aRet[1]         LRULETOTAP      1 // Alguma regra foi aplicada?
19 @return aRet[2]         NNEWTOTAL      2 // Valor total depois da aplicada(s) a(s) regra(s)
20 @return aRet[3]         NPERDESCTO     3 // Percentual de desconto total aplicado em relacao ao valor total antigo
21 @return aRet[4]         NOLDTOTAL      4 // Valor total antes da aplicada(s) a(s) regra(s)
22 @obs
23 @sample
24 */
25 //-----
26 Function STBTotRlDi( nValueTotal, cCliCode, cCliStore, lPgto,;
27                    cForma, oPnlAdconal)
28
29 Local aRet          := {}           // Retorno da Funcao
30 Local lApplyRule    := .F.         // Aplicou alguma Regra de Desconto?
31 Local nNewValueTotal := 0           // Novo Total. Desconto aplicado
32 Local nValueDiscount := 0         // Valor do desconto que foi aplicado
33 Local nDiscPerTotal  := 0         // Percentual total do desconto pela(s) regras(s). Pode ocorrer mais de uma regra.
34 Local nTotValueDiscount := 0      // Valor total dos descontos aplicados acumulados
35 Local nValSuggestion := 0         // Valor que falta para alcanca o inicial do range da regra
36 Local aRules         := {}        // Regras de Desconto no Total
37 Local aRange         := {}        // Range de valores da regra
38 Local aAppRules      := {}        // Armazena Informacoes das Regras Aplicadas
39 Local aSuggestion    := {}        // Armazena Sugestoes de regra de desconto
40 Local nJ             := 0         // Contador
41 Local nI             := 0         // Contador
42 Local nX             := 0         // Contador
43 Local nK             := 0         // Contador
44 Local cSimbMoeda     := SuperGetMV( "MV_SIMB" + Str( STBGetCurrency(),1 ) )
45 Local oModelCesta    := STDGPBModel()// Model da cesta
46 Local cREFGRD        := ""        // Chave ref. a posicao da grid para a tabela MGB - Brindes
47 Local aProduto       := {}        // Array com as informações do produto a ser consultado se existe estoque
48 Local aLocal         := {}        // Array com as informações sobre a loja do produto que devera ser consultado se existe estoque
49 Local aBrindes       := {}        // Array com os brindes disponiveis em estoque
50 Local aSldEst        := {}        // Array de retorno da função "Lj7SldEst" que consta as informações sobre o estoque
51 Local nCont          := 0         // Contador do array que armazena o brindes disponiveis no estoque
52 Local aBonificados   := {}        // Array com os produtos bonificados disponiveis

```

```

53 Local nValor          := 0           // Valor de checagem para carregar a tela de selecao de produtos bonificados
54 Local lGrava          := .T.        // Controle sobre a gravacao do brinde
55 Local lProdDA1        := ExistFunc("STDProdDA1") //Verifica se existe a funcao STDProdDA1
56 Local oCesta          := Nil        //Recuperando o model da cesta de produto
57 Local lCodTab         := .F.        //Responsavel em verificar se o campo L2_CODTAB existe no model da cesta de produtos
58 Local nPgts           := STBGetPgto() //Guarda a quantidade de pagamentos informado na venda
59 Local cMsg            := ""
60 Local lDescGrp        := .F.
61 Local oModelMaster    := STFGetTot() //Estrutura dos campos da SL1 virtuais.
62 Local aForms          := {}         // Array que retorna as formas de pagamento da função STDPayForms
63
64 Default nValueTotal   := 0
65 Default cCliCode      := ""
66 Default cCliStore     := ""
67 Default cForma        := ""         // Forma de pagamento obrigatoria
68 Default lPgto         := .F.        // Checa Forma de pagamento
69 Default oPnlAdconal   := Nil        // Objeto do Resumo de Pagamento
70
71 oCesta := oModelCesta:GetModel('SL2DETAIL') //Recupera do modelo de dados a cesta de itens (SL2)
72 lCodTab := oCesta:HasField("L2_CODTAB") //Verifica se o campo virtual L2_CODTAB existe no model da SL2
73
74 // Indica que teve desconto no Total da venda sem ser pela regra de desconto.
75 If oModelMaster:HasField("L1_ARMDESC") .And. oModelMaster:GetValue("L1_ARMDESC") > 0
76     nTotValueDiscount := oModelMaster:GetValue("L1_ARMDESC")
77 EndIf
78
79 If !STIGetRecTit() .And. Empty(STIGetPgOr()) .And. Iif(ExistFunc("STIGetImp"), !STIGetImp(), .T.)
80     If (FWIsInCallStack("STIADDNEWPAN") .OR. FWIsInCallStack("STIADDPAY")) .AND. nPgts == 0
81         STIClearDisc(.T.)
82     ElseIf nPgts == 1 .AND. IsMoney(cFormaPg) .AND. IsMoney(cForma)
83         STIZeraPay()
84         STIClearDisc(.T.)
85     EndIf
86
87     cFormaPg := ""
88
89     If nValueTotal == 0
90         nValueTotal := STFSaleTotal()
91     EndIf
92
93     If Empty(cCliCode)
94         cCliCode := STDGPBasket( "SL1" , "L1_CLIENTE" )
95     EndIf
96
97     If Empty(cCliStore)
98         cCliStore := STDGPBasket( "SL1" , "L1_LOJA" )
99     EndIf
100
101
102
103 ParamType 0 Var nValueTotal AS Numeric      Default 0
104 ParamType 1 Var cCliCode AS Character      Default ""

```

```

105 ParamType 2 Var cCliStore AS Character Default ""
106
107 /*/
108 Valor total da venda for 0,01 a regra total nao tem como ser aplicada, a venda deve ter no minimo o valor 0,01
109 /*/
110
111 nNewValueTotal := nValueTotal
112
113 If nValueTotal >= 0.01
114
115     aRules := STDTotRlDi( cCliCode , cCliStore,, lPgto )
116
117     If Len(aRules) > 0
118
119         /*/
120         Ordena o array pelo segundo campo MEJ->MEJ_PRINUM
121         /*/
122         aRules      := aSort(aRules,,{|x,y| x[MEJ_PRINUM] < y[MEJ_PRINUM]})
123         lDescGrp    := ExistFunc("STDGetGrp") .AND. STDGetGrp()
124
125         For nI := 1 To Len(aRules)
126
127             //Verifica se a regra de desconto tem tabela de preco associado
128             //Caso exista tabela de preco, precisamos analisar quais os produtos da venda esta contido nessa tabela de preco
129             If lProdDA1 .AND. lCodTab .AND. Len(aRules[nI]) >= 5 .AND. !Empty(aRules[nI][5])
130                 nValueTotal := STDProdDA1(aRules[nI][5],oCesta)
131                 nNewValueTotal := nValueTotal
132             EndIf
133
134             //Valida se o grupo de cliente é o mesmo da regra de desconto
135             If lDescGrp .AND. Len(aRules) > 0
136                 If !STBConfReg(cCliCode, cCliStore, aRules[nI][MEI_CODREG])
137                     Loop
138                 EndIf
139             EndIf
140
141             /*/
142             Busca o Range da Regra
143             /*/
144             If lDescGrp .Or. (!Empty(cForma) .And. Len(aRules) > 0 )
145                 aForms := STDPayForms(aRules[nI][MEI_CODREG], cForma)
146                 If !Empty(aForms) //Buscamos o range de acordo com a forma de pagamento informada na venda, pois antes estava pegando sempre o primeiro.
147                     For nK := 1 To Len(aForms)
148                         If AllTrim(aForms[nK]) == AllTrim(cForma)
149                             aRange := STDRangRul( aRules[nI][MEI_CODREG] )
150                         EndIf
151                     Next
152                 Else
153                     aRange := STDRangRul( aRules[nI][MEI_CODREG] ) // Quando tem a regra de desconto somente por grupo de clientes
154                 EndIf
155             EndIf
156             For nJ:= 1 To Len( aRange )

```

```

157
158     If nValueTotal >= aRange[nJ][1] .AND. nValueTotal <= aRange[nJ][2] // Valida Range
159
160     Do Case
161
162         Case aRange[nJ][6] = '1' .OR. Empty( aRange[nJ][6] )//DESCONTO - Caso o campo MB2_TPREGR nao estiver preenchido será tratado como desconto
163             /*/
164             Calculo dos valores da regra. (Acumulativo para mais de uma regra)
165             /*/
166         Do Case
167             Case aRange[nJ][nMB2_PDESC] > 0 // PERCENTUAL
168
169                 If !((nNewValueTotal - (nNewValueTotal * (aRange[nJ][nMB2_PDESC] / 100 )) < 0.01)
170                     /*/
171                     Chamada Desconto na Proxima Venda
172                     /*/
173                     If STWNxtSalD( "T" , NIL , NIL , nNewValueTotal , ;
174                         NIL , aRules[nI][MEI_CODREG] , aRange[nJ][nMB2_PDESC] , NIL )
175
176                 Else
177
178                     lApplyRule := .T.
179                     /*/
180                     Realizar calculos sempre por valor
181                     /*/
182
183                     If FWIsInCallStack("STIADDNEWPAN")
184                         //Quando clica 2x na lista de pagamentos, entra no IF
185                         If nPgts == 0
186                             //Quando não existe nenhuma forma de pagamento já lançada na venda, então considera para
187                             //calculo do desconto o total de mercadorias
188                             nNewValueTotal := STFSaleTotal("L1_VALMERC")
189                         Else
190                             //Considera para o calculo do desconto o saldo a pagar da venda, isso quando já existe uma
191                             //forma de pagamento lançada na venda
192                             nNewValueTotal := IIF(STFSaleTotal("L1_VALMERC") - STBCalcSalD("1") == 0, STFSaleTotal("L1_VALMERC"), STBCalcSalD("1")
193
194                     + STDGPBasket( "SL1", "L1_DESCONT"))
195
196                     EndIf
197                     ElseIf !FWIsInCallStack("STIADDPAY")
198                         //Ao entrar na tela de pagamento, considera o valor total para calculo do desconto
199                         nNewValueTotal := STFSaleTotal("L1_VALMERC")
200                     EndIf
201
202                     nValueDiscount := nNewValueTotal * ( aRange[nJ][nMB2_PDESC] / 100 )
203                     nTotValueDiscount := nTotValueDiscount + nValueDiscount
204                     nNewValueTotal := nNewValueTotal - nValueDiscount
205
206                     AADD( aAppRules , { aRules[nI] , aRange[nJ] } )
207
208                 EndIf
209
210             EndIf

```

```

208
209         Case aRange[nJ][nMB2_VLRFX] > 0 // VALOR
210
211             If !(aRange[nJ][nMB2_VLRFX] > nNewValueTotal) .AND. !((nNewValueTotal - aRange[nJ][nMB2_VLRFX]) < 0.01)
212                 /*/
213                 Chamada Desconto na Proxima Venda
214                 /*/
215                 If STWNxtSalD( "T"      , NIL      , NIL      , nNewValueTotal      , ;
216                             NIL      , aRules[nI][MEI_CODREG] , NIL      , aRange[nJ][nMB2_VLRFX] )
217
218             Else
219
220                 lApplyRule := .T.
221                 /*/
222                 Realizar calculos sempre por valor
223                 /*/
224                 nValueDiscount      := aRange[nJ][nMB2_VLRFX]
225                 nTotValueDiscount   := nTotValueDiscount + nValueDiscount
226                 nNewValueTotal      := nNewValueTotal - nValueDiscount
227
228                 AADD( aAppRules , { aRules[nI] , aRange[nJ] } )
229
230             EndIf
231
232         EndIf
233
234     EndCase
235
236     Case aRange[nJ][6] = '2' .AND. AliasIndic("MGB") //BONIFICACAO
237
238         cREFGRD := aRange[nJ][7]
239
240         DbSelectArea("MGB")
241         MGB->(DbSetOrder(1)) //MGB_FILIAL+MGB_CODREG+MGB_IDPROD
242         If MGB->(DbSeek(xFilial("MGB")+aRules[nI][1]+cREFGRD))
243             While MGB->(!BOF() .And. MGB->MGB_FILIAL+MGB->MGB_CODREG+MGB->MGB_IDPROD = xFilial("MGB")+aRules[nI][1]+cREFGRD)
244                 nPrice := STBSearchPrice(MGB->MGB_CODPRO)
245
246                 If !Empty(MGB->MGB_DESCPR)//Valor do produto possui desconto em Porcentagem
247                     nValor := nPrice - ((nPrice / 100) * MGB->MGB_DESCPR)
248
249                 Else
250                     If !Empty(MGB->MGB_DESCVL)//Valor do produto possui desconto em Valor
251                         nValor := (nPrice - MGB->MGB_DESCVL)
252                     Else
253                         //valor cheio do produto
254                         nValor := nPrice
255                     Endif
256                 Endif
257
258             If nValor > 0
259

```

```

260         If len(aBonificados) > 0
261             For nX := 1 to len(aBonificados)
262                 If alltrim(substr(aBonificados[nX][1],1,at('-', aBonificados[nX][1])-1)) == alltrim(MGB->MGB_CODPRO )
263                     lGrava := .F.
264                 Endif
265             Next nX
266         Endif
267
268         If lGrava
269             aAdd(aBonificados,{AllTrim(MGB->MGB_CODPRO)+" - ";
270                 +AllTrim(Posicione( "SB1",1,xFilial( "SB1" )+alltrim(MGB->MGB_CODPRO),"SB1->B1_DESC" )));
271                 MGB->MGB_DESCPR, MGB->MGB_DESCVL,nPrice})
272         Endif
273
274         nValor := 0
275     Endif
276
277     MGB->(DbSkip())
278 EndDo
279 Endif
280
281 //Executa a tela para selecionar a bonificação
282 If !Empty(aBonificados) .AND. FindFunction("STIBONIFSelection")
283     STIBONIFSelection( ,aRules[nI][1], cREFGRD, MB8->MB8_CODPROD,
284         aBonificados, .T., cForma, oPnlAdconal)//Importante: NAO usar STIExchangePanel porque ocorre erro de PANEL
285 Endif
286
287 Case aRange[nJ][6] = '3' .AND. AliasIndic("MGB") //BRINDE
288
289     cREFGRD := aRange[nJ][7]
290
291     //Consistencia para exibir os brindes por Total
292     DbSelectArea("MGB")
293     MGB->(DbSetOrder(1)) //MGB_FILIAL+MGB_CODREG+MGB_IDPROD
294     If MGB->(DbSeek(xFilial("MGB")+aRules[nI][1]+cREFGRD ))
295
296         nCont := 1
297
298         While MGB->(!BOF() .And. MGB->MGB_FILIAL+MGB->MGB_CODREG+MGB->MGB_IDPROD = xFilial("MGB")+aRules[nI][1]+ cREFGRD )
299
300             aadd( aProduto , { .T.,alltrim(oModelCesta:GetValue("SL1MASTER" , "L1_LOJA")), MGB->MGB_CODPRO ,AllTrim(Posicione( "SB1",1,xFilial(
301 "SB1" )+alltrim(MGB->MGB_CODPRO),"SB1->B1_DESC" ))), MGB->MGB_QTDPRO })
302             aAdd( aLocal, alltrim(oModelCesta:GetValue("SL1MASTER" , "L1_LOJA")))
303
304             lContinua := STBRemoteExecute("Lj7SldEst" , { aProduto , aLocal} , NIL, .F. , @aSldEst )
305
306             If lContinua
307                 If aSldEst[nCont][5][nCont][2] > 0 //Produtos disponiveis em estoque
308                     If len(aBrindes) > 0
309                         For nX := 1 to len(aBrindes)
310                             If alltrim(substr(aBrindes[nX],1,at('-', aBrindes[nX])-1)) == alltrim(MGB->MGB_CODPRO )
311                                 lGrava := .F.

```

```

311             Endif
312         Next nX
313     Endif
314
315     If lGrava
316         aAdd(aBrindes, AllTrim(MGB->MGB_CODPRO)+ " - "+
317             AllTrim(Posicione( "SB1", 1, xFilial( "SB1" )+alltrim(MGB->MGB_CODPRO), "SB1->B1_DESC" )))
318     Endif
319 Endif
320 Endif
321
322     MGB->(DbSkip())
323
324     nCont := nCont +1
325 EndDo
326 Endif
327
328 //Executa a tela para selecionar as opções de brinde
329 If !Empty(aBrindes)
330     STIPanBrindeSelection(aRules[nI][1], cREFGRD, MB8->MB8_CODPROD, aBrindes,;
331         .F., cForma, oPnlAdconal)//Importante: NAO usar STIExchangePanel porque ocorre erro de PANEL
332 Endif
333
334 EndCase
335 // SUGESTAO DE REGRA DE DESCONTO
336 ElseIf !(lApplyRule) .AND. STFGetCfg("lSugRuleDisc") //Sugestao de Desconto por regra
337
338     // Valor da venda + Valor de margem para apresentar sugestao de regra de desconto para alcançar valor inicial do range
339     If nValueTotal + STFGetCfg("nVlrRngSugRD") >= aRange[nJ][nMB2_VLRINI]
340
341         /*
342         Armazena Sugestao
343         */
344         nValSuggestion := aRange[nJ][nMB2_VLRINI] - nValueTotal // Valor que deve ser adicionado para alcançar a regra
345
346         AADD( aSuggestion , cSimbMoeda + " " + cValToChar(nValSuggestion) )
347
348     Do Case
349
350         Case aRange[nJ][nMB2_PDESC] > 0 // PERCENTUAL
351
352             AADD( aSuggestion , cValToChar( aRange[nJ][nMB2_PDESC] ) + "%" )
353
354         Case aRange[nJ][nMB2_VLRFEX] > 0 // VALOR
355
356             If !(aRange[nJ][nMB2_VLRFEX] > nNewValueTotal) .AND. !((nNewValueTotal - aRange[nJ][nMB2_VLRFEX]) < 0.01)
357                 AADD( aSuggestion , cSimbMoeda + " " + cValToChar( aRange[nJ][nMB2_VLRFEX] ) )
358             Else
359                 aSuggestion := {}
360             EndIf
361
362     EndCase

```

```

363
364         EndIf
365     EndIf
366 Next nJ
367
368     // Se acumulativo nao verifica as demais
369     If !lApplyRule .AND. aRules[nI][MEI_ACUMUL] == "N"
370         Exit
371     EndIf
372
373 Next nI
374 EndIf
375 EndIf
376
377 If !lApplyRule
378     lApplyRule := STBPromPgto(cForma) //nova verificação com desconto pela forma de pagamento
379     if !lApplyRule
380         nValueDiscount      := 0
381         nTotValueDiscount   := 0
382         nNewValueTotal      := 0
383     EndIf
384 EndIf
385
386 If lApplyRule
387
388     nDiscPerTotal := ( nTotValueDiscount * 100 ) / nValueTotal // Percentual total depois da(s) regras(s) aplicadas
389
390     If STFGetCfg("lRuleDiscShow")// Se mostra regras aplicadas
391
392         For nI:=1 To Len(aAppRules)
393             cMsg += aAppRules[nI][1][4] +"; "
394         Next nI
395         cMsg:= SUBSTR(cMsg,1, len(Alltrim(cMsg))-1)
396
397         STFMessage("STBTotRlDi","ALERT", STR0001 + cMsg) //Regra(s) de Desconto(s) Aplicado(s):
398
399     EndIf
400
401     STWTotDisc( nTotValueDiscount , "V" , "RULEDISC", ( nPgto > 0 .AND. !FWIsInCallStack("STIADDPAY")) )
402
403     // Sinaliza que teve desconto no total vindo de alguma regra cadastrada
404     ldescreg := .T.
405 Else
406
407     If STFGetCfg("lSugRuleDisc") //Sugestao de Desconto por regra
408
409         If Len(aSuggestion) > 0
410
411             STFMessage( "STBTotRlDi","ALERT", STR0003 + cValToChar(aSuggestion[1]) + ;
412                 STR0004 + STR0005 + aSuggestion[2]) // "Sugestão de Regra de Desconto: Adicionando mais: será concedido desconto de:
413         EndIf
414

```

```

415         EndIf
416
417     EndIf
418
419 EndIf
420
421 AADD( aRet , lApplyRule )
422 AADD( aRet , nNewValueTotal )
423 AADD( aRet , nTotValueDiscount )
424 AADD( aRet , nDiscPerTotal )
425 AADD( aRet , nValueTotal )
426 Iif(ExistFunc("STISetImp"),STISetImp(.F.),Nil)
427
428 STFS showMessage("STBTotRlDi")
429
430 Return aRet
431
432 //-----
433 /*/{Protheus.doc} STBPromPgto
434 Function para verificar se existe promocao por forma de pagamento
435
436 @param
437 @return
438 @obs
439 @sample
440 */
441 //-----
442 Function STBPromPgto(cForma,cTipo)
443
444 Local aRules          := {}           // Regras de Desconto no Total
445 Local aRange          := {}           // Range de valores da regra
446 Local nJ              := 0            // Contador
447 Local nI              := 0            // Contador
448 Local nK              := 0            // Contador
449 Local nValueTotal     := 0            // valor total da venda
450 Local cCliCode        := ""           // cliente da venda
451 Local cCliStore       := ""           // loja da venda
452 Local aForms          := {}           // recebe as formas de pagamento cadastrada na regra de desconto
453 Local lRet            := .F.          // variavel de retorno
454 Local lStdAdmFn        := ExistFunc("STDAdmFn")//retorna se existe a função STDAdmFn
455 Local aAdmFin         := {}           //array com registro da tabela MB6
456
457 Default cForma := ""
458 Default cTipo  := ""
459
460 If nValueTotal == 0
461     nValueTotal := STFSaleTotal()
462 EndIf
463
464 If Empty(cCliCode)
465     cCliCode := STDGPBasket( "SL1" , "L1_CLIENTE" )
466 EndIf

```

```

467
468 If Empty(cCliStore)
469     cCliStore := STDGPBasket( "SL1" , "L1_LOJA" )
470 EndIf
471
472 aRules := STDTotRlDi( cCliCode , cCliStore,,.T. )
473
474 If Len(aRules) > 0
475
476     /*/
477     Ordena o array pelo segundo campo MEJ->MEJ_PRINUM
478     /*/
479     aRules := aSort(aRules,,,{x,y| x[MEJ_PRINUM] < y[MEJ_PRINUM]})
480
481     For nI := 1 To Len(aRules)
482
483         /*/
484         Busca o Range da Regra
485         /*/
486         aRange := STDRangRul( aRules[nI][MEI_CODREG],cTipo)//Filtrar o tipo de promocao(desconto/bonificacao/brinde)
487
488         For nJ:= 1 To Len( aRange )
489
490             If nValueTotal >= aRange[nJ][1] .AND. nValueTotal <= aRange[nJ][2] // Valida Range
491                 /*/
492                 Busca as Formas de Pagamento
493                 /*/
494                 aForms := STDPayForms( aRules[nI][MEI_CODREG],cForma )
495                 IF lStdAdmFn
496                     aAdmFin:= STDAdmFn (aRules[nI][MEI_CODREG])
497                 Endif
498             Endif
499             // Caso retorne mais de uma forma de pagamento por regra de desconto
500             For nk := 1 To Len(aForms)
501                 If AllTrim(aForms[nK]) == Alltrim(cForma)
502                     nI := Len(aRules)+1 // Realizamos esse tratamento para sair do primeiro For nI
503                 EndIf
504             Next
505         Next nJ
506     Next nI
507 Endif
508
509 Endif
510
511 If !Empty(aForms)
512     For nI := 1 To Len(aForms)
513         If AllTrim( aForms[nI] ) == AllTrim( cForma ) .AND. Empty(aAdmFin)
514             lRet := .T.
515         Endif
516     Next nI
517 Endif
518

```

```

519 If !lRet .AND. !Empty(cForma) .And. ExistFunc("STDGetGrp") .AND. STDGetGrp() .And. ExistFunc("STDGetFrm") .And. !STDGetFrm()
520     lRet := .T.
521 EndIf
522
523 Return lRet
524 //-----
525 /*{Protheus.doc} STIDescRegVar
526 Verifica se teve desconto no total referente a alguma regra de desconto
527 varejo cadastrada.
528
529 @param      lRet          .T.
530 @return      ldescreg
531 @obs
532 @sample
533 */
534 //-----
535 Function STIDescRegVar(lRet)
536 DEFAULT lRet := .F.
537
538 lRet := ldescreg
539
540 Return lRet
541
542
543 //-----
544 /*{Protheus.doc} STBVldDesc
545 Funcao responsavel em validar se aplica o desconto no total ou nao.
546 O Desconto sera aplicado somente se o pagamento for feito com uma das
547 formas de pagamento configurada na regra de desconto e com valor integral.
548
549 @param      cType - Tipo de validacao
550 @return      lRet - Retorna .T. (aplica o desconto) ou .F. (nao aplica o desconto)
551 @obs
552 @sample
553 */
554 //-----
555 Function STBVldDesc(cType, oParcelas, nValor)
556
557 Local lRet      := .T. //Variavel de retorno
558 Local nTotDA1   := IIF(ExistFunc('STDGetDA1'),STDGetDA1(),0) //Verifica se houve desconto no total atraves de uma regra de desconto
559
560 Default cType   := ""
561 Default nValor  := 0
562
563 If !Empty(cType)
564
565     If cType == '1' .AND. nTotDA1 > 0 .AND. oParcelas:Length() >= 1 .AND. !Empty(oParcelas:GetValue('L4_FORMA')) .AND. STBCalcSald("1") > 0
566         lRet := .F.
567         lVerReg := .F.
568         STFMessage("STBVldDesc","STOP", STR0006) //Regra de desconto não foi aplicada pois ja constava um ou mais pagamentos efetuados para essa venda. A regra de desconto
por forma de pagamento, aceita apenas uma das formas de pagamento configurado no cadastro da regra de desconto."
569         STFShowMessage("STBVldDesc")

```

```

570     ElseIf cType == '2' .AND. lVerReg
571         If STBCalcSald('1') > nValor
572             STFMessage("STBVldDesc","STOPPOPUP", STR0007 + Chr(13) +;
573                 Chr(13) + STR0008 + AllTrim(Str(STBCalcSald('1'),12,2)) + Chr(13) + STR0009 + AllTrim(Str(nValor,12,2)) + Chr(13) + Chr(13) + STR0010) //"0 desconto
                    aplicado na venda sera desconsiderado porque o valor digitado na forma de pagamento, não é o valor integral da venda. " # "Valor integral da venda R$ " # "Valor digitado
                    R$ " # "Sera necessario realizar novamente o lançamento do pagamento!"
574             STFShowMessage("STBVldDesc")
575             STBSetCart(.F.)
576             STIClearDisc(.T.)
577         EndIf
578     ElseIf cType == '3' .AND. ((nTotDA1 > 0 .AND. oParcelas:Length() >= 1 .AND. !Empty(oParcelas:GetValue('L4_FORMA'))).OR. !SuperGetMV("MV_LJRGDES",,.F.) .OR. nTotDA1 == 0)
579         lRet := .F.
580     EndIf
581
582 EndIf
583
584 Return lRet
585
586 //-----
587 /*{Protheus.doc} STBSetReg
588 Funcao para setar valor a variavel lVerReg
589
590 @param      lReg - Recebe um valor logico
591 @return      .T.
592 @obs
593 @sample
594 */
595 //-----
596 Function STBSetReg(lReg)
597
598 Default lReg := .T.
599
600 lVerReg := lReg
601
602 Return .T.
603
604
605 //-----
606 /*{Protheus.doc} STBSetCart()
607 Funcao para setar valor a variavel lContCart
608
609 @param      lReg - Recebe um valor logico
610 @return      .T.
611 @obs
612 @sample
613 */
614 //-----
615 Function STBSetCart(lReg)
616
617 Default lReg := .T.
618
619 lContCart := lReg

```

```

620
621 Return .T.
622
623 //-----
624 /*{Protheus.doc} STBGetCart()
625 Funcao para retornar o conteudo da variavel lContCart
626
627 @param
628 @return      lContCart - Retorna o conteudo da variavel
629 @obs
630 @sample
631 */
632 //-----
633 Function STBGetCart()
634 Return lContCart
635
636
637 //-----
638 /*{Protheus.doc} STBGetPgto()
639 Função responsável por verificar se existe alguma forma de pagamento
640 já lançada na venda
641
642 @return      nRet -> Retorna a quantidade de formas de pgto lançado na venda
643 */
644 //-----
645 Function STBGetPgto()
646
647 Local oMdl      := STISetMdlPay() //Retorna o model dos pagamentos
648 Local oModel    := Nil           //Pega o modelo de Parcelas
649 Local nRet      := 0             //Variavel de retorno
650 Local nX        := 0             //Variavel de loop
651
652 If ValType(oMdl) == "0"
653     oModel := oMdl:GetModel('APAYMENTS')
654     For nX := 1 To oModel:Length()
655         oModel:GoLine(nX)
656         If !oModel:IsDeleted() .AND. !Empty(oModel:GetValue("L4_FORMA"))
657
658             nRet++
659
660             If IsMoney(oModel:GetValue("L4_FORMA"))
661                 cFormaPg := oModel:GetValue("L4_FORMA")
662             EndIf
663         EndIf
664     Next nX
665 EndIf
666
667 Return nRet
668
669 Return nRet
670
671 /*{Protheus.doc} STBSetDesc

```

```

672 Reiniciar a variavel lDescReg depois da finalização da venda
673 @type Function
674 @param lValue, lógico, reinicia a variavel lDescReg
675 @return
676 */
677 Function STBSetDesc(lValue)
678 Default lValue := .F.
679 lDescReg := lValue
680 Return
681
682 /*{Protheus.doc} STBConfReg
683 Função para confirmar se o grupo do cliente é o mesmo que esta na regra
684 @type Function
685 @param cCodCli, caractere, código do cliente
686 @param cLojCli, caractere, loja do cliente
687 @param cCodReg, caractere, código da regra de desconto
688 @return lRet, lógico, .T. se caso o cliente e regra forem do mesmo grupo de vendas, e .F. se for diferente.
689 */
690 Static Function STBConfReg(cCodCli, cLojCli, cCodReg)
691 Local lRet := .F.
692 Local aArea := GetArea()
693
694 DbSelectArea("SA1")
695 SA1->(DbSetOrder(1)) //A1_FILIAL+A1_COD+A1_LOJA
696 If SA1->(dbSeek(xFilial("SA1") + PadR(cCodCli, TamSX3("A1_COD"))[1]) + PadR(cLojCli, TamSX3("A1_LOJA"))[1]))
697
698     DbSelectArea("MEI")
699     MEI->(DbSetOrder(1)) //MEI_FILIAL+MEI_CODREG
700     If MEI->(dbSeek(xFilial("MEI") + PadR(cCodReg, TamSX3("MEI_CODREG"))[1]))
701
702         If AllTrim(SA1->A1_GRPVEN) == AllTrim(MEI->MEI_GRPVEN)
703             lRet := .T.
704         EndIf
705     EndIf
706 EndIf
707 EndIf
708
709 RestArea(aArea)
710
711 Return lRet

```

Componentes/Discount/STDItemDiscount.prw

95 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 //-----
5 /*/{Protheus.doc} STDIDTime
6 Busca configuração de momento de desconto por usuário, se antes ou depois de registrar o item
7
8 @param
9 @return cTime          Retorna configuração de momento de efetuar desconto no Item via usuário
10 @obs
11 @sample
12 */
13 //-----
14 Function STDIDTime()
15
16 Local cTime      := "A"          // Retorna configuração de momento de efetuar desconto no Item via usuário
17 Local aDados     := {"15", space(20)} // Pega Estatus 15 do ECF para saber se permite desconto apos registrar o item
18 Local aRet       := {}
19
20 /*/
21 "A" - Antes (Padrao)
22 "D" - Depois
23 /*/
24
25
26 /*/
27 Se utiliza impressora fiscal, utiliza
28 Verifica se o ECF permite desconto apos registrar o item
29 /*/
30 If STFUseFiscalPrinter()
31
32     aRet := STFFireEvent( ProcName(0)          ,; // Nome do processo
33                          "STPrinterStatus"    ,; // Nome do evento
34                          @aDados              )
35
36     // 0-Arredonda 1-Trunca
37     If !Empty(aRet) .AND. aRet[1] == 1
38         cTime := "A" //Antes
39     Else
40         cTime := "D" //Depois
41     EndIf
42
43 EndIf
44
45 Return cTime
46
47
48 //-----
49 /*/{Protheus.doc} STDIDUseDiscountFrom
50 Efetua Desconto no Item
51
52 @param

```

```

53 @return cConfig      Retorna configuração de desconto no item quanto a aplicar descontos via usuário, via regra ou ambos
54 @obs
55 @sample
56 /*/
57 //-----
58 Function STDIDUseDiscountFrom()
59
60 Local cConfig      := ""      // Retorna configuração de desconto no item quanto a aplicar descontos via usuário, via regra ou ambos
61
62 /*/
63     Fazer Facilitador para configuração se usa desconto via usuário, via regra ou ambos
64     "U" - Usuário
65     "R" - Regra de Desconto
66     "A" - Ambos
67 /*/
68 cConfig := STFGetCfg("cCtrlDesc")
69
70 // Tratamento para corrigir informação do parâmetro caso venha preenchido com ""
71 // Necessário para não alterar boletim técnico
72 If Len(cConfig) > 1
73     cConfig := SubStr(cConfig,2,1)
74 EndIf
75
76 Return cConfig
77
78
79 //-----
80 /*/{Protheus.doc} STDIDReasonDiscount
81 Busca configuração se Registra Motivo de Desconto quando uma Regra de Desconto é aplicada
82
83 @param
84 @return lRet      Retorna configuração se Registra Motivo quando uma Regra de Desconto é aplicada
85 @obs
86 @sample
87 /*/
88 //-----
89 Function STDIDReasonDiscount()
90
91 Local lRet      := .F.      // Retorna configuração se Registra Motivo quando uma Regra de Desconto é aplicada
92
93 lRet := .T.
94
95 Return lRet

```

Componentes/Discount/STDNextSaleDiscount.PRW

282 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STDNEXTSALEDISCOUNT.CH"
4
5 Static oModel      := Nil          // Model
6 Static oStructNSD  := Nil
7
8 //-----
9 /*{Protheus.doc} STDRFDGetModel
10 Function Cria estrutura de dados do Model
11
12 @param
13 @return oModel          Retorna estrutura de dados do Model
14 @obs
15 @sample
16 */
17 //-----
18 Static Function STDNSDGetModel()
19
20 Local oMasterStr      := Nil          // Estrutura Master
21
22 If oModel == Nil
23
24     oMasterStr := CreateMasterStruct()
25
26     oGridStr := CreateGridStruct()
27
28     oModel := MPFormModel():New( 'NSD' )
29
30     oModel:AddFields( "NSD_MASTER" , , oMasterStr )
31
32     oModel:AddGrid( "NSD_GRID"      , "NSD_MASTER"      , oGridStr )
33
34     oModel:SetRelation( "NSD_GRID" , { { "ITEM" } } , { "ITEM" } } , "ITEM" )
35
36     oModel:SetPrimaryKey( { "ITEM" } )
37
38     oModel:GetModel( "NSD_GRID" ):SetOptional( .T. )
39
40     oModel:SetOperation( 3 )
41
42     oModel:Activate()
43
44 EndIf
45
46 Return oModel
47
48
49 //-----
50 /*{Protheus.doc} CreateMasterStruct
51 Function Cria estrutura do Model Master
52

```

```

53 @param
54 @return oStruct      Retorna estrutura do Model Master
55 @obs
56 @sample
57 */
58 //-----
59 Static Function CreateMasterStruct()
60
61 If ValType(oStructNSD) = '0'
62     oStructNSD:Deactivate()
63     oStructNSD:Activate()
64 Else
65     oStructNSD := FWFormModelStruct():New() // Estrutura
66
67     oStructNSD:AddTable(
68         "NSD"                , ; // [01] Alias da tabela
69         {"ITEMLINE"}        , ; // [02] Array com os campos que correspondem a primary key
70         "NEXT SALE DISCOUNT" ) // [03] Descrição da tabela
71
72     oStructNSD:AddField(
73         "ITEM"                , ; // [01] Titulo do campo
74         "ITEM"                , ; // [02] Desc do campo
75         "ITEM"                , ; // [03] Id do Field
76         "N"                   , ; // [04] Tipo do campo
77         TamSX3("L2_ITEM")[1] , ; // [05] Tamanho do campo
78         0                      , ; // [06] Decimal do campo
79         Nil                    , ; // [07] Code-block de validacao do campo
80         Nil                    , ; // [08] Code-block de validacao When do campo
81         Nil                    , ; // [09] Lista de valores permitido do campo
82         Nil                    , ; // [10] Indica se o campo tem preenchimento obrigatorio
83         Nil                    , ; // [11] Code-block de inicializacao do campo
84         NIL                    , ; // [12] Indica se trata-se de um campo chave
85         NIL                    , ; // [13] Indica se o campo pode receber valor em uma operacao de update.
86         ) // [14] Indica se o campo e virtual
87 EndIf
88
89 Return oStructNSD
90
91 //-----
92 /*/{Protheus.doc} CreateGridStruct
93 Function Cria estrutura do Model Grid
94
95 @param
96 @return oStruct      Retorna estrutura do Model Grid
97 @obs
98 @sample
99 */
100 //-----
101 Static Function CreateGridStruct()
102
103 Local aArea      := GetArea() // Armazena alias corrente
104 Local oStruct    := FWFormModelStruct():New() // Estrutura

```

```

105 Local aFields := {} // Campos
106 Local nI := 0 // Contador
107
108 oStruct.AddTable( ;
109 "NSD" , ; // [01] Alias da tabela
110 {"ITEMLINE"} , ; // [02] Array com os campos que correspondem a primary key
111 "NEXT SALE DISCOUNT" ) // [03] Descrição da tabela
112
113 AADD( aFields , { "ITEMLINE" , "ITEMLINE" , "ITEMLINE" , "N" , TamSX3("L2_ITEM")[1] , 0 , , , , } )
114 AADD( aFields , { "TYPE" , "TYPE" , "TYPE" , "C" , 1 , , , , } )
115 AADD( aFields , { "ITEMCODE" , "ITEMCODE" , "ITEMCODE" , "C" , TamSX3("L2_PRODUTO")[1] , 0 , , , , } )
116 AADD( aFields , { "VALUE" , "VALUE" , "VALUE" , "N" , TamSX3("L2_VLRITEM")[1] , TamSX3("L2_VLRITEM")[2] , , , , } )
117 AADD( aFields , { "DISCOUNT" , "DISCOUNT" , "DISCOUNT" , "N" , TamSX3("L2_VALDESC")[1] , TamSX3("L2_VALDESC")[2] , , , , } )
118 AADD( aFields , { "DATE" , "DATE" , "DATE" , "D" , 8 , , , , } )
119
120 For nI := 1 To Len(aFields)
121
122 oStruct.AddField( ;
123 aFields[nI][1] , ; // [01] Titulo do campo
124 aFields[nI][2] , ; // [02] ToolTip do campo
125 aFields[nI][3] , ; // [03] Id do Field
126 aFields[nI][4] , ; // [04] Tipo do campo
127 aFields[nI][5] , ; // [05] Tamanho do campo
128 aFields[nI][6] , ; // [06] Decimal do campo
129 Nil , ; // [07] Code-block de validacao do campo
130 Nil , ; // [08] Code-block de validacao When do campo
131 Nil , ; // [09] Lista de valores permitido do campo
132 Nil , ; // [10] Indica se o campo tem preenchimento obrigatorio
133 Nil , ; // [11] Code-block de inicializacao do campo
134 NIL , ; // [12] Indica se trata-se de um campo chave
135 NIL , ; // [13] Indica se o campo pode receber valor em uma operacao de update.
136 NIL ) // [14] Indica se o campo e virtual
137
138 SX3->(DbSkip())
139
140 Next nI
141
142 RestArea(aArea)
143
144 Return oStruct
145
146 -----
147 /*{Protheus.doc} STDNxTSalAdd
148 Function Adiciona Informações do Desconto na Próxima Venda no model
149
150 @param cType Tipo. I - Item , T - Total
151 @param nItemLine Linha do Item
152 @param cItemCode Código do Produto
153 @param nValue Valor
154 @param nDiscount Valor do Desconto
155 @param dEndDate Data de Validade
156 @return lRet Retorna se adicionou no Grid do Model

```

```

157 @obs
158 @sample
159 /*/
160 //-----
161 Function STDNxtSalAdd( cType      , nItemLine , cItemCode      , nValue      , ;
162                      nDiscount , dEndDate )
163
164 Local oModel      := STDNSDGetModel() // Modelo do Motivo de Desconto (NSD)
165 Local oModelGrid  := oModel:GetModel("NSD_GRID") // Componente Grid do Motivo de Desconto
166 Local nLine       := 0 // Armazena linha do Model
167 Local lFirstItem  := .F. // Indica se primeiro item
168 Local lRet        := .F. // Retorna se adicionou Desconto na Proxima Venda
169
170 Default cType      := ""
171 Default nItemLine  := 0
172 Default cItemCode  := ""
173 Default nValue     := 0
174 Default nDiscount  := 0
175 Default dEndDate   := CToD("")
176
177 ParamType 0 Var cType      AS Character Default ""
178 ParamType 1 Var nItemLine  AS Numeric   Default 0
179 ParamType 2 Var cItemCode  AS Character Default ""
180 ParamType 3 Var nValue     AS Numeric   Default 0
181 ParamType 4 Var nDiscount  AS Numeric   Default 0
182 ParamType 5 Var dEndDate   AS Date      Default cToD("")
183
184 lFirstItem := ( oModelGrid:Length() == 1 ) .AND. Empty(oModelGrid:GetValue("DISCOUNT"))
185
186 If lFirstItem
187     lRet := .T.
188 Else
189     nLine := ( oModelGrid:Length() )
190     lRet := oModelGrid:AddLine() == ( nLine + 1 )
191 EndIf
192
193 If lRet
194
195     oModelGrid:SetValue( "ITEMLINE" , nItemLine )
196     oModelGrid:SetValue( "TYPE"     , cType )
197     oModelGrid:SetValue( "ITEMCODE" , cItemCode )
198     oModelGrid:SetValue( "VALUE"    , nValue )
199     oModelGrid:SetValue( "DISCOUNT" , nDiscount )
200     oModelGrid:SetValue( "DATE"     , dEndDate )
201
202 EndIf
203
204 Return lRet
205
206
207 //-----
208 /*/{Protheus.doc} STDRecNextSaleDiscount

```

```

209 Gera NCC dos descontos na próxima venda
210
211 @param
212 @return
213 @obs
214 @sample
215 /*/
216 //-----
217 Function STDRecNextSaleDiscount()
218
219 Local   oModelGrid      := NIL           // Componente Grid
220 Local   aArea            := GetArea()     // Armazena alias corrente
221 Local   nI               := 0            // Contador
222 Local   nTotalNCC        := 0            // Valor total da NCC a ser gerada
223 Local   lExecNow         := .F.          // Define se executou agora ou foi pra contingência
224
225 If oModel <> NIL
226
227     oModelGrid := oModel:GetModel("NSD_GRID")
228
229     For nI := 1 To oModelGrid:Length()
230
231         oModelGrid:GoLine(nI)
232
233         nTotalNCC += oModelGrid:GetValue("DISCOUNT")
234
235     Next nI
236
237     STBRemoteExecute( "LJGerTitNV" , ;
238     {
239         STDGPBasket("SL1","L1_DOC") , ;
240         STDGPBasket("SL1","L1_SERIE") , ;
241         STDGPBasket("SL1","L1_OPERADO") , ;
242         oModelGrid:GetValue("DATE") , ;
243         nTotalNCC , ;
244         STDGPBasket("SL1","L1_CLIENTE") , ;
245         STDGPBasket("SL1","L1_LOJA") , ;
246         STFGetStat("LG_SERIE") , ;
247         dDataBase , ;
248     } , ;
249     NIL , ;
250     .T. , ;
251     @lExecNow , ;
252     "TRYEXEC" )
253
254 If !lExecNow
255     STFMessage("STNextSaleDiscount","ALERT", STR0001) //"Sem conexão. A NCC será gerada quando a conexão for reestabelecida"
256     STFShowMessage("STNextSaleDiscount")
257 EndIf
258
259 EndIf
260

```

```
261 RestArea(aArea)
262
263 Return Nil
264
265
266 //-----
267 /*{Protheus.doc} STDNSDFree
268 Limpa variáveis
269
270 @param
271 @return Nil
272 @obs
273 @sample
274 */
275 //-----
276 Function STDNSDFree()
277
278 If oModel <> Nil
279     FreeObj(oModel)
280 EndIf
281
282 Return Nil
```

Componentes/Discount/STDReasonForDiscount.PRW

548 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 Static      oModel  := Nil          // Model
5
6 //-----
7 /*{Protheus.doc} STDRsnDesc
8 Function Motivo de Desconto
9
10 @param
11 @return lRet          Retorna se existe cadastro de Motivo de Desconto
12 @obs
13 @sample
14 */
15 //-----
16 Function STDRsnDesc()
17
18 Local  lRet      := .F.          // Retorna se existe cadastro de Motivo de Desconto
19 Local  aArea     := GetArea()
20
21 DbSelectArea("MDT")
22 DbSetOrder(1) // MDT_FILIAL + MDT_CODIGO
23 DbSeek( xFilial("MDT") , .T. ) // Busca aproximada
24
25 lRet := !EOF() .And. xFilial("MDT") == MDT->MDT_FILIAL
26
27 RestArea(aArea)
28
29 Return lRet
30
31
32 //-----
33 /*{Protheus.doc} STDRFDGet
34 Busca Motivos de Desconto Cadastrados
35
36 @param
37 @return aReasons      Retorna motivos de desconto cadastrados
38 @obs
39 @sample
40 */
41 //-----
42 Function STDRFDGet(cTpMov)
43
44 Local aArea      := GetArea()          // Armazena alias corrente
45 Local aReasons   := {}                // Retorno funcao
46 Local lMDT_TPMOT := (MDT->( FieldPos("MDT_TPMOT") ) > 0) // Verifica se o campo que controla o filtro pelo Tipo de Motivo existe
47
48 Default cTpMov   := "3"                // Analisa o campo MDT_TPMOV, Sendo 1=Por item, 2=Por total e 3=Ambos (Valor default = 3)
49
50 DbSelectArea("MDT")
51 DbSetOrder(1) // MDT_FILIAL + MDT_CODIGO
52 MDT->(DbGoTop())

```

```

53 If MDT->(DbSeek( xFilial("MDT") )) // Busca aproximada
54   While !EOF() .AND. xFilial("MDT") == MDT->MDT_FILIAL
55     If !lMDT_TPMOT .Or. (MDT->MDT_TPMOT == cTpMov .Or. MDT->MDT_TPMOT == "3")
56       AADD( aReasons , { MDT->MDT_CODIGO , MDT->MDT_DESCRI } )
57     EndIf
58     MDT->(DbSkip())
59   EndDo
60 EndIf
61
62 RestArea(aArea)
63
64 Return aReasons
65
66 //-----
67 /*{Protheus.doc} STDRFDGet
68 Verifica se existe Motivos de Desconto Cadastrado para o código passado
69
70 @param cMotCode - Código do Motivo
71 @return lRet      Retorna se encontrou
72 @obs
73 @sample
74 */
75 //-----
76 Function STDRFDFromRD(cMotCode)
77
78 Local lRet      := .F.           // Retorno funcao
79 Local aArea     := GetArea()     // Armazena area corrente
80
81 Default cMotCode := ""
82
83 ParamType 0 Var cMotCode AS Character Default ""
84
85 DbSelectArea("MDT")
86 DbSetOrder(1) // MDT_FILIAL + MDT_CODIGO
87 If MDT->(DbSeek( xFilial("MDT") + cMotCode ))
88   lRet := .T.
89 EndIf
90
91 RestArea(aArea)
92
93 Return lRet
94
95
96 //-----
97 /*{Protheus.doc} STDRFDGetModel
98 Cria estrutura de dados do Model
99
100 @param
101 @return oModel - Retorna estrutura de dados do Model
102 @obs
103 @sample
104 */

```

```

105 //-----
106 Static Function STDRFDGetModel()
107
108 Local oMasterStr      := Nil      // Estrutura Master
109
110 If oModel == Nil
111     /*/
112     Monta estrutura de tabela
113     /*/
114     oMasterStr := CreateMasterStruct()
115     /*/
116     Monta estrutura de grid da tabela de Motivo de Desconto (MDU)
117     /*/
118     oGridStr := CreateGridStruct()
119     /*/
120     Instacia Objeto
121     /*/
122     oModel := MPFormModel():New( 'MDU', /*bPreValidacao*/, /*bPosValidacao*/, /*bcommit*/, /*bCancel*/ )
123
124     oModel:AddFields( "MDU_MASTER" , /*cOwner*/ , oMasterStr )
125
126     oModel:AddGrid( "MDU_GRID"      , "MDU_MASTER"      , oGridStr      , /*b*/      , ;
127                    /*bLinePost*/    , /*bPreVal*/        , /*bPosVal*/    , /*bLoad*/      )
128
129     oModel:SetRelation( "MDU_GRID" , { { "MDU_FILIAL", 'xFilial( "MDU" )' } , { "MDU_CODIGO", "MDU_CODIGO" } } , 'MDU_FILIAL + MDU_CODIGO' )
130
131     oModel:SetPrimaryKey( { "MDU_FILIAL" , "MDU_CODIGO" , "MDU_ITEM" } )
132
133     oModel:GetModel( "MDU_GRID" ):SetOptional( .T. )
134
135     oModel:SetOperatin( 3 )
136
137     oModel:Activate()
138
139 EndIf
140
141 Return oModel
142
143
144 //-----
145 /*/{Protheus.doc} CreateMasterStruct
146 Cria estrutura do Model Master
147
148 @param
149 @return oStruct - Retorna estrutura do Model Master
150 @obs
151 @sample
152 /*/
153 //-----
154 Static Function CreateMasterStruct()
155
156 Local oStruct      := FWFormModelStruct():New()      // Estrutura

```

```

157 Local cX2Unico      := "" //X2_UNICO da tabela MDU
158
159 If ExistFunc('FWX2Unico')
160     cX2Unico := FWX2Unico("MDU")
161 EndIf
162
163 oStruct:AddTable(
164     FWX2CHAVE()
165     StrTokArr( cX2Unico, '+' )
166     FWX2Nome("MDU")
167
168     oStruct:AddField(
169         "FILIAL"
170         "FILIAL"
171         "MDU_FILIAL"
172         "C"
173         FWSizeFilial()
174         0
175         Nil
176         Nil
177         Nil
178         Nil
179         Nil
180         NIL
181         NIL
182     )
183
184     oStruct:AddField(
185         "CODIGO"
186         "CODIGO"
187         "MDU_CODIGO"
188         "C"
189         TamSX3("B1_COD")[1]
190         0
191         Nil
192         Nil
193         Nil
194         Nil
195         Nil
196         NIL
197         NIL
198     )
199
200 Return oStruct
201
202
203 //-----
204 /*/{Protheus.doc} CreateGridStruct
205 Cria estrutura do Model Grid
206
207 @param
208 @return oStruct - retorna estrutura do Model Grid

```

```

209 @obs
210 @sample
211 /*/
212 //-----
213 Static Function CreateGridStruct()
214
215 Local oStruct := FWFormModelStruct():New() // Estrutura
216 Local cX2Unico := "" //X2_UNICO da tabela MDU
217
218 If ExistFunc('FWX2Unico')
219     cX2Unico := FWX2Unico("MDU")
220 EndIf
221
222 oStruct:AddTable(
223     FWX2CHAVE(), ; // [01] Alias da tabela
224     StrTokArr( cX2Unico, '+' ) , ; // [02] Array com os campos que correspondem a primary key
225     FWX2Nome("MDU") ) // [03] Descrição da tabela
226
227 //Carrega informacoes de campos
228 DbSelectArea("SX3")
229 SX3->(DbSetOrder(1))
230 SX3->(DbSeek("MDU"))
231
232 While !SX3->(EOF()) .AND. SX3->X3_ARQUIVO == "MDU"
233
234     oStruct:AddField(
235         AllTrim( X3Titulo() ) , ; // [01] Titulo do campo
236         AllTrim( X3Descric() ) , ; // [02] ToolTip do campo
237         AllTrim( SX3->X3_CAMPO ) , ; // [03] Id do Field
238         SX3->X3_TIPO , ; // [04] Tipo do campo
239         SX3->X3_TAMANHO , ; // [05] Tamanho do campo
240         SX3->X3_DECIMAL , ; // [06] Decimal do campo
241         Nil , ; // [07] Code-block de validacao do campo
242         Nil , ; // [08] Code-block de validacao When do campo
243         StrTokArr( AllTrim( X3CBox() ), ';' ) , ; // [09] Lista de valores permitido do campo
244         Nil , ; // [10] Indica se o campo tem preenchimento obrigatorio
245         Nil , ; // [11] Code-block de inicializacao do campo
246         NIL , ; // [12] Indica se trata-se de um campo chave
247         NIL , ; // [13] Indica se o campo pode receber valor em uma operacao de update.
248         ( SX3->X3_CONTEXT == 'V' ) ) // [14] Indica se o campo e virtual
249
250     SX3->(DbSkip())
251
252 EndDo
253
254 // Campo de Item necessario para utilizacao do modelo (campo chave do grid)
255
256 oStruct:AddField(
257     "ITEM" , ; // [01] Titulo do campo
258     "ITEM" , ; // [02] Desc do campo
259     "MDU_ITEM" , ; // [03] Id do Field
260     "N" , ; // [04] Tipo do campo

```

```

261      TamSX3("L2_ITEM")[1] , ; // [05] Tamanho do campo
262      0 , ; // [06] Decimal do campo
263      Nil , ; // [07] Code-block de validacao do campo
264      Nil , ; // [08] Code-block de validacao When do campo
265      Nil , ; // [09] Lista de valores permitido do campo
266      Nil , ; // [10] Indica se o campo tem preenchimento obrigatorio
267      Nil , ; // [11] Code-block de inicializacao do campo
268      NIL , ; // [12] Indica se trata-se de um campo chave
269      NIL , ; // [13] Indica se o campo pode receber valor em uma operacao de update.
270      ) // [14] Indica se o campo e virtual
271
272 Return oStruct
273
274
275 //-----
276 /*{Protheus.doc} STDRFDSFree
277 Limpa variáveis
278
279 @param
280 @return Nil
281 @obs
282 @sample
283 */
284 //-----
285 Function STDRFDSFree()
286
287 If oModel <> Nil
288     FreeObj(oModel)
289 EndIf
290
291 Return Nil
292
293
294 //-----
295 /*{Protheus.doc} STDRecReason
296 Function Motivo de Desconto
297
298 @param nItemLine      Linha do Item na Venda
299 @param lDeleleAll      Se Deleta todos os Itens do Grid MDU
300 @return lRet           Retorna se deletou com sucesso
301 @obs
302 @sample
303 */
304 //-----
305 Function STDRecReason()
306
307 Local oModelGrid      := NIL           // Componente Grid MDU
308 Local aArea           := GetArea()     // Armazena alias corrente
309 Local nI              := 0             // Contador
310 Local nJ              := 0             // Contador
311 Local aStruct         := {}           // Armazena estrutura de dados do modelo
312

```

```

313 If oModel <> NIL
314
315     oModelGrid := oModel:GetModel("MDU_GRID")
316     aStruct     := oModelGrid:GetStruct():aFields
317
318     DbSelectArea("MDU")
319     DbSetOrder(1)      // MDU_FILIAL+MDU_CODIGO
320     For nI := 1 To oModelGrid:Length()
321         oModelGrid:GoLine(nI)
322
323         If !( oModelGrid:IsDeleted() )
324             RecLock("MDU",.T.)
325             For nJ := 1 To Len(aStruct)
326                 If aStruct[nJ][3] <> "MDU_ITEM"      // Esse campo nao existe no banco
327                     REPLACE MDU->&(aStruct[nJ][3]) WITH oModelGrid:GetValue(aStruct[nJ][3])
328                 EndIf
329             Next nJ
330             MsUnlock()
331
332             STFSLICreate( Nil      , "UP"          , "UP"      , "NOVO"    , ;
333                         Nil      , Nil           , Nil      , Nil      , ;
334                         "MDU"    , MDU->(RecNo()) , Nil      , Nil      , )
335         EndIf
336     Next nI
337
338 EndIf
339
340 RestArea(aArea)
341
342 Return Nil
343
344
345 //-----
346 /*{Protheus.doc} STDDelReason
347 Function Motivo de Desconto
348
349 @param  nItemLine      Linha do Item na Venda
350 @param  lDeleleAll     Se Deleta todos os Itens do Grid MDU
351 @return lRet           Retorna se deletou com sucesso
352 @obs
353 @sample
354 */
355 //-----
356 Function STDDelReason( nItemLine , lDeleleAll )
357
358 Local  oModelGrid      := NIL           // Componente Grid MDU
359 Local  aBusca          := {}           // Array para busca do Item na MDU
360 Local  lRet            := .T.          // Retorna se deletou
361 Local  nI              := 0           // Contador
362
363 Default nItemLine      := 0           //Codigo item
364 Default lDeleleAll     := .F.         // Indica se deleta todos os Itens

```

```

365
366 ParamType 0 Var nItemLine AS Numeric      Default 0
367 ParamType 1 Var lDeleleAll AS Logical      Default .F.
368
369 // Caso nao tenha sido instanciado modelo Motivo de Desconto, Nao fazer nada
370 If oModel <> NIL
371
372     oModelGrid := oModel:GetModel("MDU_GRID")
373
374     If lDeleleAll
375
376         For nI := 1 To oModelGrid:Length()
377
378             oModelGrid:GoLine( nI )
379
380             If !( oModelGrid:IsDeleted() )
381
382                 oModelGrid:DeleteLine()
383
384             EndIf
385
386         Next nI
387
388     Else
389
390         If nItemLine > 0
391
392             AAdd( aBusca, {"MDU_ITEM", nItemLine } )
393
394             If oModelGrid:SeekLine( aBusca )
395
396                 If !( oModelGrid:IsDeleted() )
397
398                     lRet := oModelGrid:DeleteLine()
399
400                 EndIf
401
402             EndIf
403
404         EndIf
405
406     EndIf
407
408 EndIf
409
410 Return lRet
411
412
413 //-----
414 /*{Protheus.doc} STDAddReason
415 Function Motivo de Desconto
416

```

```

417 @param nItemLine      Linha do Item na Venda (Passar 99999 para Desconto no Total)
418 @param cMotCode       Código do Motivo de Desconto
419 @param cObs           String obs (descrição do MDT + '-' + observação digitada)
420 @param cType          Desconto no ITEM ou TOTAL
421 @param cItemCode      Código do Item
422 @return lRet          Retorna se adicionou no Grid
423 @obs
424 @sample
425 /*/
426 //-----
427 Function STDAddReason( nItemLine , cMotCode , cObs , cItemCode , ;
428                      cType      , cCodSup )
429
430 Local oModel      := STDRFDGetModel() // Modelo do Motivo de Desconto (MDU)
431 Local oModelGrid  := oModel:GetModel("MDU_GRID") // Componente Grid do Motivo de Desconto
432 Local aBusca      := {} // array para busca do Item na MDU
433 Local lFirstItem  := .F. // Indica se primeiro item
434 Local lRet        := .F. // Retorna se adicionou motivo de desconto
435 Local nLine       := 0 // Quantidade de linhas do grid
436
437 Default nItemLine := 0 // Linha do Item na Venda
438 Default cMotCode  := "" // Código do Item
439 Default cObs      := "" // Código do Cliente
440 Default cItemCode := "" // Código da loja do Cliente
441 Default cType     := "" // Desconto no ITEM ou TOTAL
442 Default cCodSup   := "" // Código do Supervisor que liberou o desconto
443
444 AAdd( aBusca , {"MDU_ITEM", nItemLine } )
445
446 If oModelGrid:SeekLine( aBusca )
447     STDDelReason( nItemLine )
448 EndIf
449
450 lFirstItem := ( oModelGrid:Length() == 1 ) .AND. Empty(oModelGrid:GetValue("MDU_CODIGO"))
451
452 If lFirstItem
453     lRet := .T.
454 Else
455     nLine := ( oModelGrid:Length() ) + 1
456     lRet := oModelGrid:AddLine() == nLine
457 EndIf
458
459 If lRet
460
461     oModelGrid:SetValue("MDU_FILIAL" , xFilial("MDU") )
462     oModelGrid:SetValue("MDU_CODIGO" , cMotCode )
463     oModelGrid:SetValue("MDU_OBS" , cObs )
464     oModelGrid:SetValue("MDU_DATA" , dDataBase )
465     oModelGrid:SetValue("MDU_TIPO" , cType )
466     oModelGrid:SetValue("MDU_PRODUT" , cItemCode )
467     oModelGrid:SetValue("MDU_SITUA" , "00" )
468     oModelGrid:SetValue("MDU_OPERAD" , xNumCaixa() )

```

```

469 oModelGrid:SetValue("MDU_ITEM"      , nItemLine      ) // OBS: Campo não existe fisicamente
470 oModelGrid:SetValue("MDU_DOC"        , STDGPBasket("SL1" , "L1_DOC"      ) )
471 oModelGrid:SetValue("MDU_SERIE"      , STDGPBasket("SL1" , "L1_SERIE"    ) )
472
473 If oModelGrid:HasField("MDU_CODSUP")
474     oModelGrid:SetValue("MDU_CODSUP" , cCodSup      )
475 EndIf
476
477 EndIf
478
479 Return lRet
480
481 //-----
482 /*{Protheus.doc} STDAddSupD
483 Function Motivo de Desconto - Adiciona o codigo do supervisor que liberou o desconto
484
485 @param nItemLine      Linha do      Item na Venda (Passar 99999 para Desconto no Total)
486 @param cMotCode       Codigo do Motivo de Desconto
487 @param cObs           String obs (descrição do MDT + '-' + observação digitada)
488 @param cType          Desconto no ITEM ou TOTAL
489 @param cItemCode      Codigo do Item
490 @return lRet          Retorna se adicionou no Grid
491 @obs
492 @sample
493 */
494 //-----
495 Function STDAddSupD(nItemLine, cItemCode, cCodSup )
496
497 Local oModel      := STDRFDGetModel() // Modelo do Motivo de Desconto (MDU)
498 Local oModelGrid  := oModel:GetModel("MDU_GRID") // Componente Grid do Motivo de Desconto
499 Local aBusca      := {} // array para busca do Item na MDU
500
501 AAdd( aBusca , {"MDU_ITEM", nItemLine } )
502
503 If oModelGrid:SeekLine( aBusca ) .AND. oModelGrid:HasField("MDU_CODSUP")
504     oModelGrid:SetValue("MDU_CODSUP", cCodSup)
505 EndIf
506
507 Return
508
509 //-----
510 /*{Protheus.doc} STDUdpReason
511 Função responsável por atualizar alguns campos do model MDU_GRID
512
513 @param cDoc, Caracter, Documento que será atualizado
514 @param cSerie, Caracter, Serie que será atualizada
515 @return Nulo, Nil
516 @obs
517 @sample
518 */
519 //-----
520 Function STDUdpReason(cDoc,cSerie)

```

```
521
522 Local   oModelGrid      := NIL           // Componente Grid MDU
523 Local   nI               := 0             // Contador
524
525 Default cDoc             := ""
526 Default cSerie           := ""
527
528 // Caso nao tenha sido instanciado modelo Motivo de Desconto, Nao fazer nada
529 If oModel <> NIL
530
531     oModelGrid := oModel:GetModel("MDU_GRID")
532
533     For nI := 1 To oModelGrid:Length()
534
535         oModelGrid:GoLine( nI )
536
537         If !( oModelGrid:IsDeleted() ) .AND. !Empty(cDoc) .And. !Empty(cSerie)
538
539             oModelGrid:SetValue("MDU_DOC" , cDoc )
540             oModelGrid:SetValue("MDU_SERIE" , cSerie )
541
542         EndIf
543
544     Next nI
545
546 EndIf
547
548 Return
```

Componentes/Discount/STDRuleBrinde.PRW

79 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "FWMVCDEF.CH"
3
4 //-----
5 /*{Protheus.doc} STDDelBrindeCesta
6 Function Exclui item da Cesta de Brinde
7
8 @return lRet          Retorna se deletou com sucesso
9 @obs
10 @sample
11 */
12 //-----
13 Function STDDelBrindeCesta(lCheca )
14
15 Local aArea      := GetArea()          // Posicionamento atual da tabela
16 Local nX         := 0                  // Contador
17 Local lRet       := .F.                // Retorna se deletou
18 Local nQtdItens  := STDPBLength("MGC") // Quantidade total de registros na cesta MGC
19 Local oModel     := STDGPBModel()      // Objeto Model
20
21 Default lCheca := .F. //Checa se o brinde é por forma de pagto, que no caso só pode existir 1 brinde por Forma de Pagto e no caso
22                      // da Forma de Pgto ser limpa e ja exista um brinde incluído é ncessario excluí-lo da cesto
23
24 //Cancelar o item da cesta MGC
25 For nX := 1 to nQtdItens
26
27     If !lCheca
28         If alltrim(oModel:GetModel("MGCBRINDE"):acols[nX][12]) <> "" //Posicao da Forma de Pagamento
29             oModel:GetModel("MGCBRINDE"):DeleteLine()
30             lRet := .T.
31         Endif
32     Endif
33
34 Next nX
35
36 RestArea(aArea)
37 Return lRet
38
39
40 //-----
41 /*{Protheus.doc} STDSalvaBrindes
42 Function Salva brindes da Cesta de Brinde
43
44 @return lRet          Retorna se deletou com sucesso
45 @obs
46 @sample
47 */
48 //-----
49 Function STDSalvaBrindes()
50 Local aArea      := GetArea()          // Posicionamento atual da tabela
51 Local oModel     := STDGPBModel()      // Objeto Model
52 Local aMGCFields := oModel:GetModel("MGCBRINDE"):GetStruct():GetFields() // Array Campos MGC

```

```

53 Local nQtdItens      := STDPBLength("MGC")                // quantidade total de registros na cesta MGC
54 Local nX              := 0                                // Contador X
55 Local nY              := 0                                // Contador Y
56
57 //Gravacao do Brinde
58 For nX := 1 to  nQtdItens //de onde esta vindo essa quantidade de Itens
59
60     If oModel:GetModel("MGCBRINDE"):acoIs[nX][13] == .F. // Somente irá gravar na MGC se o item não estiver deletado
61
62         DbSelectArea("MGC")
63         oModel:GetModel("MGCBRINDE"):GoLine(nX)
64         MGC->(DbSetOrder())
65
66         RecLock("MGC",.T.)
67
68         For nY := 1 To Len(aMGCFields)
69             cField := aMGCFields[nY,MODEL_FIELD_IDFIELD]
70             REPLACE &(cField) WITH STDGPBasket("MGC" , cField)
71         Next nY
72
73         MGC->(MsUnlock())
74
75     Endif
76 Next nX
77
78 RestArea(aArea)
79 Return

```